

School of Computer Science

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES



UNIVERSITY OF LEEDS

**Better learning with unsupervised decorrelation and centering
learning rules**

Teo Wei Shuen

**Submitted in accordance with the requirements for the degree of
MSc Advanced Computer Science (Artificial Intelligence)**

2024/2025

The candidate confirms that the following have been submitted:

Items	Format	Recipient(s) and Date
<i>Project Report</i>	<i>Report</i>	SSO (29/8/25)
<i>Source Code</i>	<i>URL:</i> <i>https://github.com/FrenchFryKid/MSc_Project</i>	SSO (29/8/25)

Type of Project: Empirical Investigation

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.



(Signature of student)

Summary

With the constant advancement of artificial intelligence (AI), this paper aims to investigate whether the use of unsupervised decorrelation and centering learning is more efficient than traditional training methods. The project will carry out experiments to investigate the effects of unsupervised decorrelation on different gradient descent methods and determine if decorrelation has a positive or negative impact. Three gradient descent methods will be used for investigation: Standard Gradient Descent (GD), Exponentiated Gradient Descent (EG) and Exponentiated Gradient Descent with Split Bias (EGB).

The motivation stems from the fact that GD does not conform to how a human brain would work and learn. This is because GD leads to Gaussian-like weight distributions while the brain works with Log-Normal distribution (Cornford et al., 2024). Whereas EG and EGB have Log-Normal distributions and do not constantly switch between positive and negative values when updating weights. Similarly, decorrelation is inspired by biological processes in the sensory systems that are responsible for reducing redundant and overlapping features.

The research uses 3 datasets: KMNIST, MNIST and CIFAR10 to evaluate the effects of decorrelation on different gradient descent methods. An MLP was used for MNIST and KMNIST, while CNN was used for CIFAR10. The MLP and CNN models were designed with non-decorrelated and decorrelated structures.

Acknowledgements

First, I would like to thank my supervisor Dr Jonathan Cornford for his support, guidance and encouragement throughout this project. I would not have been able to complete the project to this degree of polish without his insight and input.

I am also grateful to my assessor, Dr Netta Cohen, for providing valuable insight during the assessor meeting. She pointed out critical flaws that would have had a terrible impact on the overall results and effectiveness of the study.

Lastly, I am grateful to my friends and family for their continuous love and support. Their continuous support gave me the motivation to complete this project.

Table of contents

Contents

Chapter 1 Introduction.....	1
1.1 Project Aim	1
1.2 Objectives.....	2
1.3 Deliverables.....	3
1.4 Ethical, Legal and Social issues	3
1.4.1 Ethical	3
1.4.2 Legal	4
1.4.3 Social	4
Chapter 2 Background Research	5
2.1 Literature Review.....	5
2.1.1 History of models and its relationship with the human brain	5
2.1.2 Hebbian learning	6
2.1.3 Backpropagation	6
2.1.4 Decorrelation.....	6
2.1.5 Exponentiated Gradient Descent (EG)	7
2.1.6 Motivation.....	8
2.2 Method and Techniques	9
2.2.1 Cross Entropy	9
2.2.2 Standard Gradient Descent.....	10
2.2.3 Optimizers.....	11
2.3 Choice of methods.....	16
Chapter 3 Methodology.....	17
3.1 Datasets/Data Sources	17
3.1.1 MNIST.....	17
3.1.2 CIFAR.....	17
3.2 Experimental Design	18
3.2.1 Decorrelation.....	19
3.2.2 Exponentiated Gradient Descent (EG)	22
3.2.3 Split Bias.....	23

3.2.4 Scheduler.....	23
3.2.5 Models	24
3.2.6 Fast Forward Computer Vision.....	26
3.3 High Performance Cluster	28
Chapter 4 Implementation and Validation.....	28
4.1 Loss vs Accuracy.....	28
4.2 Data.....	29
4.2.1 Dataset Preprocessing.....	29
4.2.2 Dataset Transformation.....	31
4.2.3 Dataset Loading	31
4.2.4 Result Processing	33
4.3 Results of the Empirical Investigation	33
4.3.1 Hyperparameter sweeps	33
4.3.2 Cor vs Decor MNIST	36
4.3.3 GD vs EG vs EGB MNIST	38
4.3.4 Cor vs Decor KMNIST.....	39
4.3.5 GD vs EG vs EGB KMNIST	41
4.3.6 Cor vs Decor CIFAR10.....	42
4.3.7 GD vs EG vs EGB CIFAR10	44
Chapter 5.....	45
5.1 Validation of Results.....	45
5.1.1 Covariance Matrix	45
5.1.2 Frobenius Norm	46
5.2 Result Interpretation	49
5.2.1 Hyperparameter Sweeps.....	49
5.2.2 Decorrelation for GD, EG and EGB.....	50
5.2.3 EG worse than GD	51
5.2.4 Synaptic Pruning	51
5.2.5 Multiplicative noise injection	52
Chapter 6	53
6.1 Conclusion.....	53
6.2 Objectives Achieved	53

6.3 Significance of results	54
6.4 Future Work.....	55
References	56
Appendix A	62
Datasets	62
MNIST	62
KMNIST	62
CIFAR10.....	62
Libraries	62
Pytorch	62
Torchvision	62
Numpy	63
Wandb	63
Matplotlib.....	63
Pandas	63
Python Image Library (PIL)	63
Seaborn	63
Decorrelation	63
Exponentiated Gradient	64
Appendix B	65
Appendix C	80
Appendix D	84
Notable Findings	84

Chapter 1 Introduction

1.1 Project Aim

This project is an empirical study aimed at exploring the effects of decorrelation on various gradient descent methods. Gradient descent methods are optimization algorithms used to minimize a given objective function by moving in the direction of a gradient with respect to the neural network's parameters (Goodfellow et al., 2016). Decorrelation entails removing relations between data classes in a dataset. Decorrelation removes correlations in the network's hidden layers. Multiple neural network training approaches were employed, and the methods presented in this paper were ultimately identified as the most effective for clearly assessing the impact of decorrelation across different optimization techniques. In addition, advanced strategies were utilized to determine the optimal configurations for both decorrelated and non-decorrelated training, as well as for the gradient descent methods themselves.

There are 3 different gradient descent methods that will be analyzed. Regular gradient descent (GD), Exponentiated Gradient descent (EG), and exponentiated gradient descent with split bias (EGB). Advanced strategies include hyperparameter searching, data gathering involving loss functions and accuracy calculations and dataset transformations that allow for better training resulting in higher accuracy for the model. Special loading strategies were also used to speed up training when performing hyperparameter sweeps. The 3 gradient descent equations are also explored in depth to further understand the obtained results and draw meaningful conclusions. Two main neural network models were used: MLP and CNN. Initially, MLP was used to test on a smaller dataset. Then, the experiment will continue a CNN architecture with a bigger dataset.

Early experiments with decorrelation have shown better results compared to non-decorrelated data (Ahmad, 2024). In this experiment, Ahmad set out to move beyond existing decorrelation methods and proposed a new decorrelation method instead. This decorrelation method achieves consistent decorrelation performance, independent of the current scale of the decorrelation matrix. It also maintains the overall activity magnitude within each layer during decorrelation and can be expressed as a distributed dynamic system for flexible implementation. Decorrelation is widely observed in sensory neural systems like in the retina and early visual cortex, which is where the idea of decorrelation was derived from (Ahmad, 2024).

EG and EGB tackle the biological plausibility of GD. GD is not biologically plausible because it is said to be effective specifically because GD is mathematically designed to reduce loss when the estimates of performance are restricted to just the linear estimates (Bredenberg & Savin, 2023). Additionally, GD will result in the weights of a neural network model switch signs between negative and positive due to the nature of its descent equation (Cornford et al., 2024). Lastly, GD does not initialize weights in Log-Normal distribution. This is partly because of GD switching the signs of weights, but also because GD updates are additive, not multiplicative like EG. When updates are additive, the distribution of weights will end up remaining centered and symmetric in a Gaussian like distribution (Pogodin et al., 2024). Whereas a biologically plausible fit would involve weights being distributed in a Log-Normal manner (Cornford et al., 2024).

The development of neural networks ultimately aims to create computational systems that are more human-like (Spitzer, 1999). So why use an algorithm that does not conform to biology? This is the justification for the creation and usage of EG and EGB. EG and EGB use algorithms that more closely resemble biological learning in the human brain (Cornford et al., 2024).

Now we have decorrelation and EG that both claim to improve learning in neural networks. But what if we use the two methods together? That is what this paper will set out to investigate. By using the ideal hyperparameters in a controlled setting, the effects of decorrelation on EG can be closely studied along with non-decorrelated and regular GD as the control variables.

1.2 Objectives

The objectives of this paper represent the key steps that are required to be achieved in order to properly investigate and benchmark the performance for decorrelation and EG, and compare them with GD. These objectives are set to ensure that all the proper groundwork is done in a systematic fashion. The necessary research will be done so that the best methods can be conjured to measure the performance differences. Below are the objectives set for this project:

- 1) Perform a thorough literature review on decorrelation, GD, EG, EGB and how they were derived.
- 2) Experiment with different training configurations until the best one is found. This includes finding the ideal loss function, scheduler and epoch to use for training the neural network

- 3) Develop suitable MLP and CNN architecture to run tests on. This includes developing a decorrelated counterpart which builds upon existing code that introduces decorrelated layers.
- 4) Perform hyperparameter sweeps to find the best learning rate, decorrelation learning rate and best weight decay for each of the neural network configurations. This is done on the training and validation set of the datasets
- 5) Analyze results by comparing accuracy, loss and average loss across all tested configurations and investigate whether decorrelation has a positive impact when training with EG and EGB.

1.3 Deliverables

This section describes the project's deliverables which represent the tangible outcomes expected upon the successful completion of the project. The following points detail the specific results to be provided

- 1) A GitHub repository will be provided containing the full source code created for this project. This includes the final script used for training and analysis. The repository will also include datasets used in the project.
- 2) A MSc report that presents the background research, methods employed, results obtained and an in-depth analysis on the results. The overall report will cover everything from the literature review to the final conclusions drawn from the results and also talk about the neural network used to perform tests.

1.4 Ethical, Legal and Social issues

1.4.1 Ethical

To carry out experiments and use data properly, ethical guidelines must be considered. For this section, the NeurIPS code of ethics will be referenced. The two main areas of concern regarding ethics come from the use of the CIFAR and MNIST datasets. These two datasets have been created ethically. Both MNIST and CIFAR utilize humans to label their data. NeurIPS code of ethics states that all human subjects or participants involved in the gathering of the 2 datasets must receive appropriate compensation (NeurIPS, n.d). MNIST was labeled using people mainly by high school students and the employees of National Institute of Standards and Technology (NIST). The high school students who volunteered to label the dataset were part of an educational program and the employees of NIST were compensated with salary as labelling the dataset was part of their job (LeCun et al., 1998). KMNIST was created as part of a research initiative

(Clanuwat et al., 2018). KMNIST was adapted from the National Institute of Japanese Literature, which is also a research initiative that aims to safeguard and preserve Japanese culture.

CIFAR on the other hand was manually labeled by human annotators. The participants were paid a fixed sum by the hour, which incentivized proper labelling with no rush (Krizhevsky, 2009). These practices conform with the NeurIPS code of ethics. Additionally, the experiments were done with fairness in mind, and all results were reported transparently with no manipulation involved.

1.4.2 Legal

The datasets and all the data generated from this project all comply with the General Data Protection Regulation (GDPR) guidelines (GDPR, 2018). CIFAR, MNIST and KMNIST can be obtained publicly, and are both open-source datasets. Additional legal concerns involve the use of unauthorized software, materials and libraries. All the used libraries, tools and software are all licensed and listed in Appendix A. Furthermore, all the data generated does not contain any sensitive information that infringes upon privacy rights. The intended use of this data is solely to compare the effects of decorrelation on EG and how it may or may not improve learning in neural network models.

1.4.3 Social

By using decorrelation and EG techniques which supposedly improve learning rates, it can lead to a potential reduction in bias and discrimination when making predictions. Additionally, since decorrelation and EG aim to make a neural network closer to that of a biological human's brain, future neural networks that use these new learning methods could end up opening new doors for neural networks (Ahmad, 2024; Cornford et al., 2024). These facts could ultimately end up improving accuracy and lead to more fair outcomes when using AI tools for important matters. However, it will take awhile before decorrelation makes it into Frontier AI. Frontier AI is defined as highly capable models that have the capabilities to pose severe risks to the public (Anderljung et al., 2023). Since decorrelation is still at its infant stage, it is highly unlikely that it will cause any major disruption to society in the near future.

Chapter 2 Background Research

2.1 Literature Review

This section will talk about the literature involved when designing the experiment. The section will first talk about the history of neural network models and how it relates with the human brain. The history of learning algorithms and backpropagation will be explored. Next, decorrelation will be elaborated upon, and the theory will be explained in detail. Lastly, Exponentiated Gradient Descent will be explained along with Dale's Law and how it relates to Phenomenological Matching.

2.1.1 History of models and their relationship with the human brain

The history of neural networks is intertwined with the study of the human brain. CNNs has roots stemming from both neuroscience and artificial intelligence (Lindsay, 2021). Humans used computer science to replicate the human brain into something that can be used for various scenarios without requiring a biological human to be present.

McCulloch and Pitts are credited with the initial mathematical model of a neural network. The neural network came from the concept of the nervous system and the synapses within it. An impulse must first be generated to stimulate the neurons, this impulse then triggers the neuron into a state of excitation when it exceeds a certain threshold, the impulse is then propagated to all parts of the neuron and learning starts (McCulloch & Pitts, 1943). The neurons are represented into a mathematical model that is quantifiable and thus allows researchers to manipulate it. Due to the infancy of the idea, outputs were 1 if the sum of the neurons exceeded a threshold, else 0. Additionally, the neural network was not trained using any training algorithms, all the weights were manually set.

In 1958, Rosenblatt introduced the Perceptron. Perceptron was the first algorithmic model that could learn from some dataset based on the Perceptron learning rule. Three key questions were asked:

- 1) How does a biological system sense and detect information in a physical world?
- 2) How is information stored and remembered?
- 3) How does the stored information influence recognition and behavior?

By taking biological inspiration from these 3 questions, a learning rule was created. Learning happens by changing the strength of the connections between neurons. Similar stimuli also tend to activate overlapping sets of neurons, and reinforcement can strengthen or weaken these connections (Rosenblatt, 1958). A few other factors like probability theory and generalization which derived mathematical learning curves also played a role in creating the learning rule along with Hebbian learning.

2.1.2 Hebbian learning

Hebbian learning is a synaptic mechanism that increases the strength of connections in synapses based on a form of temporal measurement. Reinforcing links that capture frequent prediction-outcome pairings and enabling predictive processing directly at the synaptic level (Hebb, 1949). In essence, Hebbian learning states that the more times something happens together, the synapses related to those occurrences will be connected more strongly together

2.1.3 Backpropagation

Prior to backpropagation, regular learning rules like the one for Perceptron worked primarily for networks with no hidden layers. One could argue that Perceptrons have pseudo hidden layers that are called feature analyzers, but they are not true hidden units due to their input connections being hardcoded (Rumelhart et al., 1986). Backpropagation works by the neural network doing the forward pass, computing the loss function, figuring out which weights are not doing their job properly and then adjusting the weights (Rumelhart et al., 1986).

2.1.4 Decorrelation

Based on the literature mentioned above, neural networks are based around the nature of human brains. However, the current existing techniques utilize gradient descent for learning, which is not a biological fit since GD is primarily geared towards minimizing the objective function (Cornford et al., 2024). This is where decorrelation comes in; the motivation for decorrelation comes from the study of the human brain. The removal of data correlations is based on the perspective of competitive and learning inhibitory control in the brain (Ahmad, 2024). However, in the past, data decorrelation was only applied to single layer networks in the context of unsupervised learning. In the study done by Ahmad, decorrelation was applied throughout the layers.

In the context of neuroscience, decorrelation is defined as the filtering that happens throughout the levels in a nervous system. Visual processing at the retinal ganglion cells have been theorized as an aid to decorrelating visual input and sending it to the brain (Pitkow & Meister, 2012). There are two types of neurons: excitatory and inhibitory. When an excitatory neuron fires, it will activate nearby inhibitory neurons. These inhibitory neurons will then inhibit other excitatory neurons from firing, preventing overlapping and redundant features (Koren et al., 2025). By reducing overlapping and redundant features, the neurons will have successfully performed decorrelation. Inhibitory plasticity, which is a term used to describe the brain's ability to adapt and modify certain synaptic connections also appears to be learned in a manner that ultimately leads to spatial decorrelation (He & Cline, 2019). Inhibitory plasticity adjusts inhibition based on postsynaptic activity. For example, if a neuron is firing too much, inhibitory inputs relating to that neuron are strengthened to dampen activity and achieve balance (Vogels et al., 2017). The act of reducing shared activity is decorrelation. Decorrelation mechanisms are based on adjusting shared activity among neurons to adjust inhibition. An example of this is when a neuron fires too much, inhibitory plasticity suppresses the fire rates of the neurons which leads to co-activation patterns between neurons being weakened, which ultimately leads to a decorrelation between the neurons.

This relates back to decorrelation in neural networks. By demeaning the data, the effect of suppressing the fire rates of neurons can be achieved and hence decorrelation. This will ultimately improve performance since the act of decorrelating the weights in the neural network will lower weights that fire too much and allow the neural network to learn more patterns to have a higher accuracy.

2.1.5 Exponentiated Gradient Descent (EG)

2.1.5.1 Dale's Law

Dale's law is a principle that originates from neuroscience. It states that a neuron releases the same set of neurotransmitters at all its synapses. Neurons can either excite or inhibit one another. When a neuron excites another, the likelihood of another neuron firing will increase. Conversely, if a neuron inhibits another, the likelihood of firing decreases. Dale's law states that a neuron either excites or inhibits and cannot switch roles during their lifetime. (Eccles, 1976).

In a study by Cornford et al. in 2024, it was found that the weights in a network trained based on gradient descent were no longer log-normal despite being initialized from random log-normal

weight distributions. Normally, brains would have a log-normal distribution in their synaptic strength (Dorkenwald et al., 2022). When weights of a model trained by gradient descent do not have log-normal distribution, it is an indicator that the model/training method could be violating biological constraints like Dale's law.

2.1.5.2 Phenomenological Match to biological learning

In the context of neural networks, weights are determined to be excitatory or inhibitory based on whether their signs are positive or negative. Gradient descent is said to not conform to biology because the update method of gradient descent can potentially allow the weights to change signs as well as weight distribution.

Unlike Gradient Descent, Exponentiated Gradient Descent (EG) does not treat all synaptic weight changes as the same. Since EG is multiplicative, it will naturally favor changing large synapses. Furthermore, due to the exponential operation found in the EG update rule, EG completely prevents weight synapses from changing signs, which also means EG prevents weight synapses from changing between inhibitory and excitatory. These properties respect Dale's law and ultimately lead to log-normal synaptic weights (Cornford et al., 2024). Since EG automatically places an emphasis on neurons that have more weight due to its exponential nature, irrelevant weights will eventually be near zero and not be active, creating soft sparsity.

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

Standard gradient descent update rule for Adam optimizer

$$\theta_t = \theta_{t-1} \cdot \exp\left(-\eta \cdot \frac{\widehat{m}_{t-1}}{\sqrt{\widehat{v}_{t-1}} + \epsilon} \cdot \text{sign}(\theta_{t-1})\right)$$

Exponential gradient descent update rule for Adam optimizer

2.1.6 Motivation

There remains a gap between decorrelation and EG. We know that the brain adjusts the rate at which synapses fire when learning, which leads to decorrelation between data gathered by synapses. If the network of neurons fires, it will inhibit the neighboring neuron network, thus reducing redundancy across the entire neuron population and achieve decorrelation (Barlow, 1961).

We also know that EG conforms to Dale's law. In the brain, neurons are classified as either excitatory or inhibitory and cannot change. Gradient descent does not conform to this since it allows synaptic weights to randomly switch signs. EG solves this by modifying the equation. Therefore, combining both decorrelation and EG would theoretically boost performance of a neural network even further. That is what the paper aims to find out. By using decorrelation and EG which are more of a biological fit, new doors for neural networks could be unlocked, with more potential for higher accuracy and less bias when predicting data.

2.2 Method and Techniques

This section will talk about the loss function, the process of gradient descent and the optimizers used. This part will detail the equations used for each method, explain how they are derived and how they are used. The section is structured based on which method comes first when training. Gradient descent uses lose function to compute weight updates while optimizers use gradient descent in each batch of data, while optimizers use gradient descent but instead calculates the moving average when training instead of just the current batch (Kingma & Ba, 2015).

2.2.1 Cross Entropy

There is multiple loss functions used when it comes to evaluating the performance of gradient descent for models. Loss function essentially consists of calculating the penalty incurred on models based on confidence value. If the model is 100% confident in the prediction of the right class, the loss is 0 and vice versa (Elharrouss et al., 2025).

Loss functions are mainly split into 2 categories, 1 for regression and 1 for classification. Popular loss functions for regression revolve around variants of Mean-Squared Error while classification revolves around Cross-Entropy. Cross-Entropy equation is as follows:

$$-\sum_{x \in \mathcal{X}} p(x) \log q(x)$$

Where $p(x)$ is a one-hot encoded label vector with 1 being the probability for the true class of that iteration and the rest being 0. $q(x)$ being the vector of predicted class probabilities output from the model. X is defined as the amount of output classes. Cross Entropy is essentially a derivation of the maximum likelihood principle (Hui, 2021).

2.2.2 Standard Gradient Descent

There are many ways to train a neural network. But neural networks that are commonly used utilize gradient descent or variants of it (Straub, 2021).

Gradient descent is used to find the minimum of a loss function and is commonly used to optimize neural networks in machine learning (Lu, 2022). Gradient Descent works by taking parameters that you want to optimize into consideration, and constantly updating them using update rules and multiple iterations to continuously update and optimize the parameters, ultimately seeking a local minimum (Khanaesar et al., 2021)

The most basic update rule is given by this:

$$\Delta w_t = -\eta g_t = -\eta \nabla_w L(w_t)$$

where η is the learning rate and, x being the parameters, y being the labels, h being the prediction function and ∇L being the partial derivative of the average of the loss function. g_t is denoted by

$$\frac{\partial L(w_t)}{\partial w_t} \in R^d$$

and represents the gradient of the parameters (Lu, 2022).

Loss function is given by:

$$L(w_t) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i; w_t), y_i)$$

where n is the amount of training data, h is the predictor function, x is the inputs, w is the weights and y is the labels. Learning rate is used to determine how big a step size for the algorithm to take in the opposite direction. This is important because a step size that is too big will make the GD bounce around multiple local minima and never reach the global minima, whereas a learning rate too small will prevent GD from exploring and finding the global minima. Gradient Descent

works best when the data is linear separable (Wan et al., 2007). Therefore, the weight update formula would be:

$$w_{t+1} = w_t + \Delta w_t$$

Gradient descent essentially involves calculating loss functions beforehand. Loss function essentially consists of calculating the penalty incurred on models based on confidence value. If the model is 100% confident in the prediction of the right class, the loss is 0 and vice versa. Cross entropy works well for classification because it heavily penalizes the model if it has high confidence in the wrong prediction, but does not penalize as much when it has low confidence in the right prediction. Backpropagation then happens, which calculates the gradients used for the optimizer to take a gradient descent step. Large loss values result in larger gradients which result in larger steps to find a better minima.

2.2.3 Optimizers

Two main optimizers come to mind when talking about neural networks, Stochastic Gradient-Descent (SGD) and Adam. SGD aims to minimize this:

$$\min_w \ell(w) := \frac{1}{n} \sum_{i=1}^n \ell_i(h(x_i; w), y_i)$$

And weight update is defined as this:

$$w_{t+1} = w_t - \eta \cdot \nabla_w \ell(h(x_i; w_t), y_i)$$

The difference here between regular gradient descent is that only 1 random pair of x_i y_i is taken from the input, as opposed to standard gradient descent which takes the entire batch of training data and their corresponding label (Tian et al., 2023). By doing this, SGD manages to achieve faster convergence at the early stages and is more efficient overall. The reasoning for this is because regular GD algorithms have redundant calculations for large scale problems since some parameters are related and thus their gradients are computed multiple times (Tian et al., 2023). The effects of SGD vs GD can be visualized in this image:

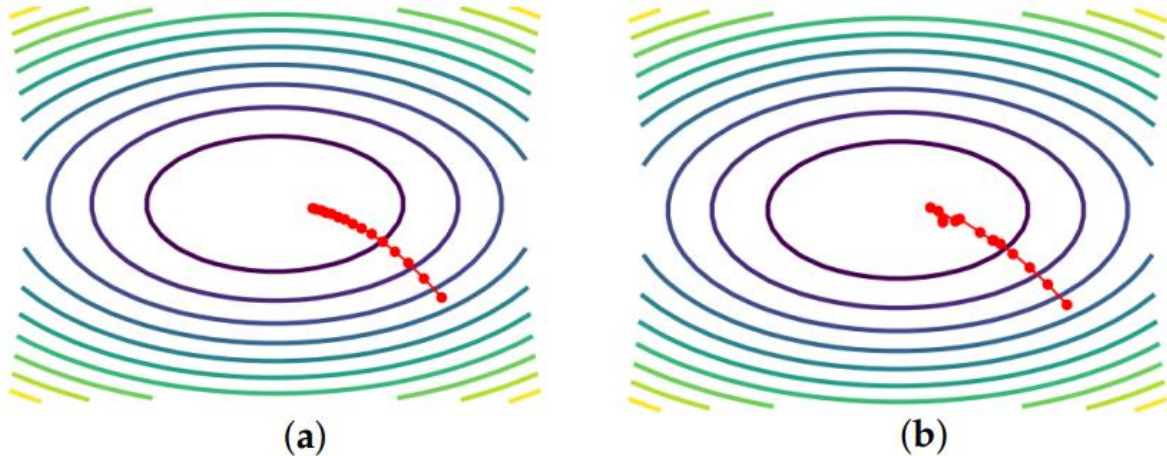


Figure 1: Comparison of Gradient Descent (a) and Stochastic Gradient Descent (b) (Tian et al., 2023)

As we can see, SGD takes larger steps and converges to the minima faster than GD. By taking only a single sample, the gradient estimate will end up being noisy, which can be a double edged, since it helps escape local minima that are shallow, and has faster initial progress, but does lead to unstable convergences (Tian et al., 2023). This results in the main problem of SGD being that it can be good but requires very fine tuning.

2.2.3.1 Adam

Adam, also known as adaptive moment estimation, is another variant of gradient descent that is as the name implies, adaptive (Kingma & Ba, 2015). Adam is based on two methods: Adagrad and RMSProp. Adagrad involves accumulating the square of past gradients and using said gradients to scale learning rate for each parameter (Duchi et al., 2011).

This makes Adagrad very good for sparse gradients, which occur when only a few elements in the gradient vector are non-zero (Kingma & Ba, 2015). RMSProp, also known as Root Mean Square Propagation is adaptive by nature and adjusts learning rate per parameter by computing moving average of squared gradients (Tieleman & Hinton, 2012).

RMSProp works well for on-line and non-stationary settings (Kingma & Ba, 2015). By combining the advantages of the two methods, Adam manages to achieve magnitudes of parameter updates that do not change when gradient is rescaled. Step sizes for ADam are approximately bounded

by the step size hyperparameter and do not require a stationary objective, thus allowing it to work well with sparse gradients and naturally has a form of step size annealing (Kingma & Ba, 2015).

Adam update parameters are split into 7 steps.

- 1) Decay first moment running average coefficient
- 2) Get gradients with respect to stochastic objective at timestep t
- 3) Update biased first moment estimate
- 4) Update biased second raw moment estimate
- 5) Compute bias-corrected first moment estimate
- 6) Compute bias-corrected second raw moment estimate
- 7) Update parameters

There are 4 parameters that can be set for Adam, which are α (learning rate), β_1 (Beta 1), β_2 (Beta 2), ϵ (Epsilon constant) and λ (Decay). Default learning rate is set to 0.001, Beta 1 to 0.9, Beta 2 to 0.999, Epsilon constant to 10^{-8} and decay to $1 - 10^{-8}$. However, in this paper, learning rate and decay will be independent variables whose impact on performance will be analyzed. The update rule for Adam is as follows:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Where $\widehat{m}_t$ is given by:

$$\widehat{m}_t = \frac{\beta_{1,t} \cdot m_{t-1} + (1 - \beta_{1,t}) \cdot g_t}{1 - \beta_1^t}$$

and $\beta_{1,t}$ and g_t are given by:

$$\begin{aligned} \beta_{1,t} &\leftarrow \beta_1 \lambda^{t-1} \\ g_t &\leftarrow \nabla_{\theta} f_t(\theta_{t-1}) \end{aligned}$$

And lastly, $\widehat{v}_t$ is given by:

$$\hat{v}_t = \frac{\beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2}{1 - \beta_2^t}$$

Equations are all based on the original Adam paper proposed by Kingma & Ba in 2015. Adam is also stochastic in nature. The stochasticity comes from the stochastic objective function f_t which takes random mini batch sub samples from data points. The moving averages are the estimate of the first moment as the mean and the second raw moment is the uncentered variance of the gradient., and these moving averages are initialized as vectors of zeroes. By factoring in the first and second moment of gradients into the update rule, it allows the learning rate to be adaptive and allows Adam to handle parameters with different scales and sparsity better. Furthermore, Adam also converges faster by incorporating momentum into gradient calculations, which is done by calculating the first moment of the gradient. Bias correction is done by dividing first moment with $1 - \beta_1^t$ and dividing second moment with $1 - \beta_2^t$.

2.2.3.2 AdamW

Despite the proposed version of Adam has B_1^t consisting of multiplying β_1 constant with decay to the power of $t-1$, the Pytorch implementation slightly differs from the proposed version. Pytorch implements L2 regularization for Adam instead of decay. This results in the β_1 update per timestep completely disappearing, and the gradient calculation is modified as so:

$$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1}) + \lambda \theta_{t-1}$$

Function f_t could be negative if maximize flag is set to true if maximize flag is set to true

Essentially shifting the “decay” to the gradient calculation. This results in $\hat{m}_t$ using β_1 constants only throughout the entire calculation and getting their “decay” from the modified gradient calculation from above (Pytorch, n.d).

$$\hat{m}_t = \frac{\beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t}{1 - \beta_1^t}$$

This in truth, is actually an L2 regularization implementation instead of a weight decay implementation. In a paper that Loshchilov & Hutter wrote in 2019, they argued that although L2 regularization and weight decay are not identical, they can be made the same by a

reparameterization trick for SGD, but this is not the case for Adam. Adam may lose out to SGD for tasks that prefer L2 regularization due to how Adam scales gradients adaptively, which undermines the effects of L2 regularization (Loshchilov & Hutter, 2019).

In SGD, L2 regularization is applied like so:

$$\begin{aligned} g_t &\leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1} \\ \eta_t &\leftarrow \text{SetScheduleMultiplier}(t) \\ m_t &\leftarrow \beta_1 m_{t-1} + \eta_t \alpha g_t \\ \theta_t &\leftarrow \theta_{t-1} - m_t \end{aligned}$$

where $\lambda \theta_{t-1}$ will pull the weights towards zero since it is applied to m_t and carries over to the update rule. But in Adam, if L2 regularization was applied in a similar fashion:

$$g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$$

And then updated via:

$$\theta \leftarrow \theta - \alpha \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

Since L2 regularization is applied at first moment and second moment $\widehat{m}_t$ and $\widehat{v}_t$, which then gets rescaled when they get divided in the update rule. This lessens the effectiveness of L2 regularization and drops performance as a whole (Loshchilov & Hutter, 2019). To counteract this, AdamW moves the decay to the update rule instead:

$$\theta_t \leftarrow \theta_{t-1} - \eta_t \left(\frac{\alpha \widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon} + \lambda \theta_{t-1} \right)$$

By applying the weight decay at the update rule, it prevents the decay from being rescaled when dividing first moment with second moment.

2.2.3.3 AdamWeg

$$\theta_t = \theta_{t-1} \cdot \exp \left(-\eta \cdot \frac{\widehat{m}_{t-1}}{\sqrt{\widehat{v}_{t-1} + \varepsilon}} \cdot \text{sign}(\theta_{t-1}) \right)$$

Where sign function is denoted as:

$$\text{sign}(x) = \begin{cases} +1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Weight decay is done like so:

$$\theta_t \leftarrow \theta_t \cdot (1 - \eta \cdot \lambda)$$

And is done right before the moment calculations. Since moment calculations do not modify the weights, the decay will not be diluted.

2.3 Choice of methods

Thus, the choice of methods are as such. The loss to be used will be AdamW as base. Then, AdamWEG will be used, one with split bias and one with no split bias. These 3 will be compared with each other. Within each of these 3 runs, decorrelation will be used for the network layers. A non-decorrelated run will also be performed for each of the 3 optimizers as control. A hyperparameter sweep will be. Learning rate, Decorrelation learning rate and decay will be the hyperparameters that will be searched.

Chapter 3 Methodology

3.1 Datasets/Data Sources

3.1.1 MNIST

The datasets used for this test were the MNIST, KMNIST, CIFAR10 and CIFAR100 datasets. MNIST, which stands for Modified National Institute of Standards and Technology, is as the name implies, a modified version of the NIST dataset. MNIST was made by modifying 2 Special Databases from NIST. Namely Special Database 1 and Special Database 3 (Lecun et al., 1998). In the Modified NIST dataset, there are 60000 train sets and 60000 test sets, but only 10000 test data which contains 5000 data from SD 1 and 5000 from SD 3 were used. In the MNIST dataset, the train and test set had nearly 30000 data from Special Database 1, and the rest were filled up with data from Special Database 3. The data are ordered according to the writers. 500 writers were split evenly between test and train and consisted of the digits 0 to 9 (Lecun et al., 1998). In MNIST, the digits are stated to be approximately uniform, hence implying that the dataset is not entirely balanced (Lecun et al., 1998). For KMNIST, it is named as the Kuzushiji MNIST. Kuzushiji means simplified script, and KMNIST is a dataset that has 10 simplified hiragana letters from the Kuzushiji-49 dataset that has 49 classes which includes the regular 48 hiragana characters and a punctuation mark. However, for KMNIST, only 10 were included which are o, ki, su, tsu, na, ha, ma, ya, re and wo hiraganas (Clanuwat, 2018). KMNIST, like MNIST, has 60000 training data and 10000 testing data. The dataset is also balanced, with the training set having 6000 data per class and testing having 1000 per class (Clanuwat, 2018). The dimensions of both KMNIST and MNIST are 28x28 and have only 1 channel since it is a black and white image.

3.1.2 CIFAR

The third is CIFAR, which stands for Canadian Institute for Advanced Research-10 class. CIFAR10 consists of airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck as the 10 classes. CIFAR was made by manually labelling a subset of 80 million images that are scraped from the net (Krizhevsky, 2009). CIFAR10 consists of colored images that are 32x32 in dimension. There are 6000 images per class, 5000 images for training and 1000 for testing per class. The images had their classes labeled by students. A few criteria also had to be followed for the images in CIFAR:

- 1) The class name should be one of the more obvious answers to the question “What is shown on this image?”
- 2) The image should be photo-realistic hence line arts were rejected
- 3) The image should contain only one prominent instance of the labeled object
- 4) The object can be partially obstructed or has an unusual angle as long as it can be clearly seen as to what the image is.

3.2 Experimental Design

An experiment was run to compare the differences between decorrelation and no decorrelation. Three datasets were used: the regular MNIST, Kuzushiji MNIST or kMNIST and CIFAR10. Three parameters were used as variables: Learning rate (1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1) , Decor Learning Rate (0, 1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1) and Weight Decays(1e-10, 1e-9, 1e-8, 1e-7, 0). This meant that a total of 4680 runs were performed for each setting. The training loop ran for 50 epochs. Within the training loop, Linear scheduler with a start factor of 1.0 and end factor of 0.1 along with Adam optimizers. Two different Adam optimizers were used, the regular AdamW optimizer with Gradient Descent and an AdamW optimizer that utilized Exponentiated Gradient. Two settings of EG AdamW were used, one with split bias and one with no split bias. This means that there are 3 optimizer configurations. 3 seeds were used: 1, 22 and 42. The final result sums up accuracies and losses across 3 seeds and divides it by 3 to find the average. This means that for 12 learning rates, 13 decorrelation learning rates, 5 decay values, 3 seeds, 3 optimizers and 3 datasets, a total of 21060 runs were performed for hyperparameter sweeping.

A few codes were used to ensure deterministic results:

- 1) `torch.manual_seed(seed)`
- 2) `torch.cuda.manual_seed(seed)`
- 3) `torch.backends.cudnn.deterministic = True`
- 4) `torch.backends.cudnn.benchmark = False`
- 5) `torch.Generator().manual_seed(seed)`
- 6) `np.random(seed)`

The first is used to ensure reproducibility for CPUs. The second is for GPUs. The third is for training algorithms to use deterministic algorithms. The fourth is to disable torch's CuDNN's auto selection of convolution algorithm based on hardware. The fifth is for generators when loading and shuffling the MNIST datasets. As for CIFAR datasets, the same scheduler was used. The CIFAR datasets were loaded with FFCV loaders, which was explained prior to this. All runs were done on the High Performance Cluster (HPC) in the University of Leeds.

Once the hyperparameter sweeps have been completed, the best parameters for each configuration were recorded down. The best parameters for these criteria are recorded down: best loss/accuracy/average loss for GD/EG/EGB for their decorrelated and non-decorrelated counterparts for 3 datasets across 3 seeds. This means that a total of 162 runs will be redone and used for plotting graphs to draw comparisons. A few notable findings regarding the effects of certain techniques like gradient clipping and different optimizers were found. The findings are documented in Appendix D

3.2.1 Decorrelation

Decorrelation is performed for Linear layers and Convolution layers. Decorrelation is performed at each of the layers when the inputs are initially parsed into it.

In the linear layers, the input is first demeaned. The running mean is first calculated like so:

$$\text{running_mean} \leftarrow (1 - \text{mean_momentum}) \cdot \text{running_mean} + \text{mean_momentum} \cdot \text{mean}$$

Regular mean is calculated using torch.mean. The torch.mean command returns the mean value of each row in the input tensor based on the given dimension (Pytorch, n.d). After that, the mean is subtracted from the input.

Then the demeaned input is einstein summated with the current weights of the model.

$$\text{output}_{n,j,\dots} = \sum_{i=1}^C \text{demeaned_input}_{n,i,\dots} \cdot \text{decor_weight}_{i,j}$$

Einstein summation consists of matrix multiplication over channel dimension for every sample and spatial location. In this step, we sum over i input features and multiply each input feature with the corresponding weight in decor_weight.

Next, data is subsampled. Samples% is a constant of 0.1 in this case.

$$N_{\text{samples}} = (\text{samples \%} \times \text{InputLength}) + 1$$

A 1 is added to ensure that the output is never 0. The output of the percentage of samples multiplied with input length is rounded down. The data is then subsampled like so:

$$\text{undecor_state} = \text{input}[:n_{\text{samples}}]$$

$$\text{decor_state} = \text{output}[:n_{\text{samples}}]$$

After subsamples have been calculated, the data is reshaped if there are multiple channels. A common CNN tensor format consists of batch, channels, height, width (4 channels). The data is first permuted into batch, height, width, channels. After that, the data is flattened into one dimension. Each row will now have one spatial location in the batch and each column is one feature channel.

Correlation matrix is computed like so:

$$C = \frac{1}{N} D^T D$$

Where D is the decor_state from above and N is the length of D

Next, Normalization is computed

$$\text{normalization} = \text{mean} \left(\frac{\sqrt{\sum(\text{undecor_state}^2)_{\text{across features}}}}{\sqrt{\sum(\text{decor_state}^2)_{\text{across features}} + \epsilon}} \right)$$

Where epsilon is 1e-8.

Lastly, the decorrelation weights are updated

$$w \leftarrow w \cdot \text{normalization}$$

$$g \leftarrow Cw$$

$$w \leftarrow w - \eta g$$

Where w is the weights of the decorrelation network, g is the gradients of the decorrelation network and η is the learning rate (Ahmad,2024).

For convolution layers, it is somewhat similar to Linear layers. Like before, input is first demeaned by using the same calculations. However, this time, the output variable is not initialized by using Einstein summation. Instead, a Convolutional layer is initialized as the output variable. Same as before, data is subsampled using 0.1 as the sample percentage.

Next, data undergoes a process called “patchifying”. This process involves unfolding the data and then folding it back in a different way (Ahmad, 2024). It involves 4 steps. First, the undecor_state is unfolded. The data then goes from [batch, channels, H, W] to [batch, channels * kH * kW, num_patches], where num_patches come from how many sliding windows can fit into the input given kernel size, stride and padding. Next, output size x and output size y are computed:

$$\text{output_size_x} = \frac{\text{dShape} - k_h + 2p_h}{s_h}$$

$$\text{output_size_y} = \frac{\text{dShape} - k_w + 2p_w}{s_w}$$

Where dShape is the shape of the demeaned input, k is the kernel, p is the padding and s is the stride. After that, undecor_state is folded back based on output size x and y with a kernel size of 1x1.

Next, the decor_state is then set up in such a way as if they are batch-samples. First, the decor_state is reshaped into this order: [num_samples, mid_dim, ...]. The dimensions are then ordered from [batch, channels, number of spatial positions] to [batch, number of spatial positions, channels]. Lastly, the decor_state's batch and spatial positions are collapsed into a single dimension. This results in each row being a feature vector for one patch position. This results in a modified decor state

Normalization is computed in the same manner as before. The correlation matrix is computed slightly differently this time:

$$C = \frac{1}{N} mD^T mD$$

Where m is the modified decor state from before. Decorrelation matrix is updated in a slightly different way:

$$w \leftarrow w \cdot \text{normalization}$$

$$g \leftarrow Cw.\text{reshape}$$

Here, decorrelation weights are reshaped into a 2D square matrix before being multiplied with the correlation matrix obtained from modified decorrelation state.

Weights are finally updated like so:

$$w \leftarrow w - \eta g.\text{reshape}$$

Where the gradient is finally reshaped back from 2D to 4D. Only a sample of 10% is used to calculate the decorrelation and correlation matrix. This is to save computational power. By using 10% of the samples to calculate, the matrices will still be statistically accurate enough to be used for updating weights.

3.2.2 Exponentiated Gradient Descent (EG)

Exponentiated Gradient is an alternative to regular Gradient Descent; By introducing biological concepts into the update rule, it is a phenomenological fit. Furthermore, EG aims to improve the accuracy of regular GD by handling irrelevant features differently.

The key difference between Exponentiated Gradient (EG) and GD lies in the fact that GD's update rule works additively with the squared error of the gradient, whereas EG's update rule works multiplicatively using the exponents of the gradients (Kivinen & Warmuth, 1997).

The benefits of EG stem from the exponential nature of its update rule. EG's benefits are that relevant weights will keep getting more and more relevant, whereas irrelevant weights will decay off and not be used, as opposed to GD where weights are updated additively, leading to a more blurry line between high impact and low impact weights (Kivinen & Warmuth, 1997).

There is indeed research to back this up. In an experiment done by Cornford et al. in 2024, synaptic pruning experiments were run using GD and EG. The results showed that EG relies on fewer synaptic weights to find solutions as opposed to GD. Furthermore, the paper also showed that weights trained by EG are easier to prune and easier to retrain after pruning. Lastly, the paper also found that EG is better at ignoring irrelevant features (Cornford et al., 2024). This conforms to the idea mentioned above where useful weights update exponentially and irrelevant weights decay off (Kivinen & Warmuth, 1997).

3.2.3 Split Bias

$$\begin{aligned} m_t^+ &= \beta_1 m_{t-1}^+ + (1 - \beta_1) g_t^+ \\ v_t^+ &= \beta_2 v_{t-1}^+ + (1 - \beta_2) (g_t^+)^2 \\ m_t^- &= \beta_1 m_{t-1}^- + (1 - \beta_1) g_t^- \\ v_t^- &= \beta_2 v_{t-1}^- + (1 - \beta_2) (g_t^-)^2 \end{aligned}$$

In split bias, biases in the moment calculation are split into positive and negative. By doing this, the positive bias can increase and the negative one can shrink over time, which can promote a more effective bias. Split bias is used to promote cleaner weight distribution. When used in conjunction with AdamWEG optimizer, split bias helps induce a more log-normal-like weight and bias distributions (Cornford et al., 2024). By doing so, split bias is said to be designed in such a way that mimics excitatory/inhibitory separation.

3.2.4 Scheduler

The scheduler used for this experiment is the LinearLR scheduler. A LinearLR scheduler takes in the optimizer and decays the learning rate based on the factor that was inputted (Pytorch, n.d). The graph would look something like this:

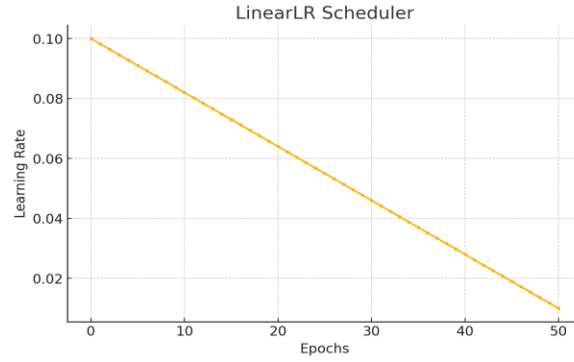


Figure 2: LinearLR Scheduler learning rate decay graph

LinearLR scheduler was chosen because it smoothly reduces the step size. Larger steps are used initially to explore the loss landscape, and as training goes on, learning rate is decayed so the model can fine-tune. LinearLR was also chosen for its simplicity and predictability.

3.2.5 Models

Two models were used, a multilayer perceptron with decorrelation and a multilayer perceptron with no decorrelation. The model structure are as follows:

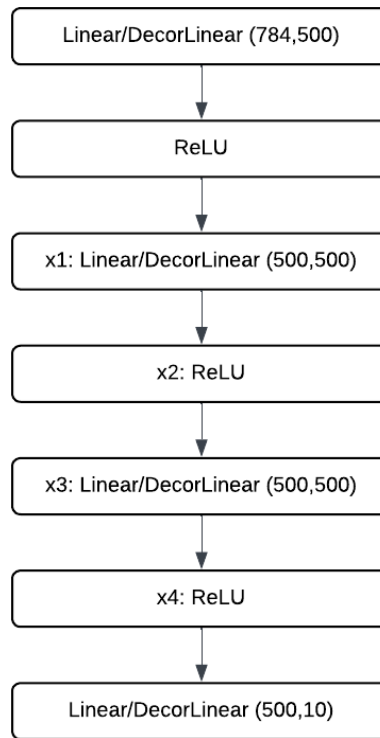


Figure 3: MLP Structure with decorrelated/non-decorrelated layers

For the non-decorrelated model, Linear layer will be used. For decorrelated model, DecorLinear will be used. The labels x1, x2, x3, x4 will be the layers where the covariance matrix is calculated.

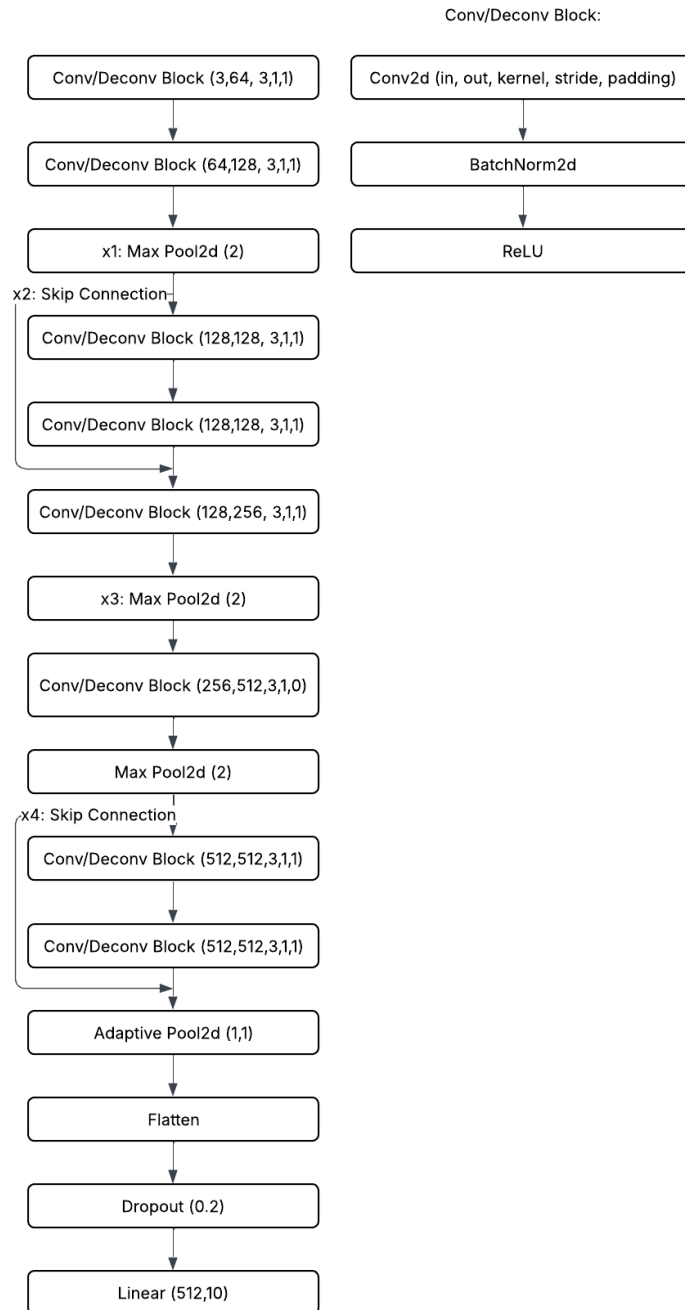


Figure 4: CNN Structure with decorrelated/non-decorrelated layers

The image above show the models used for CIFAR 10 and 100. There are skip connections between certain layers which are shown in the picture. The model is based on a paper done by He et al in 2015. It follows the ResNet9 architecture.

3.2.6 Fast Forward Computer Vision

Fast Forward Computer Vision (FFCV) is a loader that allows neural networks to dramatically speed up training. FFCV uses multiple tricks to increase speed. The first are .beton files. Beton files work by storing datasets into a compact format so that it is easier and faster for disks to read. Furthermore, .beton files are memory mappable and removes the need for constantly re-reading the disk unnecessarily (Leclerc et al., 2023). FFCV also utilizes Just-In-Time transforms that are compiled using Numba, which are much faster as opposed to using regular Python transformations. Asynchronous Data Loading also affects speed. Data is loaded by multiple CPU threads, and when the GPU is training one batch, the next batch is being loaded and processed simultaneously. Finally, batches are prefetched and transferred to GPU before training happens, this prevents idle time for GPU (Leclerc et al., 2023).

3.2.6.1 Issues

A few issues arose when trying to use FFCV. The first thing is that this error message breaks the entire code despite it only being a warning. This error message happens when performing data augmentation in the loader.

```
if name == 'train':
    image_pipeline.extend([
        RandomHorizontalFlip(),
        RandomTranslate(padding=2),
        Cutout(8, tuple(map(int, mean))), # Note Cutout is done before normalization.
    ])
image_pipeline.extend([
    ToTensor(),
    #ToDevice('cuda:0', non_blocking=True),
    ToTorchImage(),
    Convert(torch.float32),
    torchvision.transforms.Normalize(mean, std)
])
```

Figure 5: Picture of FFCV transform codes

When trying to perform data augmentation, the code runs a check on the next_state device's type and checks if it is a CUDA type. However, this error happens:

```

AttributeError                                Traceback (most recent call last)
Cell In[8], line 48
    39     image_pipeline.extend([
    40         ToTensor(),
    41         ToDevice('cuda:0', non_blocking=True),
    (...)  44         torchvision.transforms.Normalize(mean, std)
    45     ])
    47     # Create loaders
--> 48     loaders[name] = Loader(f'./tmp/cifar_{name}_42.beton',
    49                           batch_size=BATCH_SIZE,
    50                           num_workers=8,
    51                           order=OrderOption.RANDOM,
    52                           seed=seed,
    53                           drop_last=(name == 'train'),
    54                           pipelines={'image': image_pipeline, 'label': label_pipeline},
    55                           ) # \ Required for GPU transforms
    57     print(loaders['val'])

File c:\Users\teowe\AppData\Local\Programs\Python\Python311\Lib\site-packages\ffcv\loader\loader.
 204         self.pipeline_specs[field_name] = spec
 206     self.graph = Graph(self.pipeline_specs, self.reader.handlers,
 207                       self.field_name_to_f_ix, self.reader.metadata,
 208                       memory_read)
--> 210 self.generate_code()
...
 335         "detrimental to the performance, consider moving the augmentation"
 336         "on the GPU or using an FFCV native transform")
 337     warnings.warn(msg, ResourceWarning)

AttributeError: 'str' object has no attribute 'type'

```

Figure 6: Error message caused by data augmentation

The reason is that the `next_state` type in either the loader or the dataset somehow becomes a `str` object, and it causes the trigger condition for the warning to instead cause a runtime error. To circumvent this, the chunk of code containing the `if` statement had to be commented out.

```

if next_state.device.type != 'cuda' and isinstance(operation, ModuleWrapper):
    msg = ("Using a pytorch transform on the CPU is extremely"
          "detrimental to the performance, consider moving the augmentation"
          "on the GPU or using an FFCV native transform")
    warnings.warn(msg, ResourceWarning)

```

Figure 7: Image of FFCV library code that had to be removed

The chunk of code triggers a warning telling users to switch to GPU. However, the check itself is flawed and will instead cause everything to crash in this scenario

3.3 High Performance Cluster

All experiments were done on the High Performance Cluster in the University of Leeds. The platform used was the Aire platform, which contains 84 Nvidia L40S GPUs and 9072 Cores (Advanced Research Computing, University of Leeds, n.d). This allows for multiple configurations to be run concurrently. Additionally, the L40S is a powerful GPU which makes each run faster, thus further shortening the total time taken for all experiments to run.

Chapter 4 Implementation and Validation

4.1 Loss vs Accuracy

Loss and accuracy are the 2 most used metrics when benchmarking the performance of a neural network (Terven et al., 2025). These 2 metrics fall under performance metrics. However, what exactly do these values mean? And what are the conclusions that can be drawn from them?

Loss function measures how confident the model is in its prediction. The loss function is inherently an objective function that the neural network aims to minimize and acts like a guide for the neural network to know how well it is doing and not descend towards wrong directions (Terven et al., 2025).

Loss varies from scenario to scenario. Unlike accuracy, when presented with a loss value with no prior context, it is difficult to draw conclusions. This is because loss values do not conform to a fixed scale like 0 to 100% and oftentimes their scale changes based on the task at hand (Terven et al., 2025). However, when analyzing loss continuously throughout the training epochs within a fixed scenario and across same datasets, it can tell us how well the model is learning as training goes on. A sharper decrease in loss would mean that the model learns faster. In the case of this experiment, the scale of losses will be compared within the same model with the same configurations and the same dataset, but only with slight variations with the same parameters. This makes it so that the scenario is effectively still the same, which means that the scale of losses can be compared within each scenario. Average loss will also be used as the metric to determine whether decorrelation alongside EG and split bias helps the model learn better. Along with final loss at the end of training, the experiment will also look at average loss throughout the training. Average loss throughout the training will tell us how fast the model is learning as training continues.

Accuracy, on the other hand, is simple and intuitive. It follows a fixed scale of 0 to 100% accuracy universally. Accuracy defines how many times a model predicts the class correctly (Terven et al., 2025). It tells us how often the model is correct. However, there are some concerns regarding accuracy, mainly in regard to class imbalance. Assume that a class has data where 95% of the data is class A, and 5% is from class B. If the model were to just predict class A, it would have an accuracy of 95% regardless (Terven et al., 2025). However, this is not the case with MNIST, KMNIST and CIFAR. All the classes are balanced; hence accuracy can be used as a good measure of performance.

4.2 Data

This section will talk about how the datasets were processed, loaded and transformed. This section will also go into detail on how the results were analyzed after the raw data was obtained.

4.2.1 Dataset Preprocessing

Although most datasets only provide training and testing data, it is best practice to further split the training data into training and validation, resulting in a total of 3 splits. Training data would be split from 60000 to 50000, 10000 for MNIST and KMNIST. For CIFAR, the training data is split from 50000 to 40000, 10000 training and validation respectively, while testing data remains the same. Typically, the third validation set is split in order to assess the model and select the best parameters, the final test set is only used once to assess the model after the hyperparameter sweep has been done (Hastie et al., 2001). To achieve this split, random split from Pytorch's library was used. Pytorch was selected so that a seed can be assigned to the split and thus making the split predictable and replicable.

Additionally for CIFAR datasets, a special technique involving FFCV loader was utilized. To use FFCV loaders efficiently, a .beton file had to be generated for each dataset. This .beton file allows FFCV to skip steps that would otherwise take a long time for standard methods to load and therefore cuts down on training time. The code is as follows:

```

if __name__ == "__main__":
    seeds = [1,22,42]
    for seed in seeds:
        train_len = 40000
        val_len = 10000
        train_set = torchvision.datasets.CIFAR10('./cifar10', train=True, download=False)
        train_set, val_set = random_split(train_set, [train_len, val_len], generator=torch.Generator().manual_seed(seed))
        test_set = torchvision.datasets.CIFAR10('./cifar10', train=False, download=False)

        train_data = np.stack([np.array(img) for img, _ in train_set]) # shape: (40000, 32, 32, 3)

        # Compute mean and std per channel
        train_mean = np.array(train_data.mean(axis=(0, 1, 2))) # shape: (3,)
        train_std = np.array(train_data.std(axis=(0, 1, 2))) # shape: (3,)

        print("train_mean:", train_mean)
        print("train_std:", train_std)
        print(len(train_mean))

        val_data = np.stack([np.array(img) for img, _ in val_set]) # shape: (10000, 32, 32, 3)

        # Compute mean and std per channel
        val_mean = np.array(val_data.mean(axis=(0, 1, 2))) # shape: (3,)
        val_std = np.array(val_data.std(axis=(0, 1, 2))) # shape: (3,)

        print("val_mean:", val_mean)
        print("val_std:", val_std)
        print(len(val_mean))

        test_data = np.stack([np.array(img) for img, _ in test_set]) # shape: (10000, 32, 32, 3)

        # Compute mean and std per channel
        test_mean = np.array(test_data.mean(axis=(0, 1, 2))) # shape: (3,)
        test_std = np.array(test_data.std(axis=(0, 1, 2))) # shape: (3,)

        print("test_mean:", test_mean)
        print("test_std:", test_std)
        print(len(test_mean))

        wrapped_train = FFCVCIFAR10Wrapper(train_set)
        wrapped_test = FFCVCIFAR10Wrapper(test_set)
        wrapped_val = FFCVCIFAR10Wrapper(val_set)

```

Figure 8: Data preprocessing showing the data split

```

writer_train = DatasetWriter(f'./beton/cifar_train_{seed}.beton', {
    'image': RGBImageField(),
    'label': IntField()
})

writer_val = DatasetWriter(f'./beton/cifar_val_{seed}.beton', {
    'image': RGBImageField(),
    'label': IntField()
})

writer_test = DatasetWriter(f'./beton/cifar_test_{seed}.beton', {
    'image': RGBImageField(),
    'label': IntField()
})

writer_train.from_indexed_dataset(train_set)
writer_val.from_indexed_dataset(val_set)
writer_test.from_indexed_dataset(test_set)

```

```

class FFCVCIFAR10Wrapper:
    def __init__(self, dataset):
        self.dataset = dataset

    def __getitem__(self, idx):
        img, label = self.dataset[idx]
        return np.array(img), label # Convert PIL → NumPy

    def __len__(self):
        return len(self.dataset)

```

Figure 9: Image showing how the data is written into a .beton file

Figure 10: Image showing the FFCVCIFAR10 Wrapper

3 main things happen in the script that converts the raw CIFAR10 files into the mentioned .beton files. The first is that the script splits the training data in CIFAR10 into training and validation at a 40000, 10000 split. The second is that the script calculates the mean and standard deviation for

the specific training/validation/testing data. This is important when applying transformation to the CIFAR dataset later for training. Lastly, the creation of the .beton file happens. The resulting outcome is 3 .beton files ready for the FFCV loader to use.

4.2.2 Dataset Transformation

For the MNIST and KMNIST dataset, the only transformations applied were to transform the dataset into a tensor for Pytorch model to use, this is due to the fact that a regular MLP can already predict MNIST and KMNIST very well, so no additional transformation techniques were applied to potentially boost the final accuracy of the models. CIFAR on the other hand had more thought put into it since CIFAR is a more complex dataset that responds well to advanced transformation techniques. The training datasets in CIFAR10 were subjected to 5 transformation techniques. The first is Random Horizontal Flip. Random Horizontal Flip subjects all images in the dataset to a 50% chance to be flipped horizontally. The next is Random Translate. Random Translate randomly shifts the image horizontally and/or vertically by a small amount. A padding of 2 was applied to the image on all sides. This eliminates the possibility of the image being shifted into a position where it eats into pixels that do not exist. The third transformation is called a Cutout transformation. A Cutout transformation involves cutting out a specific area of the image, and then filling the cutout area with some value. In the experiment, an 8x8 area of all images in the train set was cut out, and the missing area was filled with a tuple of the mean value calculated from before. This results in the missing area being more like a smudge instead of a literal black void. The fourth transformation involves Normalizing the dataset. The equation is given as such:

$$output[c] = \frac{input[c] - mean[c]}{std[c]}$$

Where c is defined as the color for each channel (R,G,B) and mean and std are the mean and standard deviation values calculated from before. By doing this, the image is centered around 0 to remove brightness bias and scales the values so that they have a variance of approximately 1 which makes optimization more stable. Lastly, the dataset was converted to Tensor for Pytorch to use. For validation and testing datasets, only Normalization and conversion to Tensor were applied as transformations.

4.2.3 Dataset Loading

For KMNIST and MNIST, the data was loaded using Pytorch's DataLoader.

```
train_loader = torch.utils.data.DataLoader(
    train_data,
    batch_size=32,
    shuffle=True,
    num_workers=0,
    generator=g
)
```

Figure 11: Image showing a dataloader from Pytorch

A batch size of 32 was used for all data in MNIST and KMNIST regardless of training, validation or testing sets. Batch size represents the amount of data that can be processed in each iteration of training. By grouping data into batches, it allows the GPU to process multiple data at once as opposed to having the GPU process every single data in an unbatched scenario (Paszke et al., 2019). This reduces the amount of times a training loop will iterate in one epoch. Additionally, gradients are also averaged over the batch, which reduces noise compared to unbatched data. In this experiment, `num_workers` was set to 0. The number of workers dictate how many worker processes PyTorch will load in order to preprocess data in parallel, and 0 number of workers means the data will be processed sequentially. However, since the experiment was run in the HPC, loading the data sequentially prevents potential issues from happening. Furthermore, since KMNIST and MNIST is a relatively simple dataset, the performance impact is not that big.

For CIFAR, the loader looks like this:

```
loaders[name] = Loader(f'./beton/cifar_{name}_{seed}.beton',
    batch_size=512,
    num_workers=8,
    order=OrderOption.RANDOM,
    seed=42,
    drop_last=(name == 'train'),
    pipelines={'image': image_pipeline, 'label': label_pipeline},
)
```

Figure 12: Image showing FFCV's data loader

In this case, a bigger batch size of 512 was used. This is to maximize the performance gained by using FFCV since FFCV allows for GPU to be used more efficiently. 8 workers were used here to further push performance. `OrderOption.RANDOM` means that the data will be loaded randomly, essentially achieving a data shuffle. The randomness of this was controlled using the `seed` parameter. Pipelines dictate the transformations used which were mentioned above.

4.2.4 Result Processing

After each training run, the final statistics are saved into a .pt file for later review. The specific statistics saved are the final training loss, training accuracy, validation loss and validation accuracy. The .pt files are saved in a very specific naming format. The format is as so:

`{cor/decor}-{dataset}{learning rate},{decor learning rate},{decay},{seed}.pt`. By saving it in this specific naming convention, it allows for easy analysis and data compilation. Next, the data for each configuration is summed up across the 3 seeds and the average of each statistic is taken. This is to prevent bias. After the average for all config has been created, the script runs through all the processed average data to find the best configurations for decorrelation and without decorrelation for GD, EG and EGB. In the meantime, heatmaps showing the distribution for accuracy, loss and average loss across all configurations are also generated. Once the best configuration for each data has been found, the experiment with the best configuration for each data is run again but this time, instead of using a validation set, the test set will be used. Graphs are then plotted to draw comparisons and conclusions are drawn. The percentage difference between each gradient descent method is also calculated. The way this is calculated is by comparing the 3 gradient descent methods with the best gradient descent method for that current metric. This means that if loss is being compared for GD, EG and EGB, and GD ends up having the lowest loss, the percentage difference mentioned will be for EG vs GD and EGB vs GD. Additionally, it is important to mention that when comparing metrics, the higher the accuracy, the better. While for loss, the lower the loss, the better. Accuracy is scaled on a scale of 0 to 100%, while loss, as previously mentioned, is problem dependent and generally has no fixed scale. This means that losses between different datasets should not be directly compared, thus this report will only draw conclusions based on losses within each dataset measured by the 3 different gradient descent algorithms.

4.3 Results of the Empirical Investigation

4.3.1 Hyperparameter sweeps

The results of all hyperparameter sweeps are listed in Appendix B. Going forward, all the learning rate, decorrelation learning rate and weight decay listed in the graphs below are the best hyperparameters found for the specific decorrelation/gradient descent setup. This ensures the fairest comparison between all different setups by comparing each of the setups at their peak. By performing experiments in this nature, the larger picture can be better visualized as opposed to

comparing everything using a fixed learning and decay rate. The best hyperparameters found for GD, EG and EGB in terms of accuracy, loss and total average loss as a metric are listed as such:

Learning Rate	Decorrelation Learning Rate	Weight Decay	Method	Dataset	Metric	Final Value
1e-04	1e-10	0	GD	MNIST	Accuracy	98.33%
1e-03	1e-07	0	EG	MNIST	Accuracy	98.17%
1e-03	1e-08	0	EGB	MNSIT	Accuracy	98.07%
1e-05	1e-10	0	GD	MNIST	Loss	0.0650
1e-03	1e-10	0	EG	MNIST	Loss	0.0953
1e-03	1e-08	0	EGB	MNIST	Loss	0.0999
1e-04	1e-05	0	GD	MNIST	Average Loss	0.0762
1e-03	1e-10	0	EG	MNIST	Average Loss	0.0859
1e-03	1e-08	0	EGB	MNIST	Average Loss	0.0858

Table 1: Summary of best parameters MNIST

Learning Rate	Decorrelation Learning Rate	Weight Decay	Method	Dataset	Metric	Final Value
1e-04	1e-06	0	GD	KMNIST	Accuracy	91.87%
1e-03	1e-11	0	EG	KMNIST	Accuracy	91.13%
1e-03	1e-10	0	EGB	KMNIST	Accuracy	91.13%
1e-05	1e-06	0	GD	KMNIST	Loss	0.3550
1e-03	1e-06	0	EG	KMNIST	Loss	0.5334

1e-03	1e-06	0	EGB	KMNIST	Loss	0.5478
1e-04	1e-05	0	GD	KMNIST	Average Loss	0.4440
1e-03	1e-09	0	EG	KMNIST	Average Loss	0.4889
1e-03	1e-11	0	EGB	KMNIST	Average Loss	0.4963

Table 2: Summary of best parameters for KMNIST

Learning Rate	Decorrelation Learning Rate	Weight Decay	Method	Dataset	Metric	Final Value
1e-02	1e-03	1e-07	GD	CIFAR10	Accuracy	89.17%
1e-01	1e-09	1e-08	EG	CIFAR10	Accuracy	87.83%
1e-01	1e-03	1e-09	EGB	CIFAR10	Accuracy	88.13%
1e-02	1e-03	1e-07	GD	CIFAR10	Loss	0.7807
1e-01	1	1e-10	EG	CIFAR10	Loss	0.8022
1e-01	1e-03	1e-09	EGB	CIFAR10	Loss	0.8048
1e-02	1e-08	1e-08	GD	CIFAR10	Average Loss	0.9416
1e-01	1e-06	1e-07	EG	CIFAR10	Average Loss	0.9124
1e-01	1e-06	1e-07	EGB	CIFAR10	Average Loss	0.9170

Table 3: Summary of best parameters CIFAR10

The three tables show the best hyperparameters for the specific gradient descent method along with the metric and the dataset that the model was trained on. All the learning rate, decorrelation learning rate and weight decay are shown in scientific notation. From this table, it can be seen that all the best hyperparameter configurations for MNIST and KMNIST do not contain weight decay. Additionally, the best learning rate for MNIST and KMNIST seem to float between 1e-03

to $1e-05$. Most configurations prefer either $1e-03$ or $1e-04$, but some configs prefer $1e-05$. There does not seem to be any trends available for decorrelation learning rate in MNIST and KMNIST as a wide variation of decorrelation learning rate was employed as the best hyperparameter based on the metrics measured. For CIFAR, the consensus is that the best learning rate floats between $1e-02$ and $1e-01$. Decorrelation learning rate again, is all over the place, with values ranging from $1e-09$ to 1. Additionally, CIFAR10 with ResNet9 seems to prefer weight decay, with all configurations containing weight decay.

4.3.2 Cor vs Decor MNIST

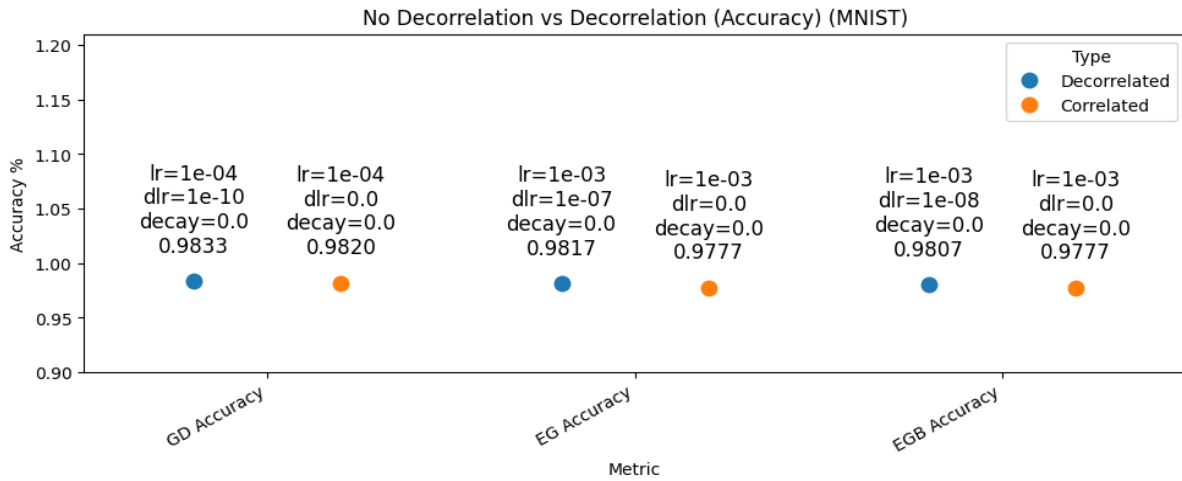


Figure 13: No Decorrelation vs Decorrelation for MNIST with Accuracy as the metric across GD, EG, EGB

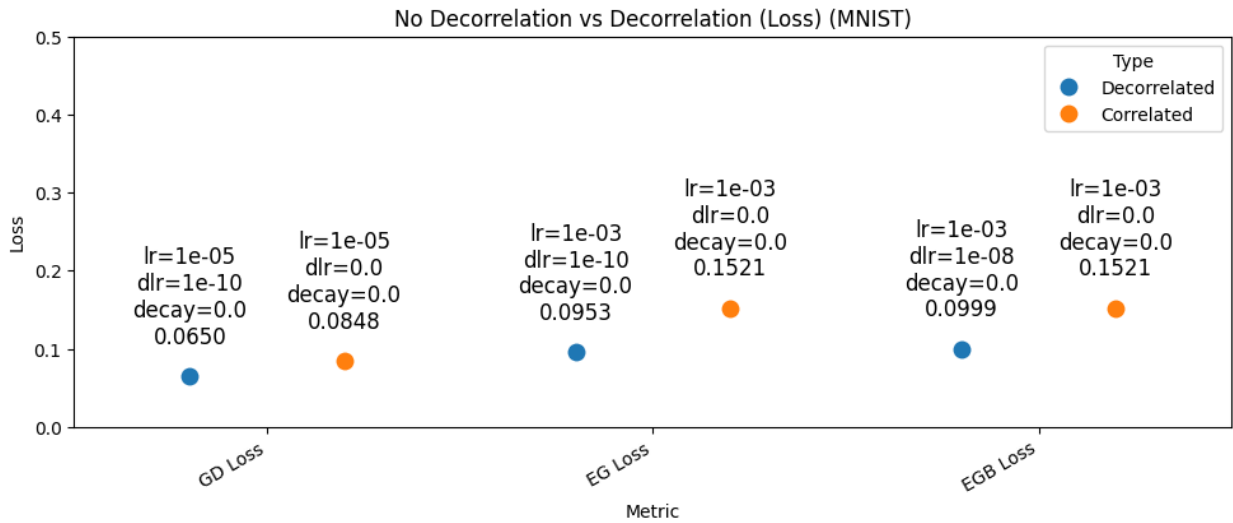


Figure 14: No Decorrelation vs Decorrelation for MNIST with Loss as the metric across GD, EG, EGB

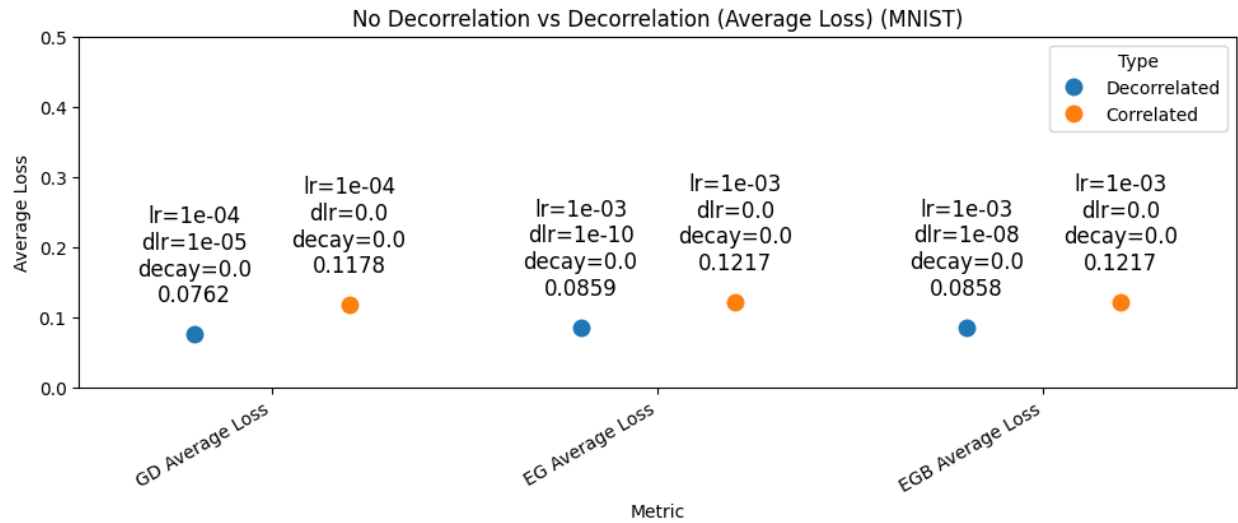


Figure 15: No Decorrelation vs Decorrelation for MNIST with Average Loss as the metric across GD, EG, EGB

The graph above visualizes accuracy, loss and average loss for MNIST. In this graph, the difference between no decorrelation and decorrelation can be clearly seen. A few interesting conclusions can be drawn from the graphs. The first is that weight decay does absolutely nothing for performance. None of the best runs have weight decay. The second is the same observation which can be drawn from the graph before this. EG is worse than GD and EGB is the worst. However, an interesting phenomenon is that the data between EG and EGB for no decorrelation is the same across all 3 fields.

In the accuracy department, the difference between decorrelation and no decorrelation is approximately 0.1% for GD and 0.4% for both EG and EGB. For loss, the difference is approximately 30% for GD, 60% for EG and 52% for EGB. For average loss, the difference is approximately 55% for GD, 42% for EG and 42% for EGB. All the results here are in favor of decorrelation

4.3.3 GD vs EG vs EGB MNIST

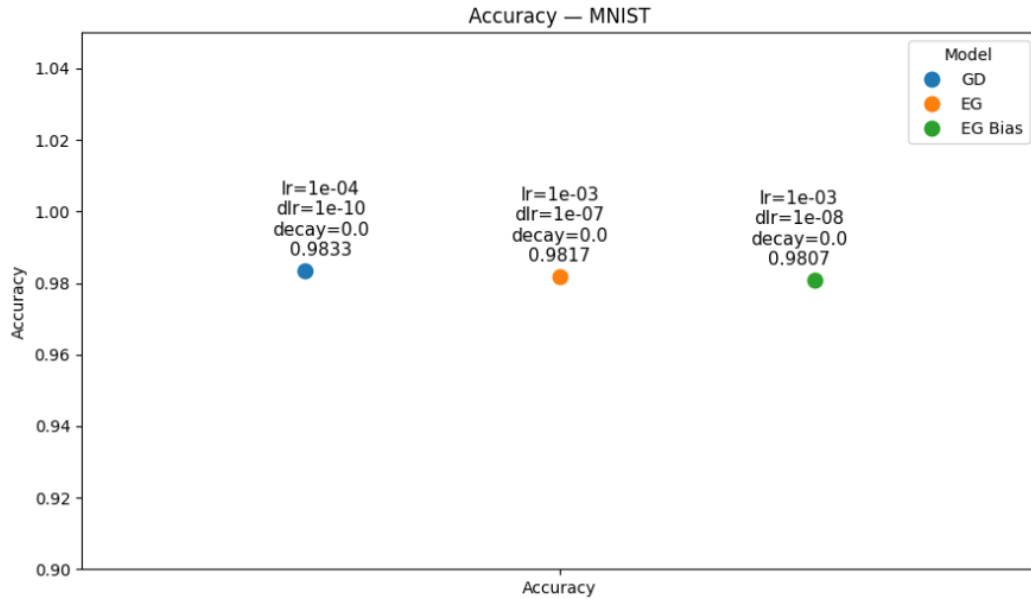


Figure 16: GD vs EG vs EGB accuracy comparison for MNIST

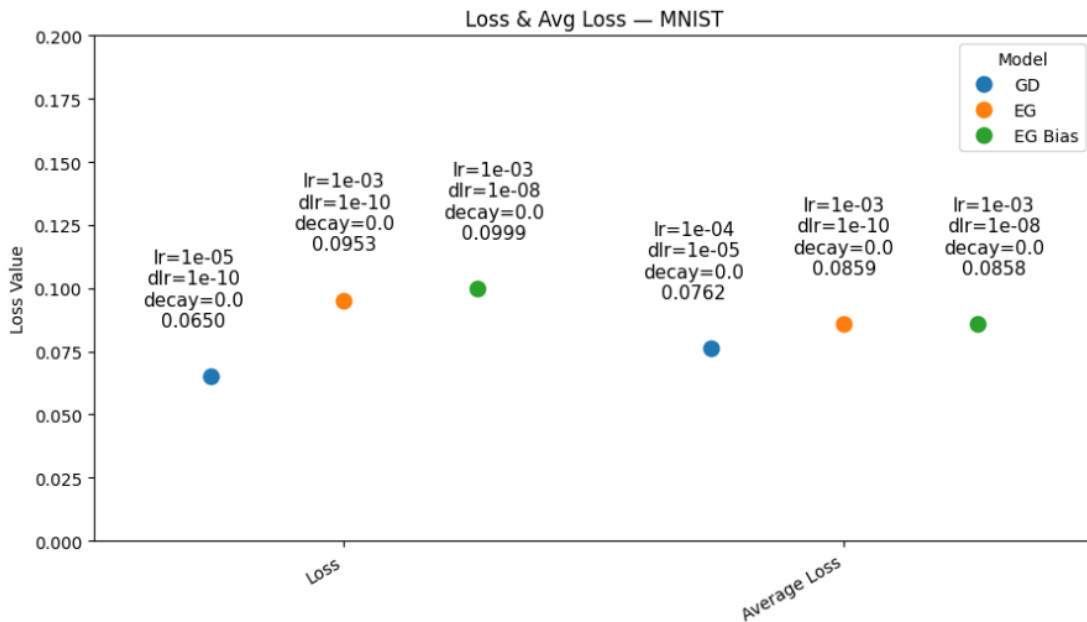


Figure 17: GD vs EG vs EGB loss and average loss comparison for MNIST

The results above show the best statistics for the specific run. The text label above the point shows the best learning rate, decorrelation learning rate and weight decay. The graph shows the best achieved accuracy, loss and total loss for the 3 configurations. The results show that overall,

EG and EG with bias have slightly worse performance than just running a regular AdamW optimizer. The results do however prove that decorrelation does indeed have a better performance than correlation across all 3 statistics. Additionally, it can be seen that for all the best runs in each configuration, decorrelation learning rate is higher than 0. This means that all the best runs for each config had decorrelation which means that decorrelation beats no decorrelation overall.

The percentage difference between gradient descent methods of MNIST will be detailed here. In terms of accuracy, EG performed approximately 0.2% worse than GD and EGB performed approximately 0.3% worse than GD. In terms of loss, EG performed approximately 47% worse than GD and EGB performed approximately 54% worse than GD. For average loss, EG performed approximately 13% worse than GD while EGB also performed approximately 12% worse than GD.

4.3.4 Cor vs Decor KMNIST

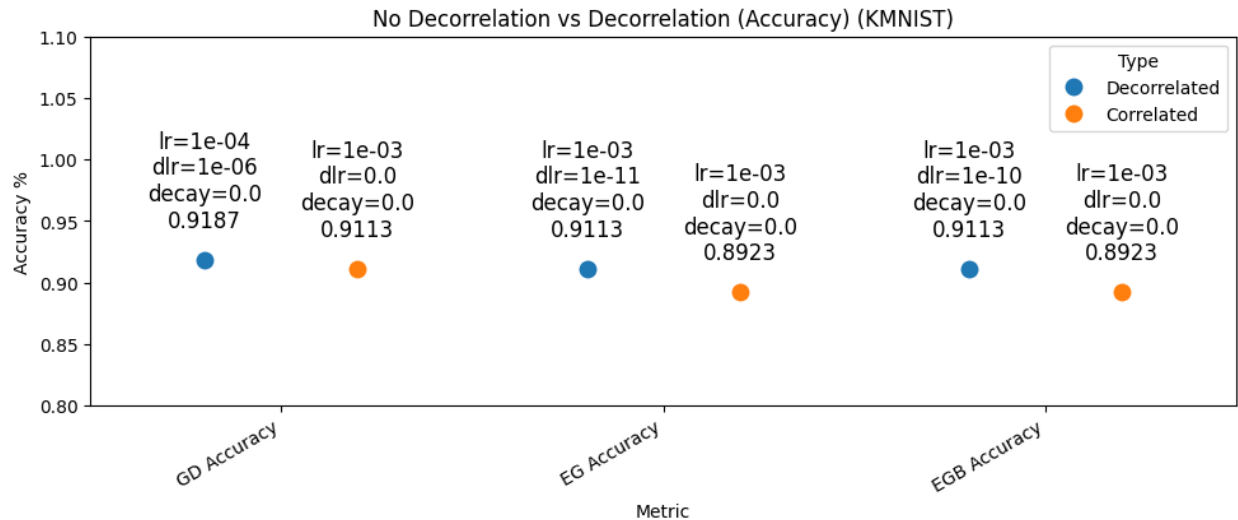


Figure 18: No Decorrelation vs Decorrelation for KMNIST with Accuracy as the metric across GD, EG, EGB

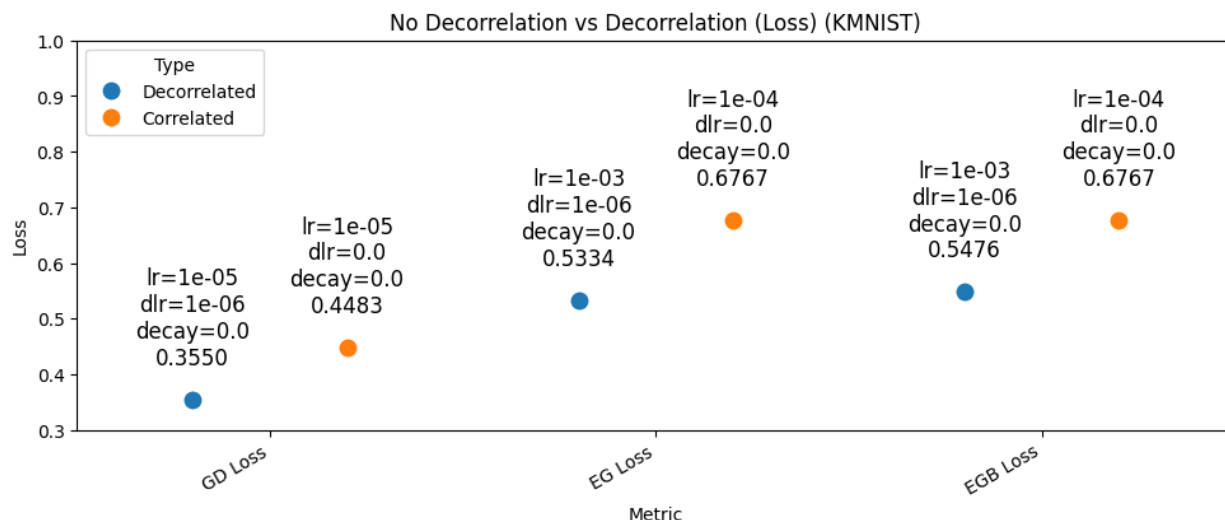


Figure 19: No Decorrelation vs Decorrelation for KMNIST with Loss as the metric across GD, EG, EGB

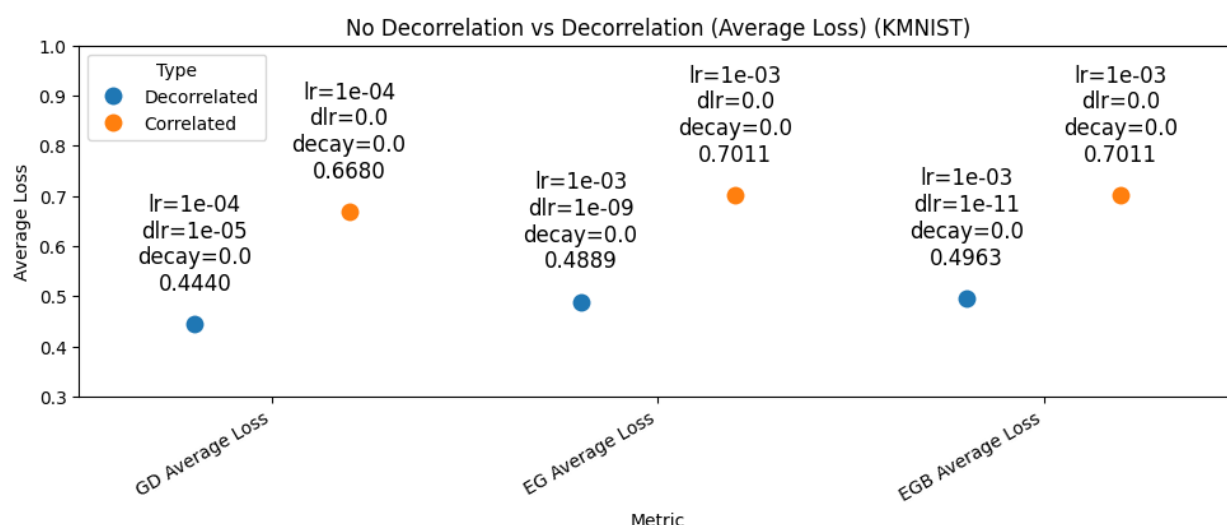


Figure 20: No Decorrelation vs Decorrelation for KMNIST with Average Loss as the metric across GD, EG, EGB

This section shows results between no decorrelation and decorrelation for the KMNIST dataset. In this section, the same conclusions as MNIST can be drawn. EG is worse than GD and EGB is worse than EG. Again, the EG and EGB have the exact same best data and the exact same best configurations. An interesting thing here is that EG and EGB had the exact same best accuracy for decorrelation, but the configurations are slightly different. In KMNIST, the overall accuracy is slightly lower compared to MNIST while the losses are slightly higher.

In terms of accuracy, the difference between decorrelation and no decorrelation is approximately 0.8% for GD and 2% for both EG and EGB. On the loss side of things, the difference is approximately 26% for GD, 27% for EG and 24% for EGB. For average loss, the difference is approximately 50% for GD, 43% for EG and 41% for EGB. Again, the results here are in favor of decorrelation.

4.3.5 GD vs EG vs EGB KMNIST

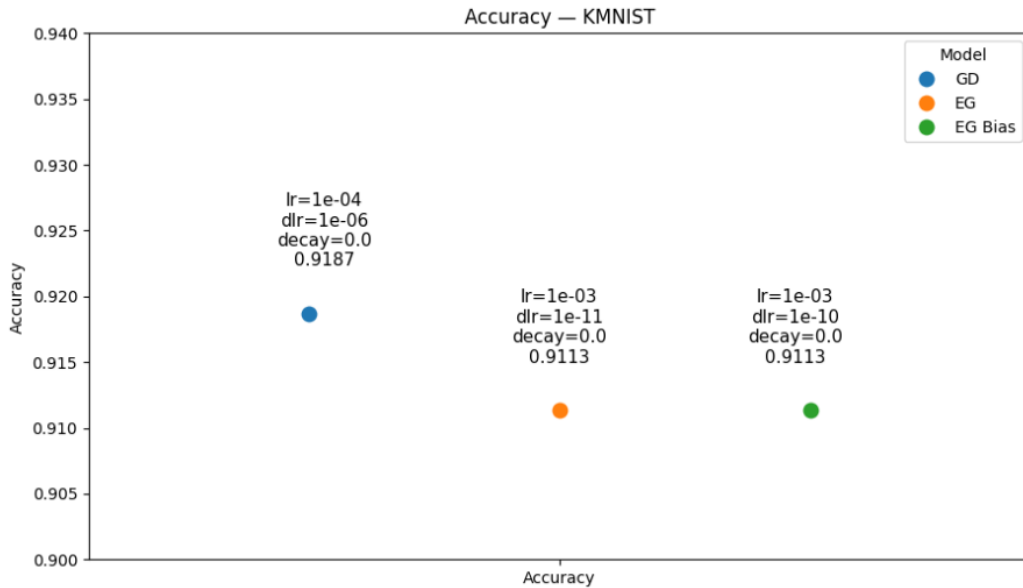


Figure 21: GD vs EG vs EGB accuracy comparison for KMNIST

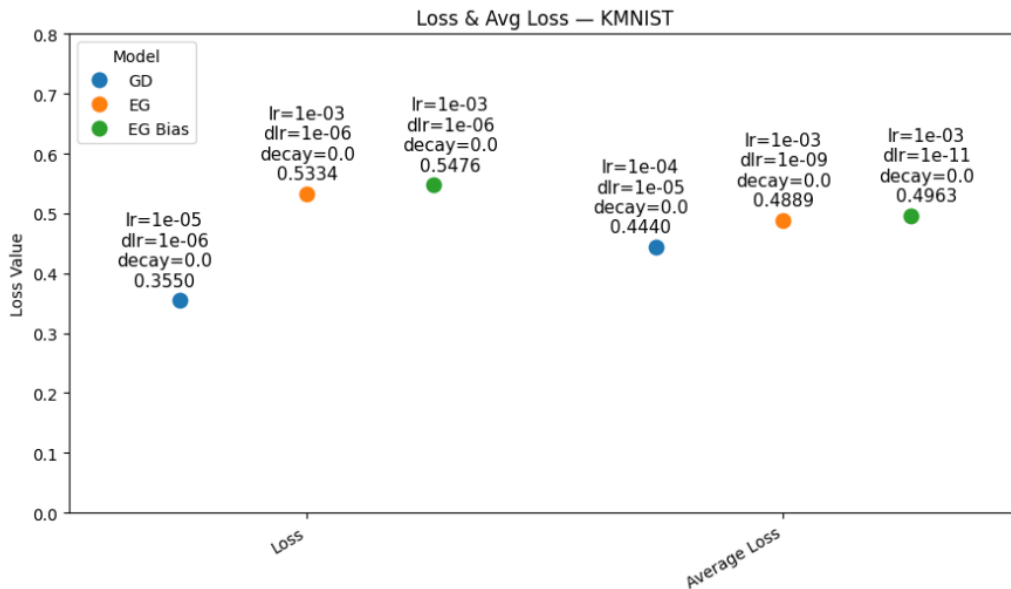


Figure 22: GD vs EG vs EGB Loss and Average Loss comparison for MNIST

The percentage difference between gradient descent methods of KMNIST will be detailed here. In terms of accuracy, Both EG and EGB performed approximately 0.8% worse than GD. In terms of loss, EG performed approximately 50% worse than GD while EGB performed approximately 54% worse than GD. In terms of average loss, EG performed approximately 10% worse than GD while EGB performed approximately 12% worse than GD.

4.3.6 Cor vs Decor CIFAR10

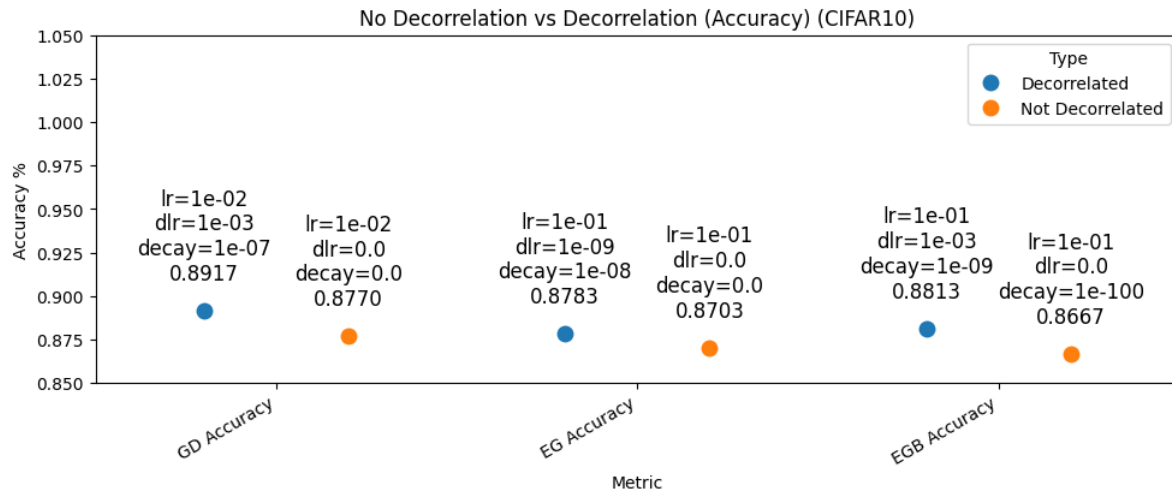


Figure 23: No Decorrelation vs Decorrelation for CIFAR10 with Accuracy as the metric across GD, EG, EGB

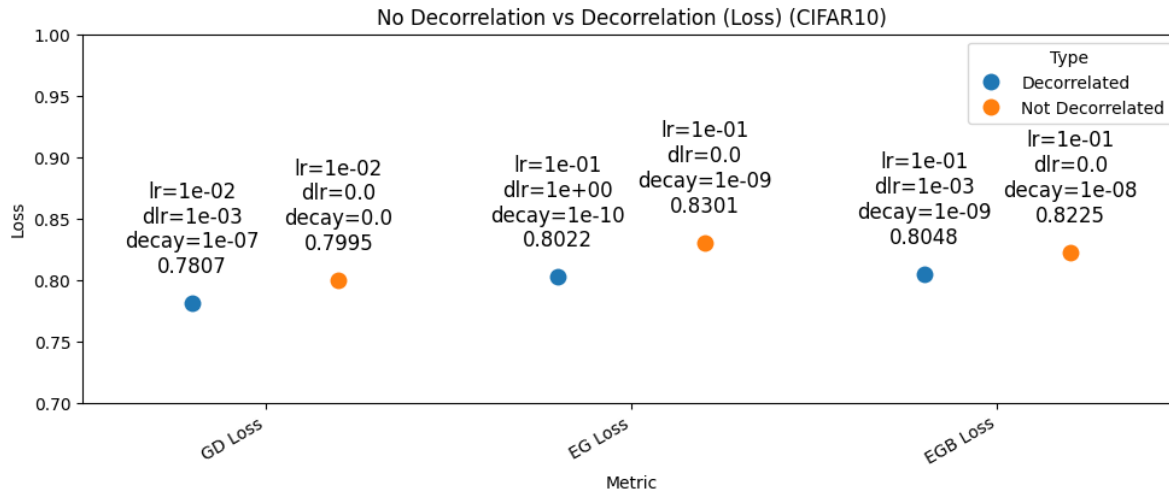


Figure 24: No Decorrelation vs Decorrelation for CIFAR10 with Loss as the metric across GD, EG, EGB

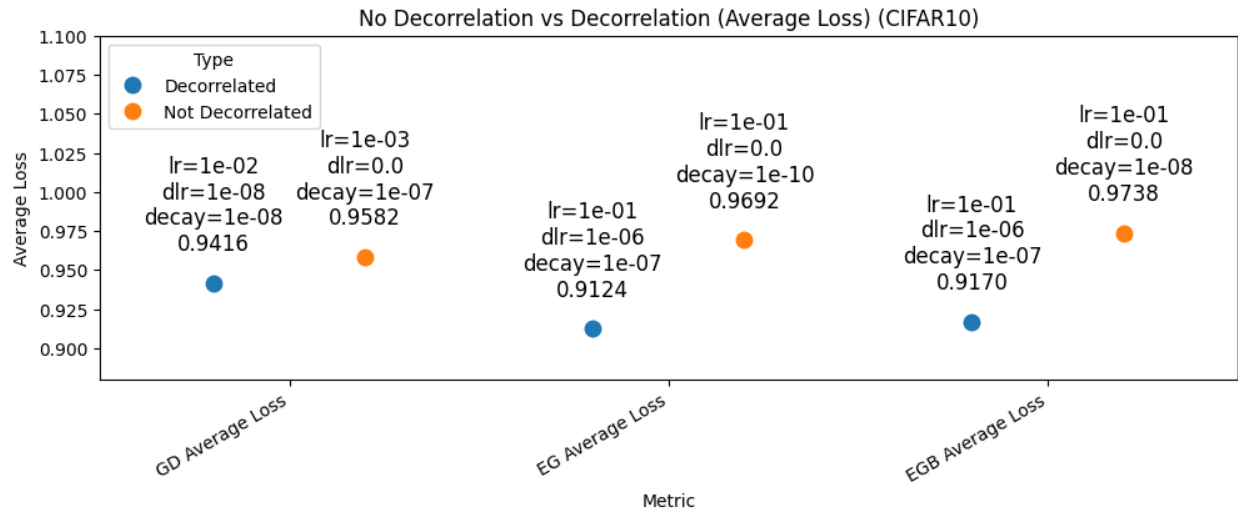


Figure 25: No Decorrelation vs Decorrelation for CIFAR10 with Average Loss as the metric across GD, EG, EGB

The graph above visualizes accuracy, loss and average loss for CIFAR10 trained using a ResNet9 CNN model. In this graph, the difference between no decorrelation and decorrelation can be clearly seen. A few interesting conclusions can be drawn from the graphs. The first is that weight decay this time is actually favored, as all configurations have some amount of weight decay in them. The second is that again, decorrelation is consistently better for all gradient descent and all metric of comparisons. When purely looking at no decorrelation data, we can see the same trend observed in MNIST and KMNIST, where GD is the best, followed by EG and lastly EGB. However, as previously mentioned, decorrelation does not follow the same trends.

In terms of accuracy, the difference between decorrelation and no decorrelation is approximately 2% for GD, 0.9% for EG and 2% for EGB. On the loss side of things, the difference is approximately 2% for GD, 3% for EG and 2% for EGB. For average loss, the difference is approximately 2% for GD, 6% for EG and 6% for EGB. Once again, all the results here favor decorrelation like KMNIST and MNIST.

4.3.7 GD vs EG vs EGB CIFAR10

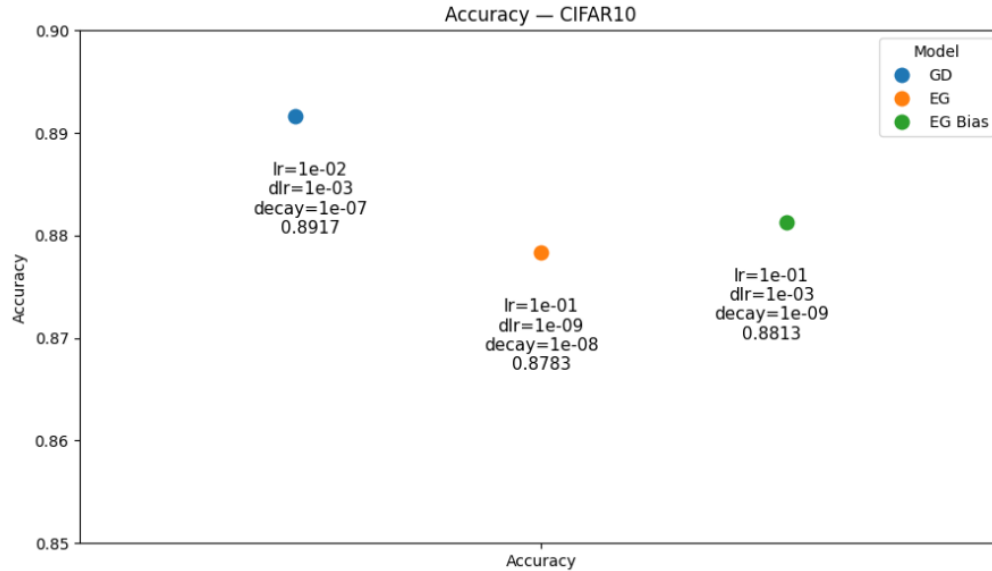


Figure 26: GD vs EG vs EGB accuracy comparison for CIFAR10

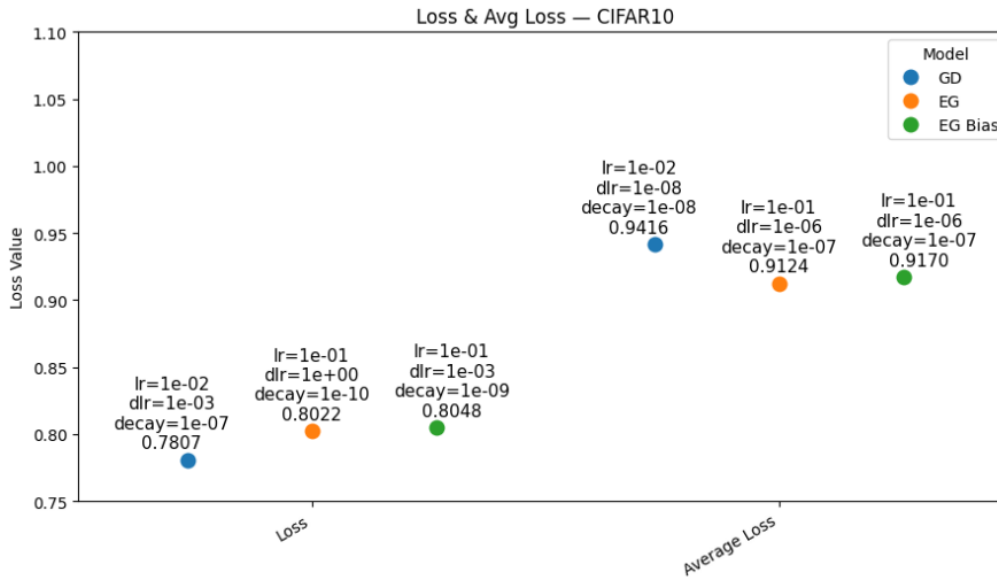


Figure 27: GD vs EG vs EGB accuracy comparison for CIFAR10

This section shows the statistics for CIFAR10. The graph draws comparison between GD, EG and EGB in terms of their accuracy, loss and average loss differences. The overall conclusion that can be drawn is similar to KMNIST and MNIST when an MLP model was used. However, in this part, a few things regarding best configuration stand out. Firstly, the overall learning rate used is larger than KMNIST and MNIST. Secondly, weight decay is actually preferred when training a CNN on CIFAR10. Lastly, GD does not have absolute dominance in terms of result. Previously,

the results followed a trend where GD was the best, followed by EG and lastly EGB. We can see that this is not the case here. GD wins in terms of accuracy and loss, but loses out to EGB in terms of average loss. Additionally, EGB is better than EG in terms of loss, but still loses to GD.

The percentage difference for accuracy is as such. EG is approximately 2% worse than GD while EGB is approximately 1% worse than GD. For loss, Both EG and EGB are approximately 3% worse than GD. For average loss, GD and EGB are approximately 3% worse than EG

Chapter 5

5.1 Validation of Results

This section will talk about the analysis on the effects of decorrelation. Covariance matrix along with Frobenius norm will be the metric of choice when analyzing decorrelation effects.

5.1.1 Covariance Matrix

To verify the effects of decorrelation, the covariance matrix between two MLP models trained on the MNIST dataset was visualized. Decorrelation removes correlation between other classes of data. This means that if data was completely decorrelated, the off diagonals of a covariance matrix would be 0. However, it is not realistic to expect data to be completely decorrelated. This is due to reasons like model constraints and nonlinear dependencies (Kessy et al., 2018). When training a neural network, it is not possible to rotate and rescale features of a dataset arbitrarily. The weights are also trying to optimize for classification, and this optimization cannot always reduce correlations to zero. Additionally, datasets have nonlinear dependencies which refer to quadratic relationships or relationships of higher order (Comon, 1994).

Covariance matrix is calculated using this equation:

$$\hat{\Sigma} = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T$$

Where N is the number of data samples and $(x_i - \bar{x})(x_i - \bar{x})^T$ is a $d \times d$ matrix capturing how each pair of features co-vary for sample i. x_i is the i-th data vector in the dataset while $\bar{x}$ is the sample mean of feature l across all N samples.

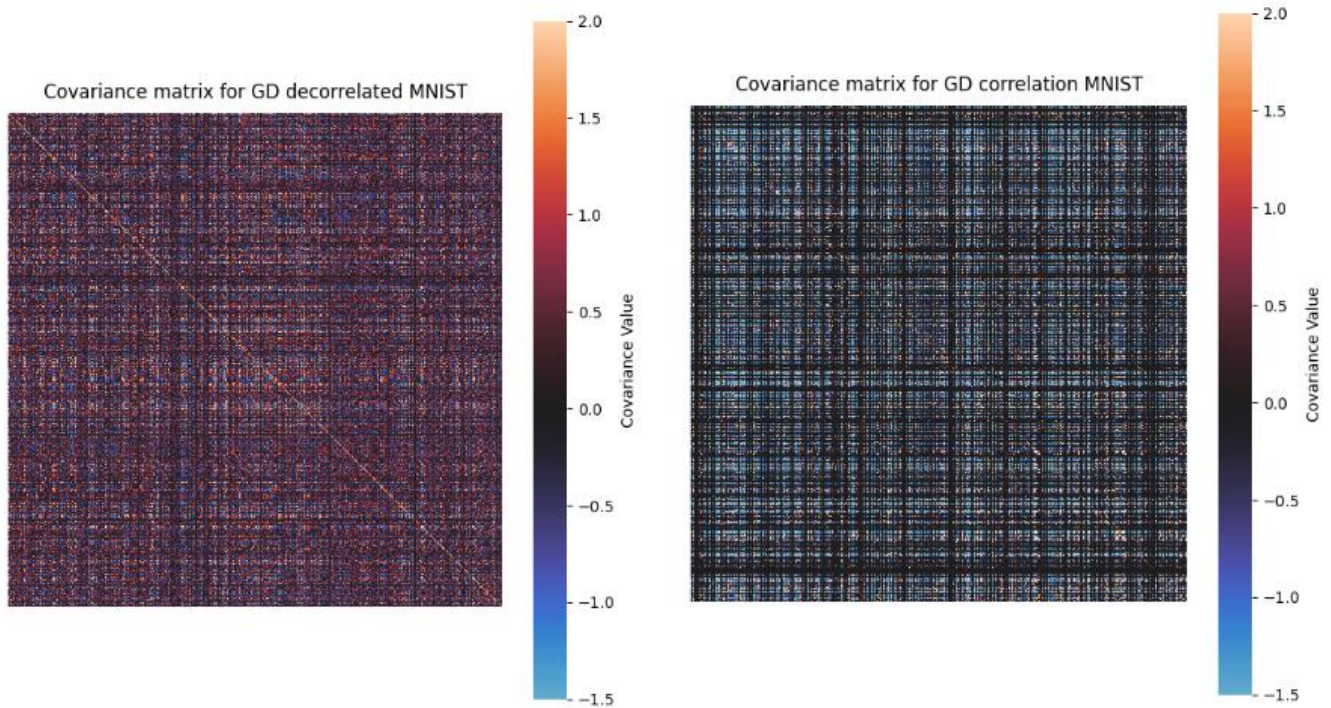


Figure 28: Covariance matrix with decorrelation (left) and no decorrelation (right)

The left matrix comes from training the model with $1e-4$ learning rate, and with a decorrelation learning rate of $1e-10$ with no weight decay. These parameters were selected based on the best hyperparameters found for GD with accuracy as the metric. The covariance matrix was taken from the model weights after the third linear layer after ReLU activation. The same was done for the covariance matrix on the right but the model with no decorrelation was used this time. For the right matrix, an MLP with learning rate of $1e-3$ with no decorrelation learning rate nor weight decay was used. From the image above, we can see that the decorrelation covariance matrix is darker. The diagonal line is more prevalent in the decorrelation covariance matrix. Whereas in the non-decorrelated matrix, the diagonal line is less visible and the values for the covariance matrix are everywhere. To properly gauge the effects of decorrelation, the Frobenius norm was calculated.

5.1.2 Frobenius Norm

Frobenius norm is the square root of sum of squares. It is used for measuring strength of correlations between different features. The larger the values, the stronger the correlations across the whole matrix and vice versa. The equation is as such:

$$|A|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

The equation takes every element of the matrix and sums it all together. However, since we are aiming to find the difference between correlation and decorrelation, it is important that the diagonal of the covariance matrix is zero. To validate the results, the Frobenius Norm of each dataset was calculated. The hyperparameters used were the ones found to be the best based on accuracy as the metric. All the data gathered here is using models that were trained using the best hyperparameters found for GD, EG and EGB. The Frobenius Norm is calculated at 4 different points in the neural network's hidden layer, namely x1, x2, x3, x4. These points in the neural network have been marked in section 3.2.5 in the figures detailing the structure of the neural networks.

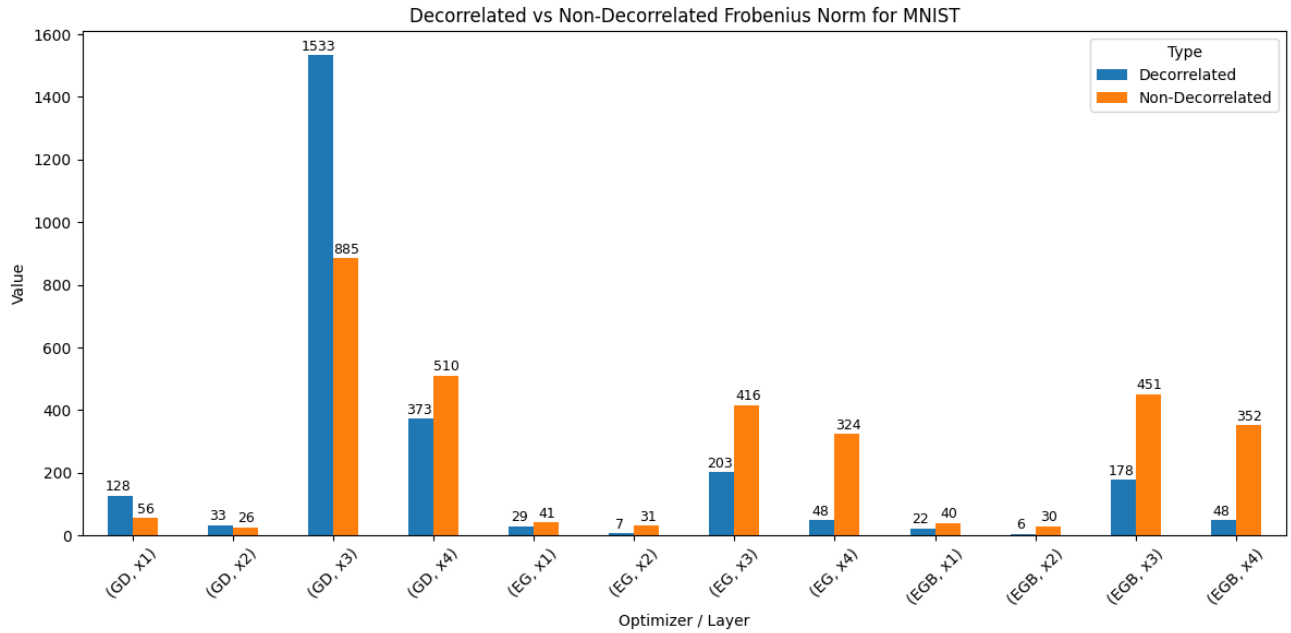


Figure 29: Frobenius Norm comparison for MNIST

The first to be analyzed is MNIST. For EG and EGB, we can see that the Frobenius norm for all layers are lower than the non-decorrelated counterpart, which is within expectations. However, GD presents an interesting case where the Frobenius norm for all 3 layers are higher for decorrelation than their non-decorrelated counterpart, but is lower at the x4 mark, which is right before the output layer.

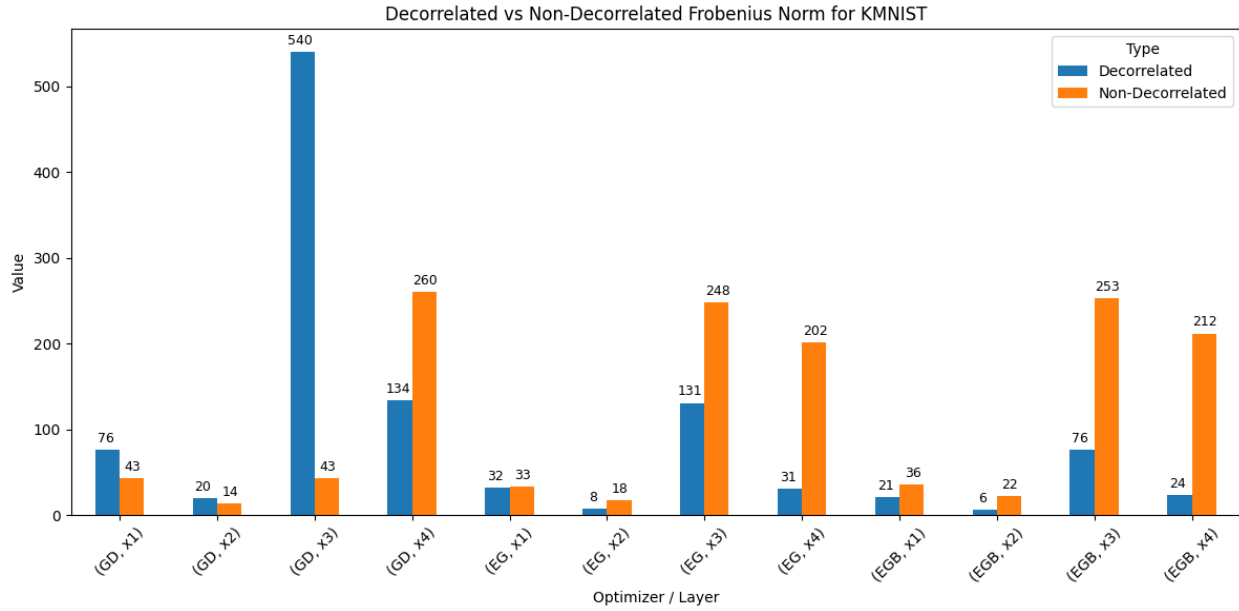


Figure 30: Frobenius Norm comparison for KMNIST

The same phenomenon can be found in KMNIST. EG and EGB have Frobenius norms that are lower than their non-decorrelated counterpart across all 4 layers, but GD only has a Frobenius norm that is lower than their non-decorrelated counterpart at the x4 mark.

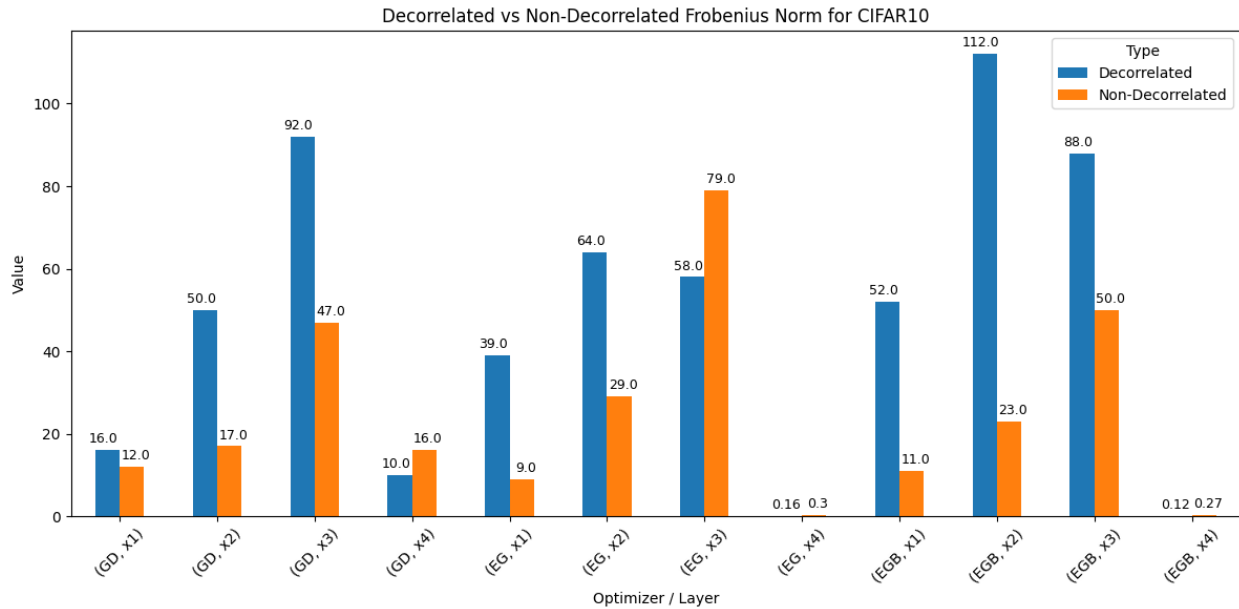


Figure 31: Frobenius Norm comparison for CIFAR10

When analyzing the Frobenius norm for CIFAR10, the phenomenon where Frobenius norm is higher than non-decorrelated all the way until the last layer is apparent for all 3 gradient descent methods.

A few theories can be provided for the higher Frobenius norm in all layers before x4. When performing decorrelation, weight vectors tend to spread out in the parameter space (Cogswell, 2015). By doing so, the magnitude of the weight increases since the weights are pushed to occupy more independent directions rather than scaling into lower-dimensional, redundant patterns. The reason x4 does not follow this trend is because x4 is right before the output layer. As the data approaches the output layer, the representations within x4 must be linearly separable. This means that x4 is constrained by the classification objective, which works well with decorrelation and leads to a decrease in Frobenius norm values (Cogswell et al., 2016).

Additionally, an interesting phenomenon can be observed as the dataset gets more complex. The Frobenius norm value gets lower for both decorrelated and non-decorrelated as dataset becomes more complex. In a simple dataset like MNIST, features cluster around a few dominant directions, meaning multiple features will point towards the same direction (Ayinde et al., 2019). When many features point toward the same direction, their correlation becomes even higher. Conversely, as dataset becomes more complicated, features become less redundant since the network must spread feature representation across many directions, which is why the overall Frobenius norm is lower when comparing CIFAR10 to MNIST and KMNIST.

Lastly, it can be speculated that EG and EGB had lower Frobenius norm across all 4 layers for decorrelation when comparing KMNIST and MNIST dataset because EG and EGB updates weights multiplicatively. The multiplicative weight updates overshadowed the effects of decorrelation on increasing Frobenius norm in early layers. However, the multiplicative effects of EG and EGB were not enough to reduce Frobenius norm in early layers when comparing CIFAR10 since more features started pointing towards different directions, and the effects of multiplicative updates were diminished when reflected in the form of Frobenius norm.

5.2 Result Interpretation

5.2.1 Hyperparameter Sweeps

Three main things can be concluded from the hyperparameter sweeps. The first is that KMNIST and MNIST do not want weight decay at all while CIFAR10 needs weight decay to do better. The second is that MNIST and KMNIST prefer a slightly smaller learning rate while CIFAR10 prefers a slightly larger learning rate. The last is that there are no clear trends for decorrelation learning rate across the board.

KMNIST and MNIST may not work with weight decay because it is a relatively simple dataset. KMNIST and MNIST have low-complexity patterns that a model can learn without heavy regularization. In this case, weight decay would unnecessarily penalize large weights even though they are beneficial for representing data accurately. This reduces the model's ability to fully fit simple, but important patterns found in the dataset, which leads to underfitting instead. Since simple datasets like MNIST and KMNIST are less prone to overfitting in the first place, the main advantage of weight decay is not really utilized and instead will lead to harmful results (Smith, 2018).

The reason that MNIST and KMNIST prefer a smaller learning rate is simply because KMNIST and MNIST are simpler datasets, which therefore means easier patterns. A simpler dataset with easier patterns naturally means that the decision boundaries between each class are clearer. By using a smaller learning rate, it allows the optimizer, which in this case are the gradient descent methods, to settle into a more well-defined minima, reducing overshooting and increasing precision (Yedida et al., 2019). Conversely, a more complex dataset like CIFAR10 would require a higher learning rate. This is because complex datasets have a more rugged, high dimensional nonconvex loss surface. Complex datasets benefit from larger learning rates because larger learning rates inject more noise into updates, which in turn helps the optimizer escape poor local minima and explore broader parameter space more efficiently (Mohtashami et al., 2023). Additionally, starting with a larger learning rate helps the model avoid memorizing noisy or overcomplicated patterns (You et al., 2019). This means that larger learning rates help the model to regularize and generalize better initially, provided that the learning rate is decayed off. This is exactly why a LinearLR was used, since the LinearLR scheduler gradually decays off the learning rate as training epoch goes on, which achieves the effect of the gradient descent method escaping poor minima at the start with large learning rates and eventually settling at a good minima when learning rate is eventually decayed off near the end.

5.2.2 Decorrelation for GD, EG and EGB

The full comparisons between non-decorrelation and decorrelation for each gradient descent method and comparison metric can be found in Appendix C broken down into. From the results gathered above, we can now answer the initial question that was posed at the start of the paper. Decorrelation boosts the performance of models trained with EG and EGB by some amount when looking at accuracy, and boosts the confidence of models by a decent amount when measuring

loss. Not only that, standard GD also benefits from the benefits that decorrelation provides by a relatively equal amount when compared to EG and EGB. Decorrelation also speeds up learning in neural network models. The average loss for decorrelation when compared against non-decorrelated examples with the same configurations are consistently lower. A lower average loss means that the model has higher overall confidence when learning overall and achieves that confidence more quickly. This implies that when using decorrelation, lesser training epochs may be required to train the model to a point where it is on par to a non-decorrelated model.

5.2.3 EG worse than GD

An interesting observation can be seen when comparing GD, EG and EGB side by side. EG and EGB consistently perform worse than GD in every scenario except 1. EGB beats both EG and GD only in CIFAR10 when comparing average loss. There are a few potential reasons for this.

5.2.4 Synaptic Pruning

Synaptic pruning is a process that removes weakest weight synapses from the neural network. This is done by zeroing out the smallest weights in terms of their absolute value. Synaptic pruning is useful in real life because when the brain is developing, it produces too many synapses, which can reach up to double the amount compared to an adult brain (Gemm Learning, n.d).

Thus synaptic pruning is needed to remove unnecessary synapses. By removing unnecessary synapses, it also improves signal processing and information flow since there will be less noise (Sakai, 2020)

EG enables synaptic pruning to happen in neural network training. Normally, gradient descent gives importance to all weights equally since it is linear, which means synaptic pruning will hurt the model a lot due to the fact that there are no “weakest” synaptic weights. EG scales weights exponentially, which allows there to be weaker and stronger weights and hence allows synaptic pruning to be performed without a performance loss (Cornford et al., 2024). On the contrary, EG with pruning actually shows better results compared to GD with no synaptic pruning. However, this experiment does not run EG with synaptic pruning since the main objective was to find out the effects of decorrelation on EG. If synaptic pruning were to be introduced, there would be more variables needed to be taken into consideration and could greatly increase the complexity without answering the main aims proposed by this paper.

5.2.5 Multiplicative noise injection

EG updates weights exponentially as the name implies. This also means that the weights could potentially blow up as the weight values increase exponentially. According to a study done by Nishida et al in 2025, EG, which was labeled as multiplicative weight updates in the paper, did worse than standard GD. This result aligns with what the current results in this study has shown us. However, the study proposed an interesting solution to counteract this issue. The paper introduced something called multiplicative noise injection. Multiplicative noise injection first samples the log-normal noise based on the weights in the current update and then multiplies the sampled noise into the current weight median to get a new sampled weight. Since the algorithm samples noise based on the current weight, it means that the weights are perturbation relative to its size. This in turn encourages control and stability by ensuring that big sized weights get more noise and small size weights get lesser noise. The reason why log-normal noise was chosen instead of Gaussian noise is because log-normal noise is motivated by biology. Log-normal noise contains proportional fluctuations which conforms with biology and also works better with the multiplicative nature of EG (Nishida et al., 2025). By introducing multiplicate noise injection, it increases the model's ability to generalize. Ultimately, EG with noise injection was found to be better than EG without noise injection, and can even reach accuracies higher than standard GD (Nishida et al., 2025).

Chapter 6

6.1 Conclusion

This study set out to determine whether combining unsupervised decorrelation with Exponential Gradient (EG) and its split bias variant (EGB) could improve neural networks compared to traditional Gradient Descent (GD). Across MNIST, KMNIST, and CIFAR10, results consistently showed that decorrelation enhanced performance for all optimization methods, improving both accuracy and loss metrics, with the effect being most pronounced in loss reduction. However, EG and EGB generally underperformed compared to GD, except in one specific scenario where EGB performed better than GD and EG when comparing average loss for CIFAR10. The findings confirm that decorrelation does indeed provide measurable benefits regardless of the optimizer used, which support the biological inspiration and the hypothesis that using a biological method will open new doors for neural networks.

However, EG and EGB themselves perform worse than GD. The primary reason lies in EG's multiplicative weight update mechanism, which, while biologically plausible and beneficial for enforcing constraints like Dale's law, can introduce instability by disproportionately amplifying large weights and shrinking small ones. This behavior can lead to slower convergence, less balanced weight distributions, and a heightened sensitivity to hyperparameters. Additionally, without complementary techniques such as synaptic pruning or multiplicative noise injection—both shown in other studies to stabilize EG and improve generalization—EG's advantages in sparsity and biological realism are outweighed by GD's more stable, additive updates, which tend to produce consistently higher accuracy and lower loss in standard training settings.

6.2 Objectives Achieved

This project successfully achieved all the objectives listed down in Section 1.2. In Chapter 2, a thorough literature review of decorrelation, GD, EG and EGB was conducted. The Chapter not only talks about the history of decorrelation and gradient descent methods, it also listed down all the equations pertaining to the 4 key topics. Furthermore, the derivation and conception of the equations regarding the 4 key topics were also shown step by step, explained in a way that is simple enough to understand to most people. The second objective was achieved by performing hyperparameter sweeps. The full results and heatmaps generated by the hyperparameter sweeps can be found in (Appendix B). It was found that LinearLR scheduler works best when

experimenting with hyperparameter sweeps since OneCycleLR schedulers tend to use a range of different learning rates which completely negates the purpose of performing a hyperparameter sweep. Suitable MLP and CNN structures were developed to test the datasets, along with their counterparts that swaps out the hidden layers for decorrelated layers. The neural network structures were deemed suitable because after they were trained, the models managed to predict MNIST with around 98% accuracy and around 90% accuracy for both KMNIST and CIFAR10. Additionally, visible differences can be seen when swapping out the regular hidden layers for decorrelated hidden layers, which proves that it is not just pure coincidence that decorrelation had a higher performance despite every other configuration remaining the same between the non-decorrelated and decorrelated model. 3 datasets were prepared and loaded for this experiment. As mentioned throughout this paper, the 3 datasets are KMNIST, MNIST and CIFAR10. MNIST and KMNIST were loaded traditionally using torchvision's dataloader library while CIFAR10 was loaded with the special FFCV loader. The required splits and transformations were applied to the dataset which have been discussed in detail above. Experiments were done across 3 seeds and lastly a test set, and the results have been thoroughly analyzed and discussed.

6.3 Significance of results

The findings of this project carry significant implications for training neural networks. First of all, the results where decorrelation shows a performance boost were successfully replicated, claims made by Ahmad in 2024 about decorrelation showing positive gains in performance have been verified in this paper. Secondly, EG was shown to be worse than GD, however that is expected and the findings in this paper are on par with the ones found in a study done by Nishida et al in 2025. Additionally, the results show that EG is worse than GD regardless of whether decorrelation is applied or not. This fact tells us that decorrelation alone does not change the fact that EG will be worse than GD, and measures like synaptic pruning and multiplicative noise injection must be performed to boost EG to a state where its performance is higher than GD.

Since decorrelation is shown to be clearly working in multiple scenarios regardless of simple datasets (MNIST, KMNIST) or complex datasets with colored images (CIFAR10), decorrelation can be said to be the future of training neural networks. By employing decorrelation in training for neural networks used in society, a slew of social impacts will happen. Healthcare, a field that heavily cares about accuracy, will be one of the fields that will be impacted heavily by higher accuracies. Medical AI can deliver faster and more precise diagnoses, such as when identifying cancer (Doya, 2022). Normally, biases or inaccuracy will pose huge risks when attempting to use

medical AI for diagnoses, but decorrelation could alleviate the risks by removing biases in data and increasing accuracy. Lastly, since decorrelation provides a lower average loss overall, it implies that decorrelation trains the model faster and lesser epochs may be required. This means that lesser power will be used when training decorrelated models, which would imply lesser greenhouse gas produced when talking about sustainability.

6.4 Future Work

A number of things can still be done to further investigate the effects of decorrelation on gradient descent methods. The first would be to try out different loss methods. In this experiment, Cross Entropy Loss was used throughout as the objective function. This is because image classification is about predicting probabilities over classes. Cross entropy comes from the concept of measuring distance between true distribution represented in one-hot labels and the model's predicted distribution. The concept is derived from Maximum Likelihood Principle (Hui, 2021). However, other loss functions like Hinge loss or Kullback-Leibler divergence can be used as the loss function. Decorrelation may or may not work better with these loss functions, which is what can be investigated in the future. Different schedulers could also be used aside from LinearLR. In the future, the experiment can be more targeted towards EG by introducing techniques that are made specifically for EG to thrive, namely synaptic pruning and multiplicative noise injection. Decorrelation then can be run in conjunction with EG backed by synaptic pruning and multiplicative noise injection, and the effects can be investigated and compared again with GD. Different datasets can also be used. CIFAR10 is only considered a dataset with a mediocre difficulty, harder datasets like ImageNet10 exist. The experiment can push the limits of decorrelation even further and investigate the impact of decorrelation when training a neural network on even harder datasets. The models used are also not optimized. Models with better configurations clearly exist out there. Although chasing accuracy is not the main objective of the experiment, there still remains the possibility that different model structures may benefit decorrelation more. Lastly, the entire nature of the experiment itself can be changed. Why limit ourselves to only image classification? AI is used in a multitude of areas in real life. The experiment can investigate decorrelation's effects on text and semantic analysis, speech and audio processing and also decision making. Image classification is just a drop in the bucket for what AI can do. The ultimate goal would be to see if decorrelation is a universal upgrade or just an upgrade for image classification.

References

- Ahmad, N. (2024). Correlations are ruining your gradient descent. arXiv preprint arXiv:2407.10780.
- Anderljung, M., Barnhart, J., Korinek, A., Leung, J., O'Keefe, C., Whittlestone, J., ... & Wolf, K. (2023). Frontier AI regulation: Managing emerging risks to public safety. arXiv preprint arXiv:2307.03718.
- Ayinde, B. O., Inanc, T., & Zurada, J. M. (2019). On correlation of features extracted by deep neural networks. In *Proceedings of the 2019 International Joint Conference on Neural Networks (IJCNN)**. IEEE.
- Barlow, H. B. (1961). Possible principles underlying the transformation of sensory messages. *Sensory communication*, 1(01), 217-233.
- Bredenberg, C., Williams, E., Savin, C., Richards, B., & Lajoie, G. (2023). Formalizing locality for normative synaptic plasticity models. *Advances in Neural Information Processing Systems*, 36, 5653-5684.
- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep learning for classical japanese literature. arXiv preprint arXiv:1812.01718.
- Cogswell, M., Ahmed, F., Girshick, R., Zitnick, L., & Batra, D. (2015). Reducing overfitting in deep networks by decorrelating representations. arXiv preprint arXiv:1511.06068.
- Comon, P. (1994). Independent component analysis, a new concept? *Signal Processing*, 36(3), 287–314. [https://doi.org/10.1016/0165-1684\(94\)90029-9](https://doi.org/10.1016/0165-1684(94)90029-9)
- Cornford, J., Pogodin, R., Ghosh, A., Sheng, K., Bicknell, B. A., Codol, O., ... & Richards, B. A. (2024). Brain-like learning with exponentiated gradients. *bioRxiv*, 2024-10.

- Dorkenwald, S., Turner, N. L., Macrina, T., Lee, K., Lu, R., Wu, J., ... & Seung, H. S. (2022). Binary and analog variation of synapses between cortical pyramidal neurons. *Elife*, 11, e76120.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7).
- Eccles, JC (1976). "From electrical to chemical transmission in the central nervous system: The closing address of the Sir Henry Dale Centennial Symposium Cambridge, 19 September 1975". *Notes and Records of the Royal Society of London*. 30 (2): 219–30. doi:10.1098/rsnr.1976.0015. PMID 12152632. S2CID 35451783.
- Elharrouss, O., Mahmood, Y., Bechqito, Y., Serhani, M. A., Badidi, E., Riffi, J., & Tairi, H. (2025). Loss Functions in Deep Learning: A Comprehensive Review. *arXiv preprint arXiv:2504.04242*.
- Gemm Learning. (n.d.). Why synaptic pruning is important. Gemm Learning. Retrieved August 28, 2025, from https://www.gemmlearning.com/blog/learning_science/why-synaptic-pruning-is-good/
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Chapter 4: Numerical Computation, Section 4.3 Gradient-Based Optimization).
- Hastie, T., Tibshirani, R., Botstein, D., & Brown, P. (2001). Supervised harvesting of expression trees. *Genome Biology*, 2(1), research0003-1.
- He, H. Y., & Cline, H. T. (2019). What is excitation/inhibition and how is it regulated? A case of the elephant and the wisemen. *Journal of experimental neuroscience*, 13, 1179069519859371.
- Hebb, D. O. (1949). The first stage of perception: growth of the assembly
- Hui, L., & Belkin, M. (2020). Evaluation of neural architectures trained with square loss vs cross-entropy in classification tasks. *arXiv preprint arXiv:2006.07322*.

- Kessy, A., Lewin, A., & Strimmer, K. (2018). Optimal whitening and decorrelation. *The American Statistician*, 72(4), 309–314. <https://doi.org/10.1080/00031305.2016.1277159>
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- Kivinen, J., & Warmuth, M. K. (1997). Exponentiated gradient versus gradient descent for linear predictors. *Information and computation*, 132(1), 1-63.
- Koren, V., Malerba, S. B., Schwalger, T., & Panzeri, S. (2025). Efficient coding in biophysically realistic excitatory-inhibitory spiking networks. *Elife*, 13, RP99545.
- Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images.
- Leclerc, G., Ilyas, A., Engstrom, L., Park, S. M., Salman, H., & Madry, A. (2023). FFCV: Accelerating training by removing data bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. <https://github.com/libffcv/ffcv/>
- LeCun, Y., & Bengio, Y. (1998). Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*.
- Lindsay, G. W. (2021). Convolutional neural networks as a model of the visual system: Past, present, and future. *Journal of cognitive neuroscience*, 33(10), 2017-2031.
- Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101.
- Lu, J. (2022). Gradient descent, stochastic optimization, and other tales. arXiv preprint arXiv:2205.00832.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in neural nets. *Bull Math. Biophys*, 5, 133-137.

- Mishra, P., & Sarawadekar, K. (2019, October). Polynomial learning rate policy with warm restart for deep neural network. In TENCON 2019-2019 IEEE Region 10 Conference (TENCON) (pp. 2087-2092). IEEE.
- Mohtashami, A., Jaggi, M., & Stich, S. U. (2023). Special Properties of Gradient Descent with Large Learning Rates. In Proceedings of the 40th International Conference on Machine Learning (ICML) (Vol. 202, pp. 25082–25104). PMLR
- NeurIPS. (n.d.). Ethics guidelines. NeurIPS. Retrieved August 28, 2025, from <https://neurips.cc/public/EthicsGuidelines>
- Nishida, K., Kiral, E. M., Bannai, K., Khan, M. E., & Möllenhoff, T. (2025). Log-Normal Multiplicative Dynamics for Stable Low-Precision Training of Large Networks. arXiv preprint arXiv:2506.17768.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019).
- Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32.
- Pitkow, X., & Meister, M. (2012). Decorrelation and efficient coding by retinal ganglion cells. Nature neuroscience, 15(4), 628-635.
- Pogodin, R., Cornford, J., Ghosh, A., Gidel, G., Lajoie, G., & Richards, B. (2023). Synaptic weight distributions depend on the geometry of plasticity. arXiv preprint arXiv:2305.19394.
- PyTorch Contributors. (n.d.). `*torch.mean*` (PyTorch documentation). <https://docs.pytorch.org/docs/stable/generated/torch.mean.html>
- PyTorch Contributors. (n.d.). `torch.optim.AdamW` (PyTorch documentation). <https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html>
- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6), 386.

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088), 533-536.
- Sakai, J. (2020). Core concept: How synaptic pruning shapes neural wiring during development and, possibly, in disease. *Proceedings of the National Academy of Sciences of the United States of America*, 117(28), 16096–16099. <https://doi.org/10.1073/pnas.2010281117>
- Smith, L. N. (2018). A disciplined approach to neural network hyper-parameters: Part 1--learning rate, batch size, momentum, and weight decay. *arXiv preprint arXiv:1803.09820*.
- Spitzer, M. (1999). *The mind within the net: Models of learning, thinking, and acting*. Mit Press.
- Straub, J. (2021). Gradient descent training expert system. *Software Impacts*, 10, 100121.
- Terven, J., Cordova-Esparza, D. M., Romero-González, J. A., Ramírez-Pedraza, A., & Chávez-Urbiola, E. A. (2025). A comprehensive survey of loss functions and metrics in deep learning. *Artificial Intelligence Review*, 58(7), 195.
- Tian, Y., Zhang, Y., & Zhang, H. (2023). Recent advances in stochastic gradient descent in deep learning. *Mathematics*, 11(3), 682.
- Tieleman, T., & Hinton, G. (2012). Lecture 6.5-rmsprop, coursera: Neural networks for machine learning. University of Toronto, Technical Report, 6.
- University of Leeds. (n.d.). Platforms. Advanced Research Computing. <https://arc.leeds.ac.uk/platforms/>
- Vogels, T. P., Sprekeler, H., Zenke, F., Clopath, C., & Gerstner, W. (2011). Inhibitory plasticity balances excitation and inhibition in sensory pathways and memory networks. *Science*, 334(6062), 1569-1573.
- Wan, K., Sun, H., Ji, M., & Caire, G. (2022). On secure distributed linearly separable computation. *IEEE Journal on Selected Areas in Communications*, 40(3), 912-926.

Yedida, R., Saha, S., & Prashanth, T. (2021). Lipschitzlr: Using theoretically computed adaptive learning rates for fast convergence. *Applied Intelligence*, 51(3), 1460-1478.

You, K., Long, M., Wang, J., & Jordan, M. I. (2019). How does learning rate decay help modern neural networks?. *arXiv preprint arXiv:1908.01878*.

Appendix A

Datasets

MNIST

The first dataset used to benchmark decorrelation and EG was downloaded via the torchvision library, namely torchvision.datasets.MNIST. MNIST is a dataset that contains images of digits 0 to 9 in 28x28 black and white. The official documentation can be found here: <https://docs.pytorch.org/vision/mqain/generated/torchvision.datasets.MNIST.html>

KMNIST

The second dataset used was sourced from Kaggle. It follows the same structure as MNIST, containing 10 classes, but this time instead of digits from 0 to 9, it contains 10 Japanese Hiragana letters and is also 28x28 black and white. The dataset can be accessed here: <https://www.kaggle.com/datasets/anokas/kuzushiji>

CIFAR10

The last dataset used was also downloaded via the torchvision library, namely torchvision.datasets.CIFAR10. CIFAR10 contains images of various objects, and is a colored 32x32 image. The official documentation for the library can be found here: <https://docs.pytorch.org/vision/main/generated/torchvision.datasets.CIFAR10.html>

Libraries

Pytorch

Pytorch is an open-source deep learning framework that allows for GPU-accelerated tensor computation for training and building neural networks

Torchvision

Torchvision is a Pytorch library that provides the ability to create datasets and dataloaders, along with image transformation utilities to improve training and allow for datasets to be loaded for neural networks to use.

Numpy

Numpy allows for fast numerical computing that provides n-dimensional array objects. Numpy allows for manipulation of vectors and matrices, which is useful when processing datasets.

Wandb

Weights & Biases (Wandb) is a tool for experiment tracking, visualization, and collaboration in machine learning: it logs hyperparameters, metrics, system stats, and model artifacts; visualizes results with interactive dashboards. This library is mainly used to log the performance of hyperparameter sweeps

Matplotlib

Matplotlib is used for creating various plots. Most of the plots used in this report are generated with Matplotlib. Matplotlib also allows for combining multiple plots into one and improves the structure of the graph in general

Pandas

Pandas is a Python library that provides fast, flexible tools for working with structured data, using Series and DataFrame objects. It is mainly used for making datasets to hold all the compiled data obtained after training

Python Image Library (PIL)

PIL (Python Imaging Library) is a Python library for opening, manipulating, and saving many different image formats, offering tools for tasks like resizing, cropping, filtering, and format conversion.

Seaborn

Seaborn is also used for creating various plots. However, Seaborn provides more options that allow for more complicated graphs. One example of this would be the heatmaps that are plotted using the Seaborn library

Decorrelation

Decorrelation here is not a library that is downloaded via PIP package manager, but is instead downloaded from a GitHub repo, which can be accessed here:

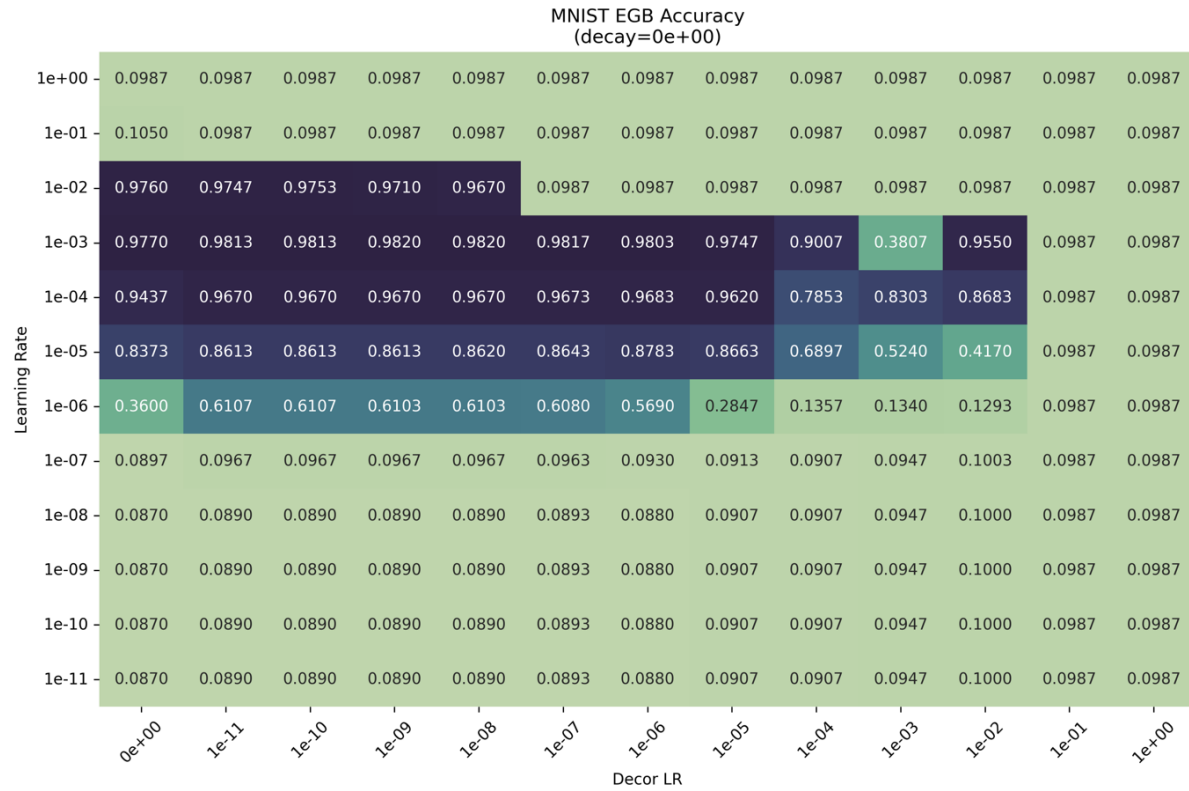
<https://github.com/nasiryahm/CorrelationsRuinGD/blob/main/README.md>

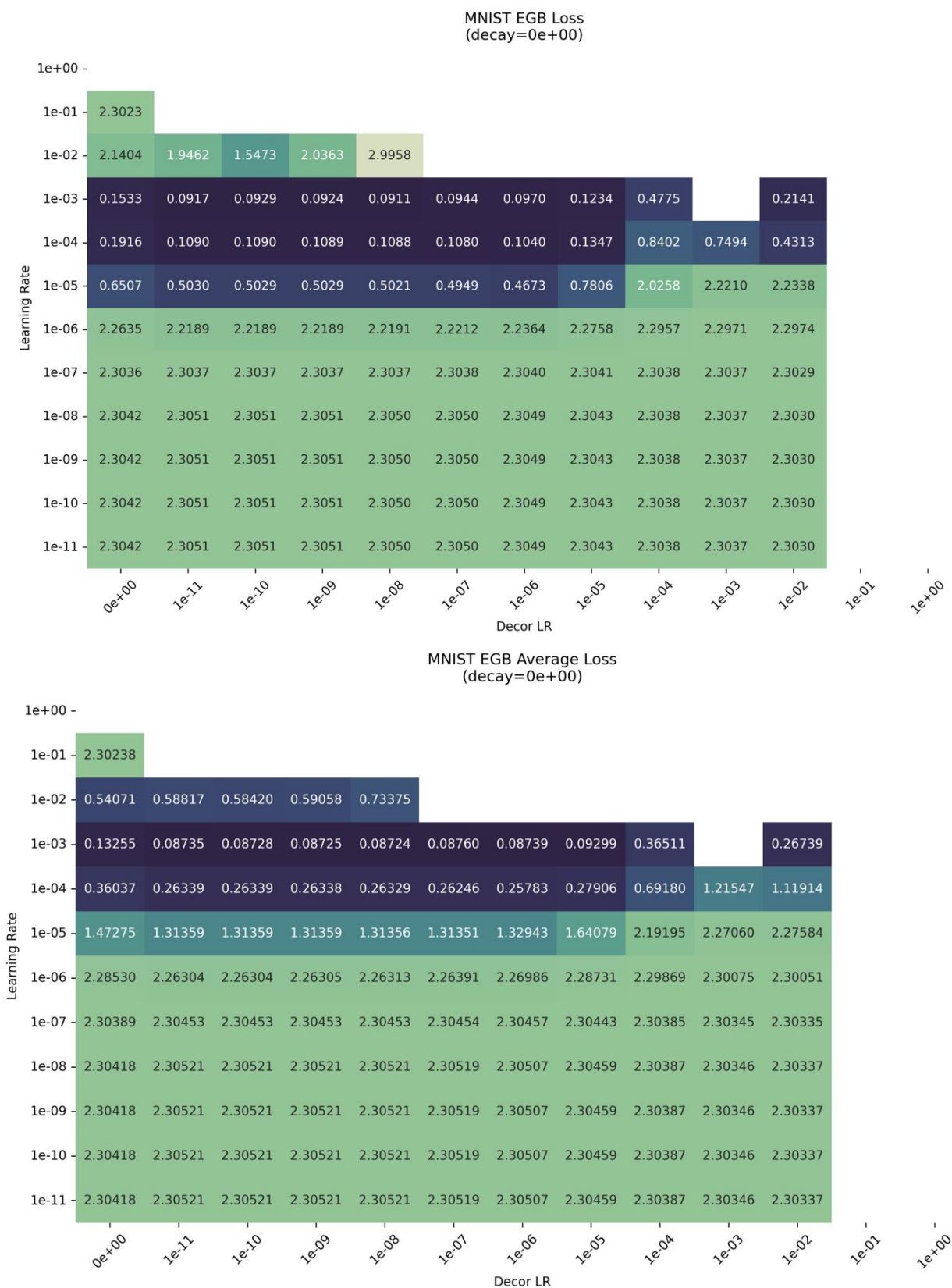
Exponentiated Gradient

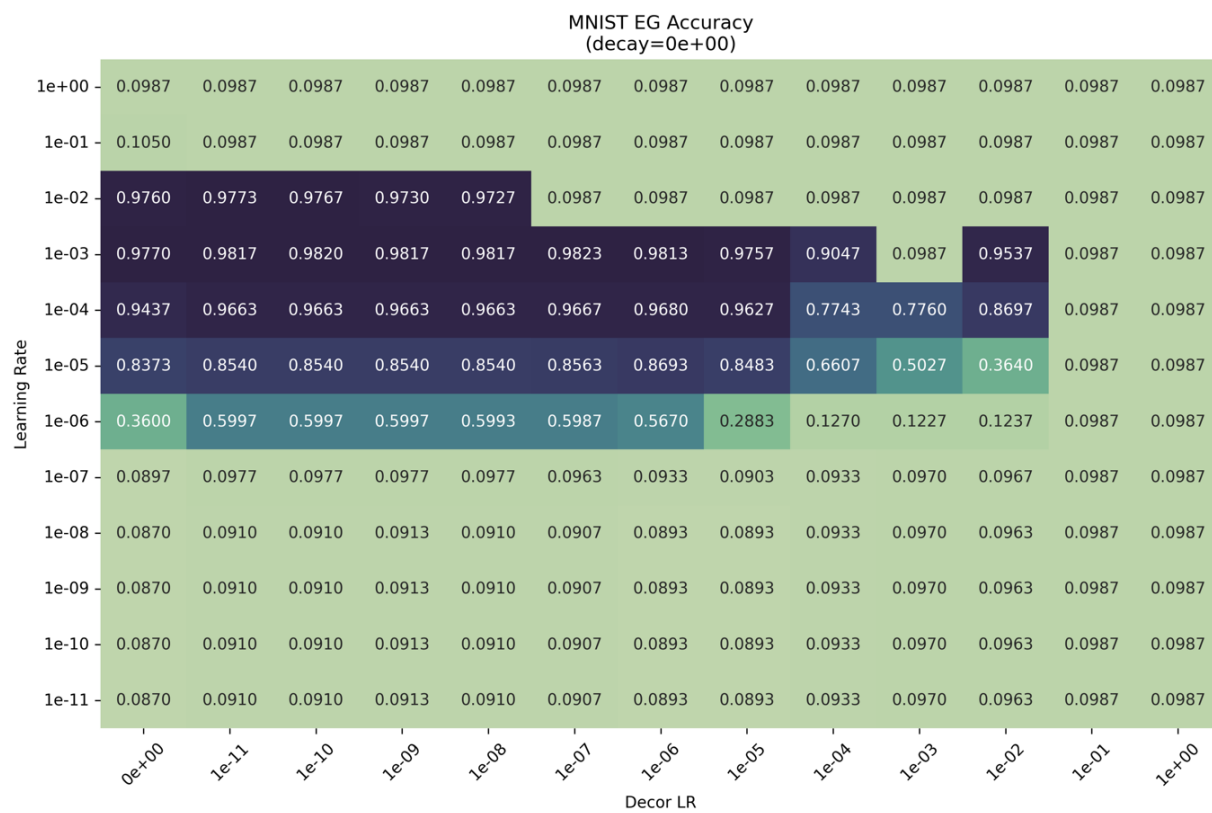
Exponentiated Gradient is also not a library that is downloaded via PIP package manager, but is also instead downloaded from a GitHub repo, which can be accessed here:

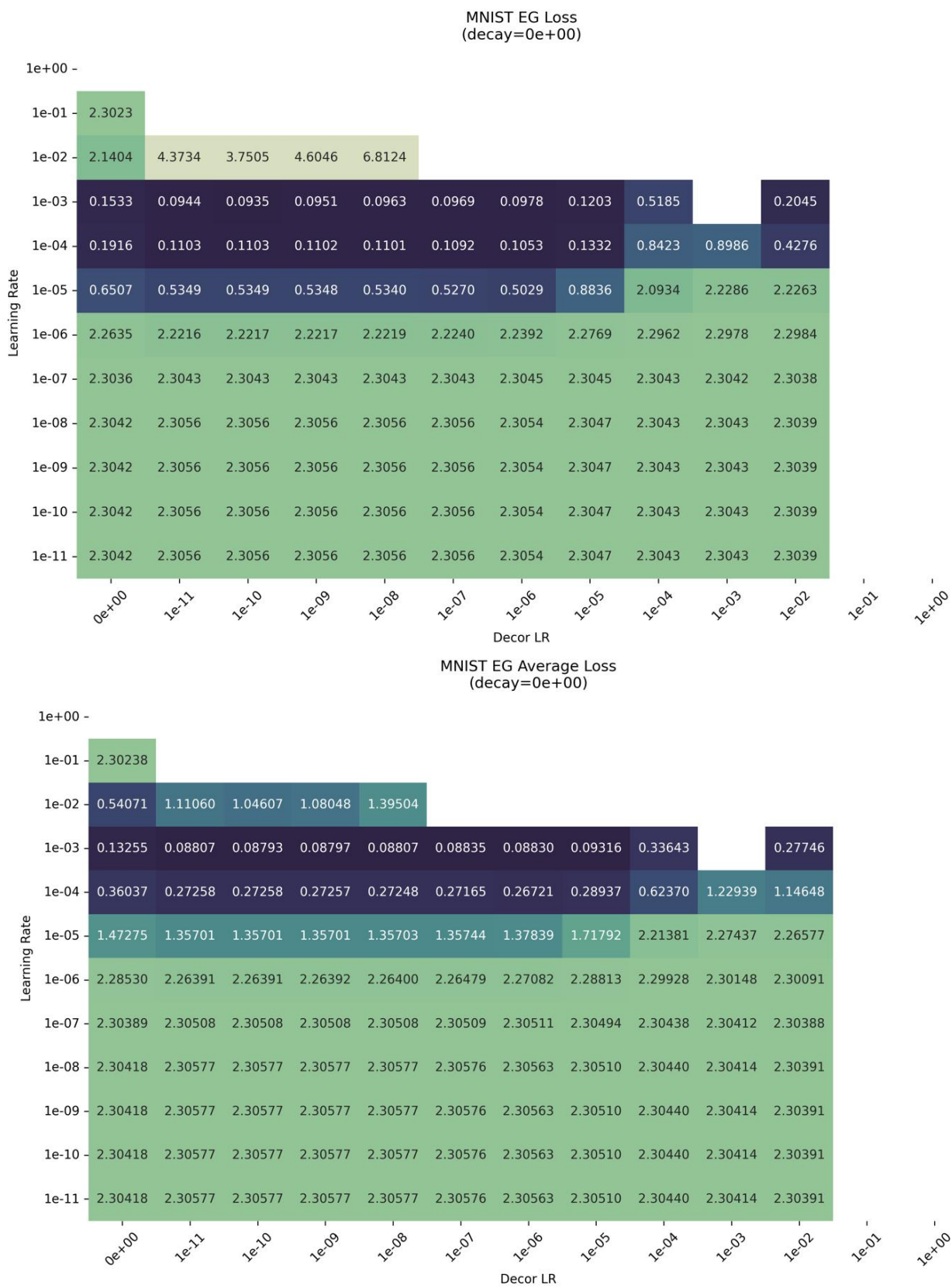
https://github.com/linclab/EG_optimiser/blob/main/example.ipynb

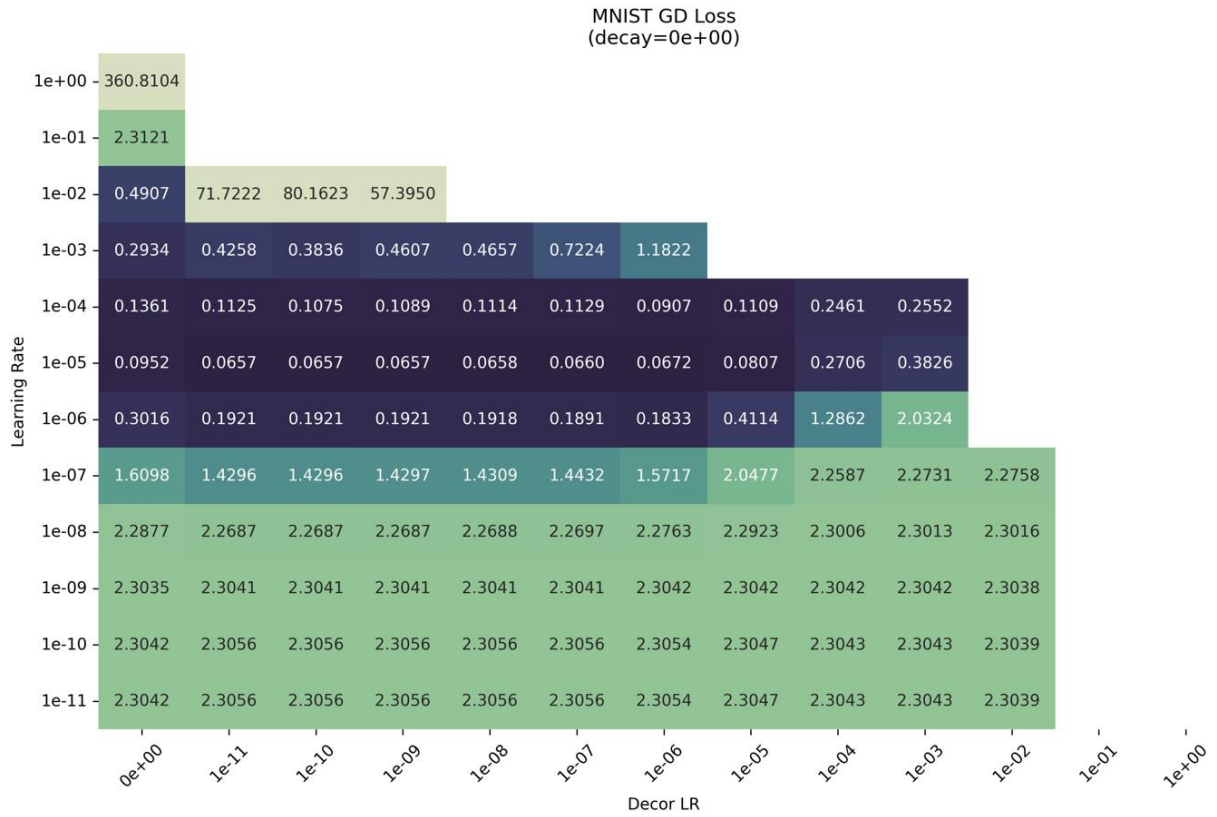
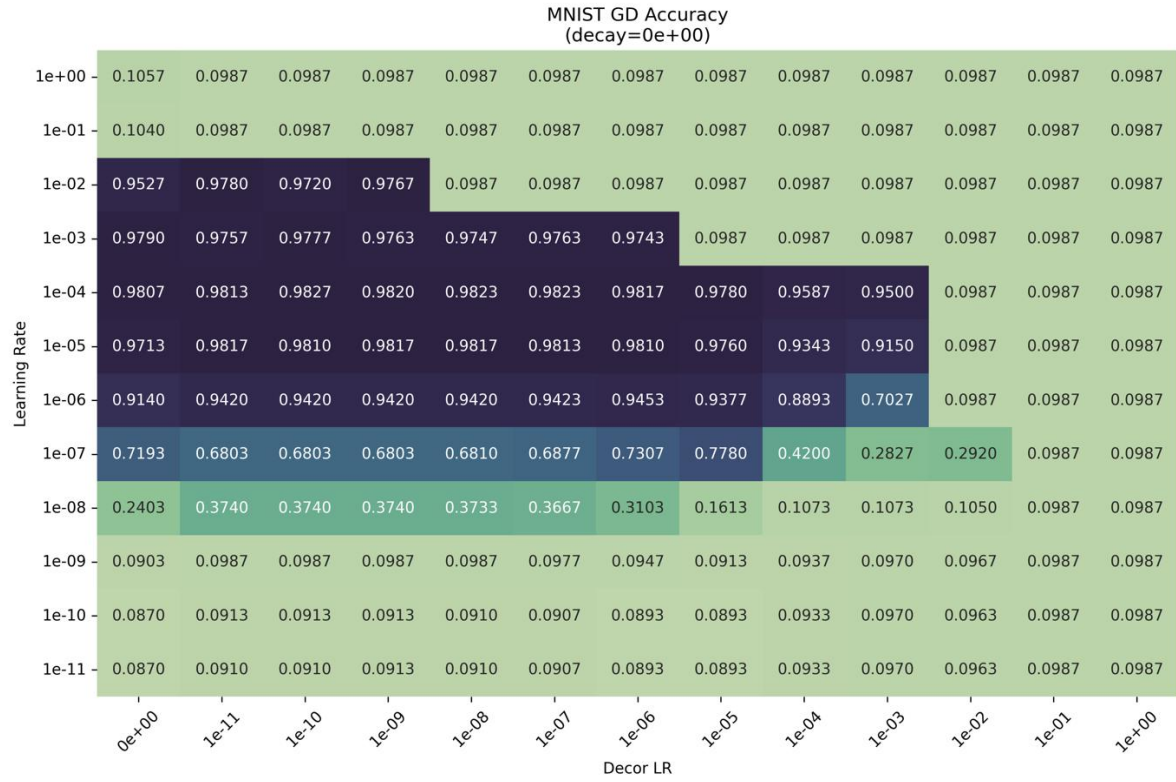
Appendix B

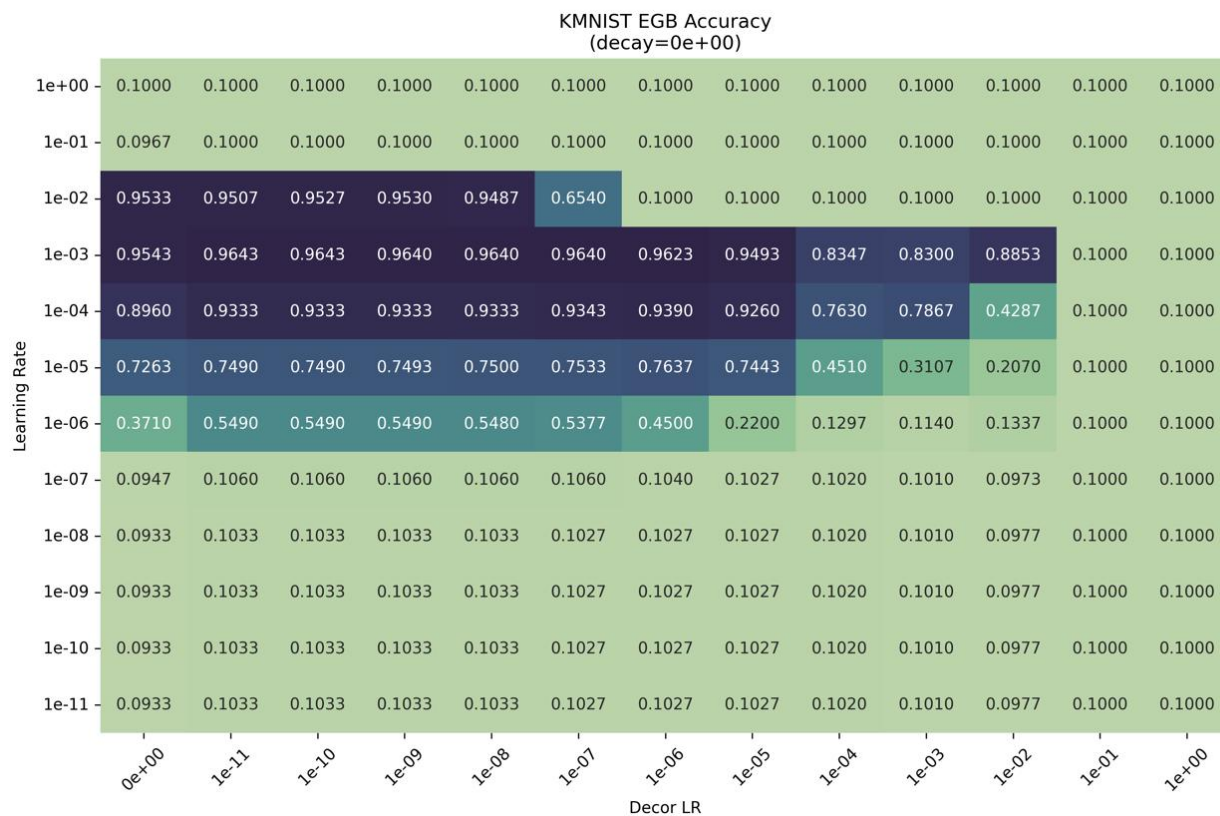
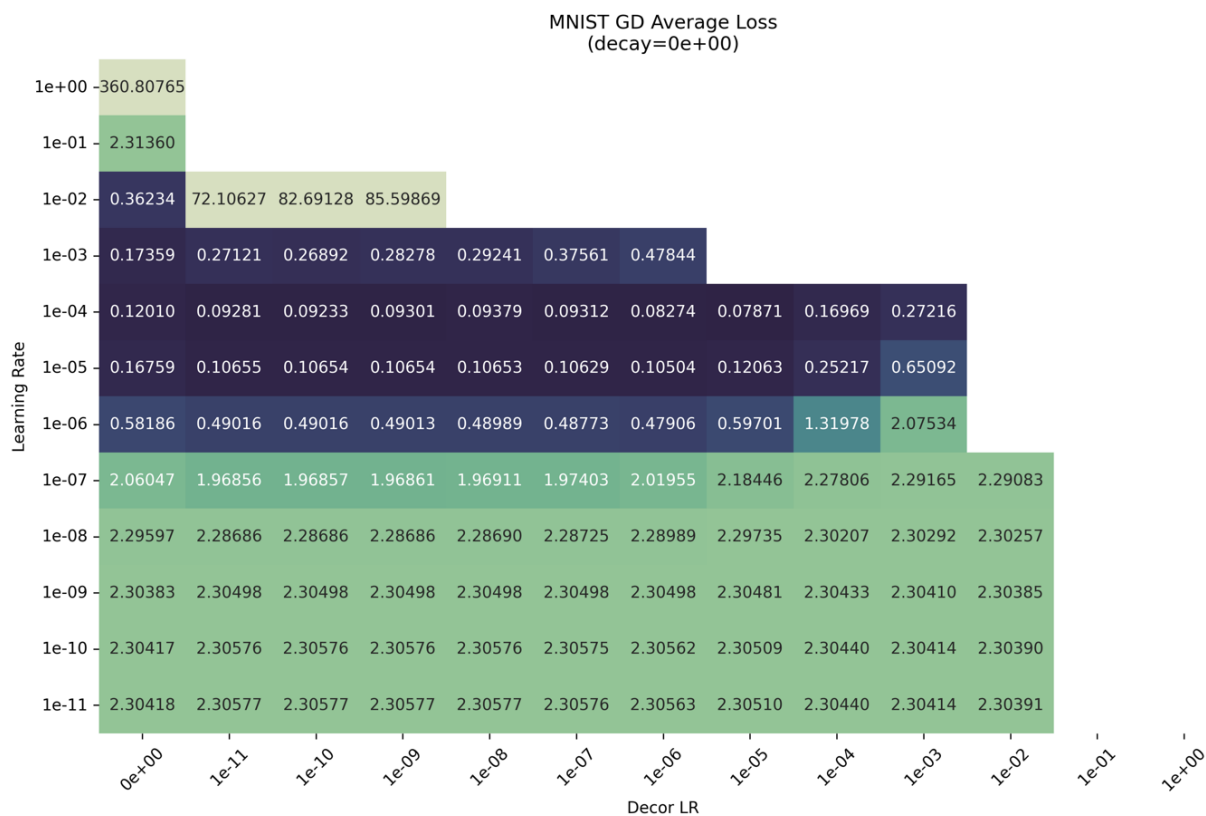


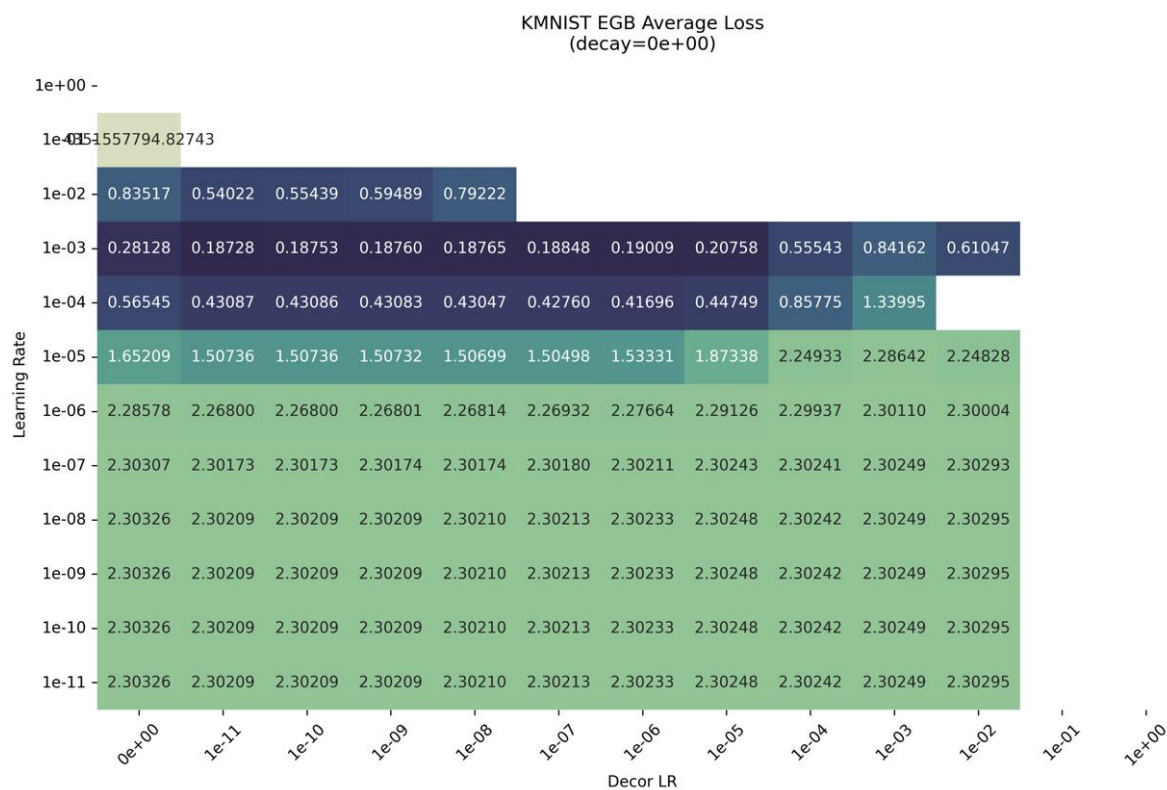
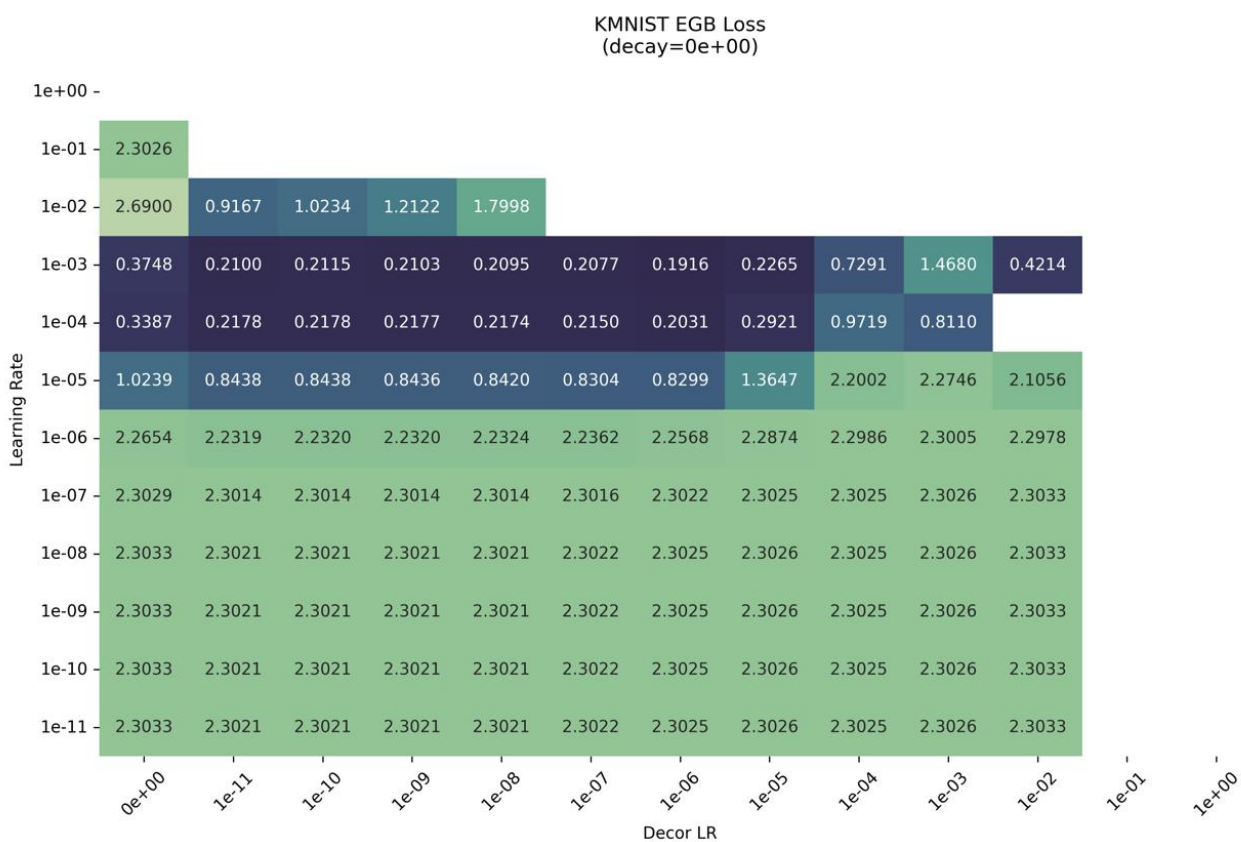


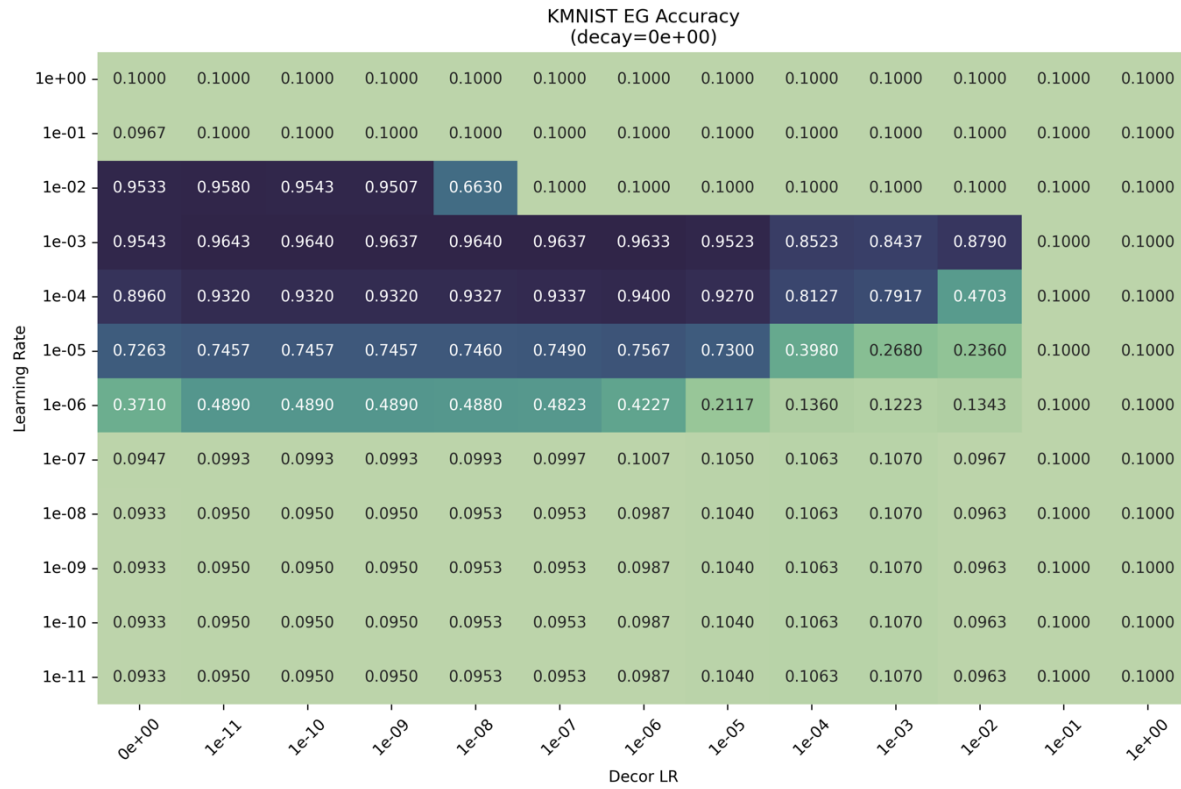


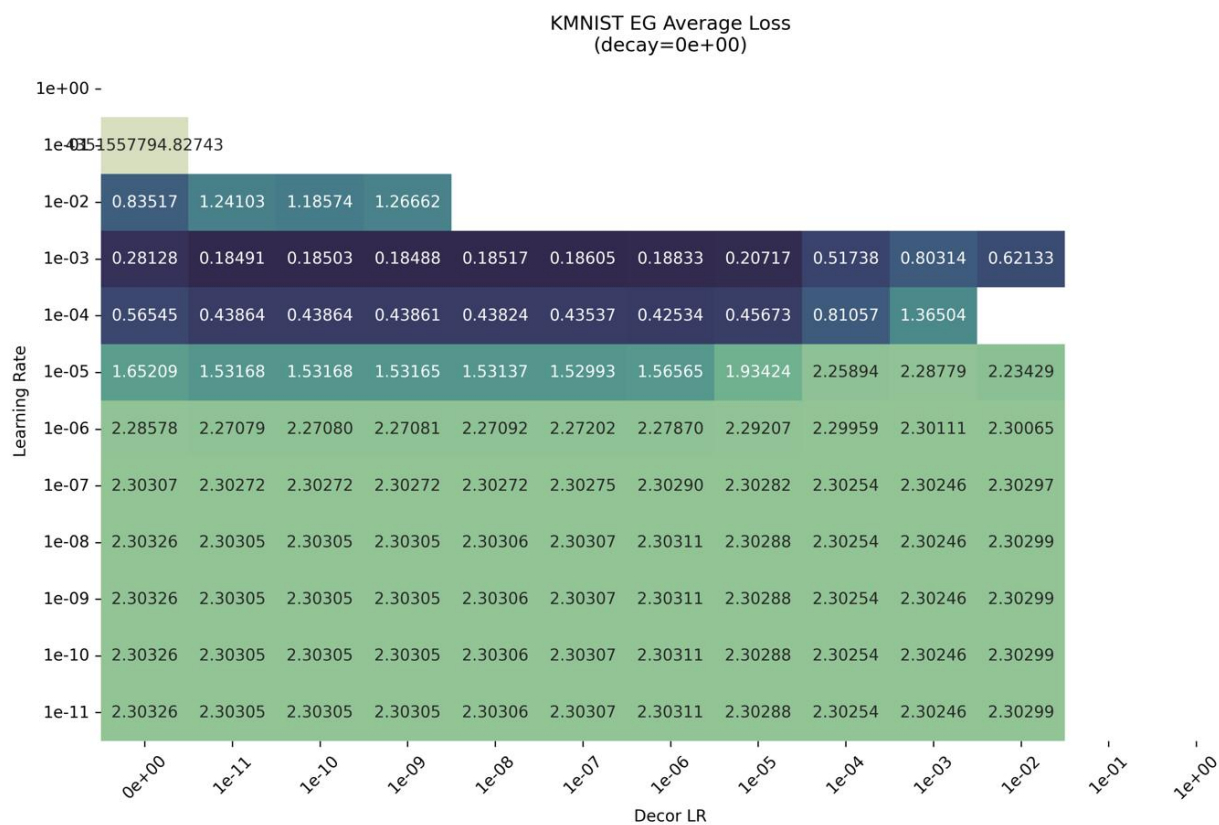
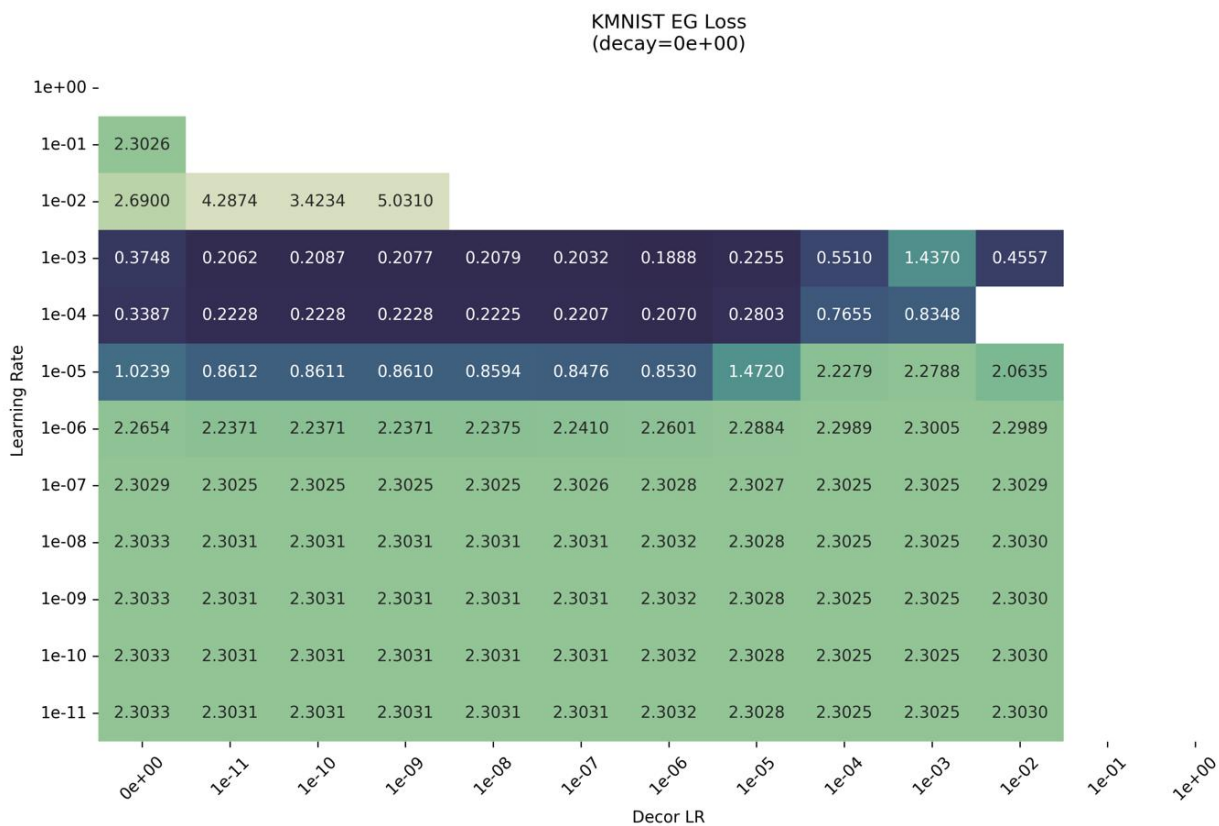


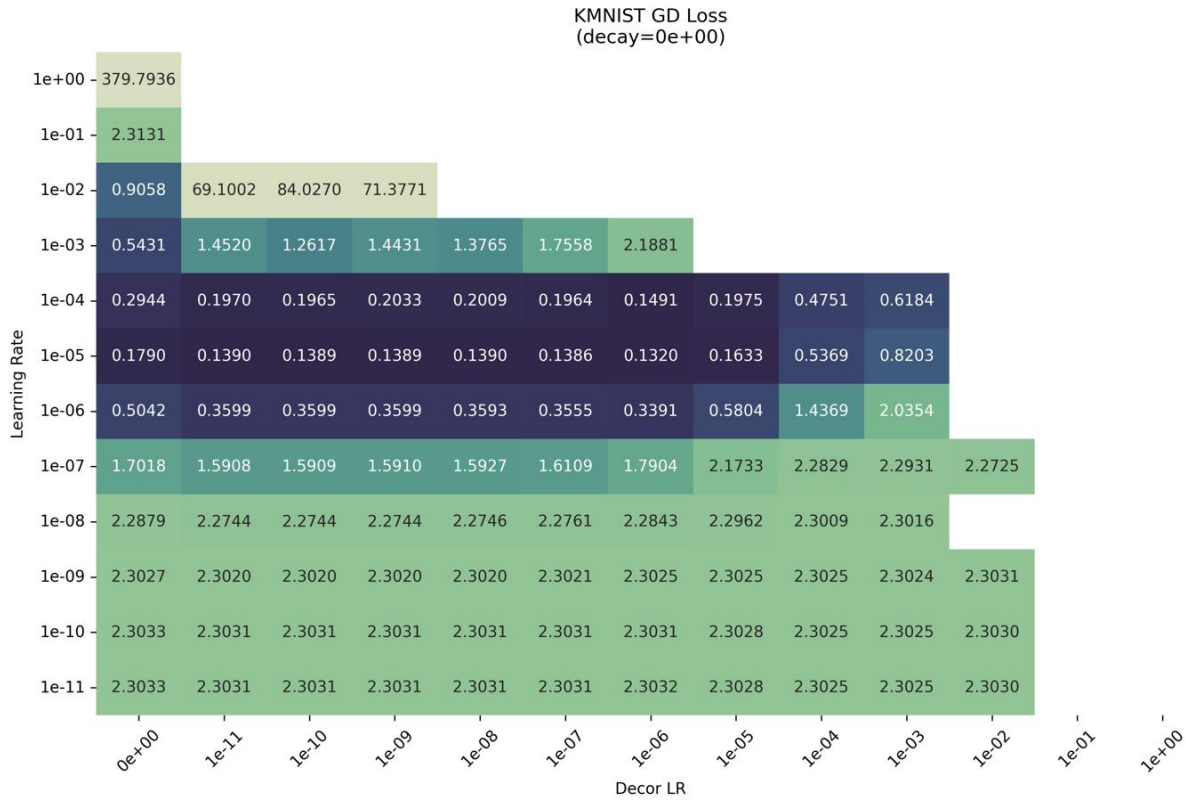
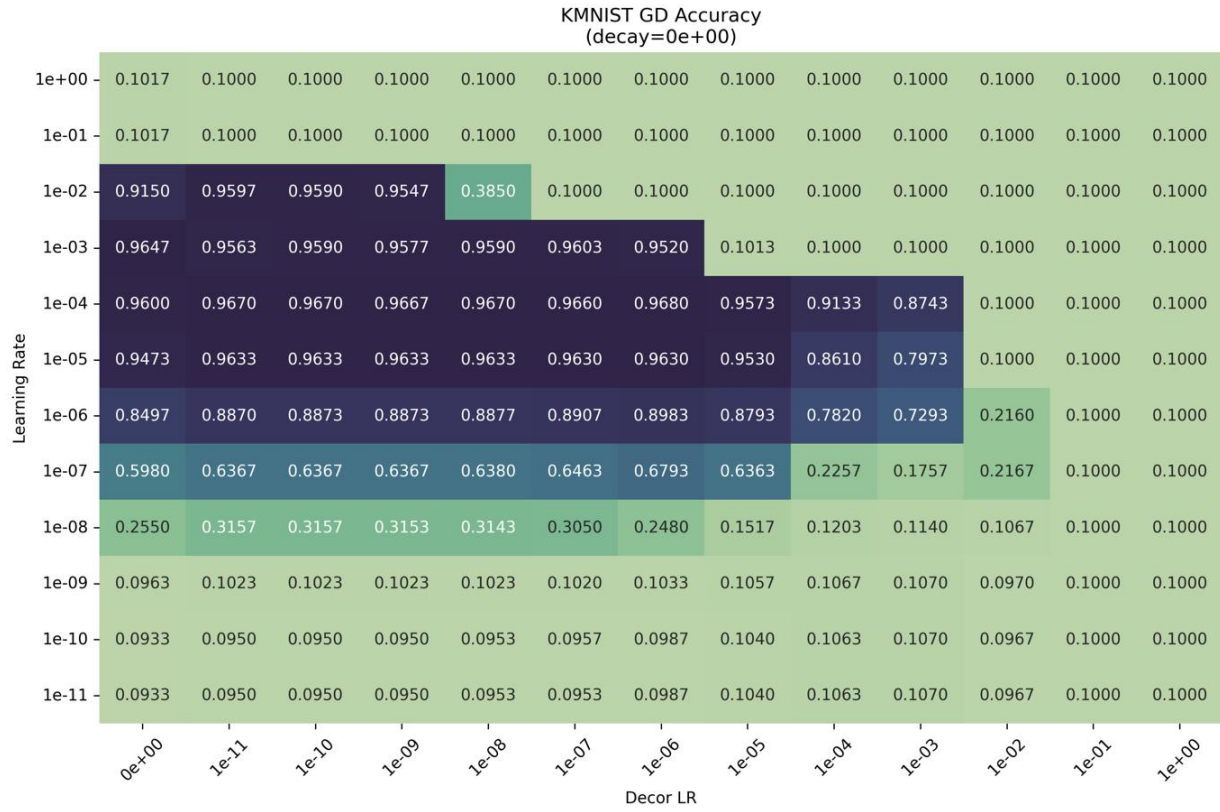


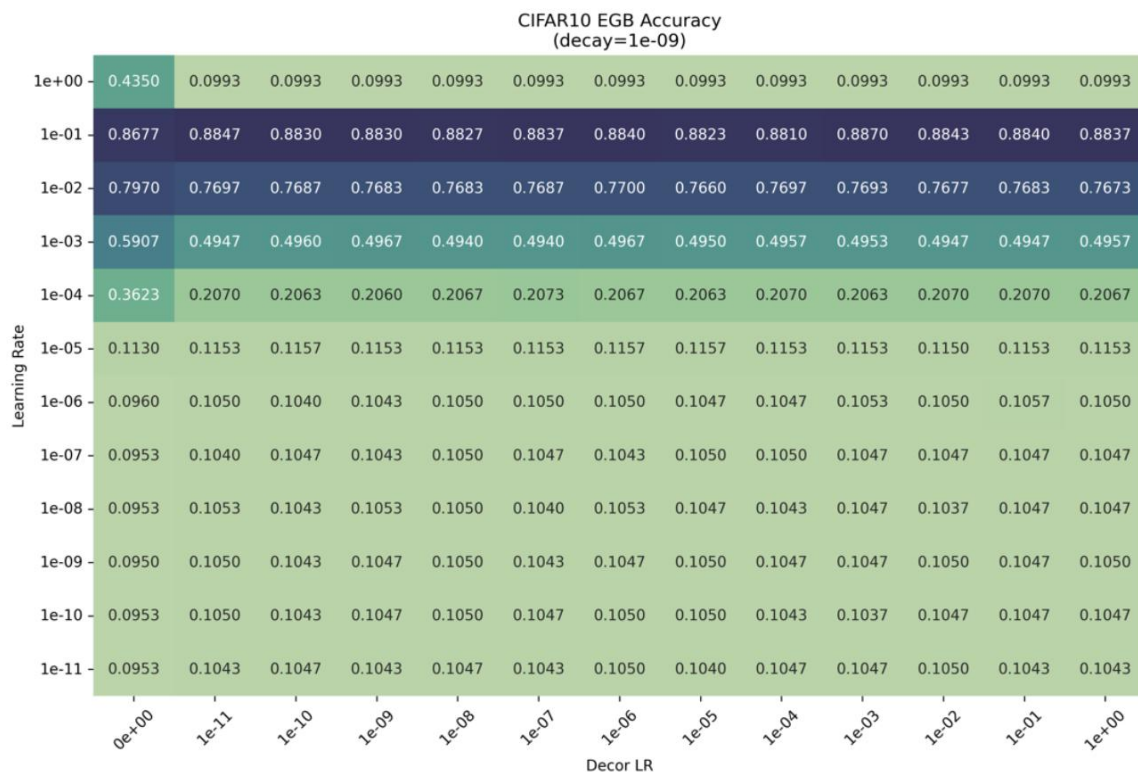
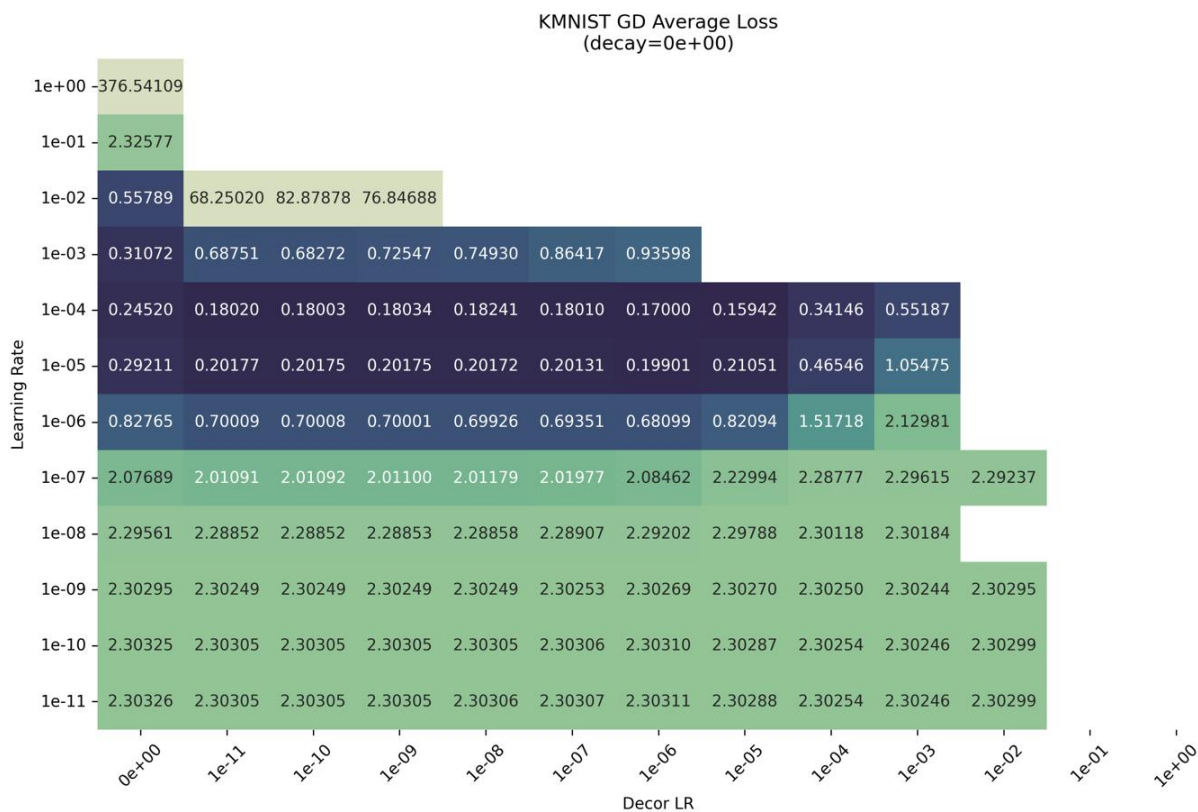












CIFAR10 EGB Loss
(decay=1e-09)

[illegible]

CIFAR10 EGB Average Loss
(decay=1e-07)

Learning Rate	1e+00	(Learning Rate = 1.0)												
	1e-01	0.97038	0.91044	0.90730	0.91116	0.91552	0.90984	0.90275	0.90461	0.91081	0.91133	0.90697	0.90426	0.91178
	1e-02	1.14574	1.21773	1.21602	1.21773	1.21888	1.21473	1.21680	1.22074	1.21605	1.21481	1.21587	1.21633	1.21660
	1e-03	1.57825	1.74790	1.74761	1.74820	1.74852	1.74793	1.74792	1.74877	1.74834	1.74762	1.74917	1.74832	1.74868
	1e-04	2.23027	2.33805	2.33826	2.33781	2.33782	2.33779	2.33838	2.33798	2.33827	2.33809	2.33818	2.33805	2.33773
	1e-05	3.22777	2.57491	2.57451	2.57489	2.57494	2.57439	2.57429	2.57453	2.57477	2.57471	2.57460	2.57488	2.57422
	1e-06	3.47206	2.60941	2.61005	2.60995	2.60915	2.60896	2.60967	2.60931	2.61004	2.61027	2.60941	2.60975	2.61004
	1e-07	3.49637	2.61035	2.61056	2.61059	2.61004	2.61076	2.61067	2.61054	2.61076	2.61047	2.61044	2.61057	2.61086
	1e-08	3.49647	2.61060	2.61063	2.61054	2.61068	2.61050	2.61048	2.61047	2.61082	2.61072	2.61024	2.61044	2.61012
	1e-09	3.49637	2.61084	2.61041	2.61084	2.61044	2.61091	2.61120	2.61047	2.61071	2.61075	2.61120	2.61032	2.61033
	1e-10	3.49657	2.61038	2.61062	2.61066	2.61044	2.61095	2.61042	2.61016	2.61081	2.61051	2.60993	2.61095	2.61066
	1e-11	3.49665	2.61080	2.61100	2.61064	2.61075	2.61084	2.61086	2.61043	2.61047	2.61016	2.61081	2.61048	2.61071
	0e+00	1e-11	1e-10	1e-09	1e-08	1e-07	1e-06	1e-05	1e-04	1e-03	1e-02	1e-01	1e+00	
		Decor LR												

Learning Rate	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR	Decor LR
1e+00	0.4480	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993
1e-01	0.8727	0.8833	0.8847	0.8813	0.8850	0.8837	0.8840	0.8850	0.8823	0.8833	0.8807	0.8837	0.8820
1e-02	0.7987	0.7673	0.7683	0.7700	0.7670	0.7663	0.7663	0.7670	0.7693	0.7663	0.7683	0.7707	0.7687
1e-03	0.5897	0.4960	0.4947	0.4950	0.4963	0.4937	0.4947	0.4947	0.4943	0.4960	0.4943	0.4967	0.4950
1e-04	0.3627	0.2067	0.2073	0.2067	0.2067	0.2080	0.2063	0.2070	0.2077	0.2080	0.2063	0.2070	0.2060
1e-05	0.1133	0.1157	0.1150	0.1160	0.1157	0.1157	0.1150	0.1157	0.1157	0.1160	0.1153	0.1153	0.1160
1e-06	0.0963	0.1047	0.1050	0.1043	0.1050	0.1047	0.1047	0.1050	0.1043	0.1047	0.1057	0.1043	0.1047
1e-07	0.0950	0.1050	0.1050	0.1040	0.1043	0.1043	0.1060	0.1047	0.1047	0.1043	0.1043	0.1050	0.1040
1e-08	0.0953	0.1047	0.1050	0.1050	0.1053	0.1050	0.1050	0.1040	0.1043	0.1037	0.1047	0.1040	0.1050
1e-09	0.0953	0.1050	0.1043	0.1047	0.1047	0.1047	0.1050	0.1050	0.1047	0.1047	0.1040	0.1043	0.1043
1e-10	0.0953	0.1040	0.1040	0.1043	0.1050	0.1057	0.1050	0.1047	0.1047	0.1050	0.1043	0.1043	0.1047
1e-11	0.0953	0.1047	0.1050	0.1043	0.1047	0.1050	0.1043	0.1050	0.1050	0.1040	0.1043	0.1047	0.1040

CIFAR10 EG Loss
(decay=1e-10)

1e+00	1.6188												
1e-01	0.8157	0.7963	0.7953	0.7952	0.7951	0.7928	0.7945	0.7953	0.7971	0.7945	0.7951	0.7909	0.7889
1e-02	0.9980	1.0595	1.0593	1.0543	1.0614	1.0615	1.0562	1.0569	1.0569	1.0559	1.0584	1.0588	1.0554
1e-03	1.4170	1.6257	1.6256	1.6260	1.6252	1.6266	1.6255	1.6262	1.6264	1.6258	1.6252	1.6261	1.6268
1e-04	1.8884	2.2833	2.2824	2.2828	2.2828	2.2826	2.2815	2.2822	2.2835	2.2814	2.2834	2.2830	2.2817
1e-05	3.0181	2.5586	2.5577	2.5579	2.5604	2.5567	2.5622	2.5590	2.5585	2.5597	2.5577	2.5590	2.5576
1e-06	3.4505	2.6052	2.6033	2.6020	2.6016	2.6029	2.6003	2.6040	2.6021	2.6016	2.6011	2.6009	2.6032
1e-07	3.4945	2.6051	2.6021	2.6017	2.6030	2.6033	2.6006	2.6028	2.6025	2.6028	2.6028	2.6031	2.6036
1e-08	3.4954	2.5999	2.6040	2.6029	2.6021	2.6035	2.6053	2.6012	2.6022	2.6024	2.6011	2.6032	2.6038
1e-09	3.4941	2.6017	2.6045	2.6023	2.6008	2.6047	2.6013	2.5999	2.6037	2.6027	2.6028	2.6054	2.6038
1e-10	3.4944	2.6042	2.6045	2.6034	2.6037	2.6024	2.6033	2.6023	2.6037	2.6037	2.6042	2.6049	2.6038
1e-11	3.4939	2.6058	2.6032	2.6036	2.6015	2.6004	2.5985	2.6033	2.6033	2.6027	2.6012	2.6040	2.6000

Decor LR

CIFAR10 EG Average Loss
(decay=1e-07)

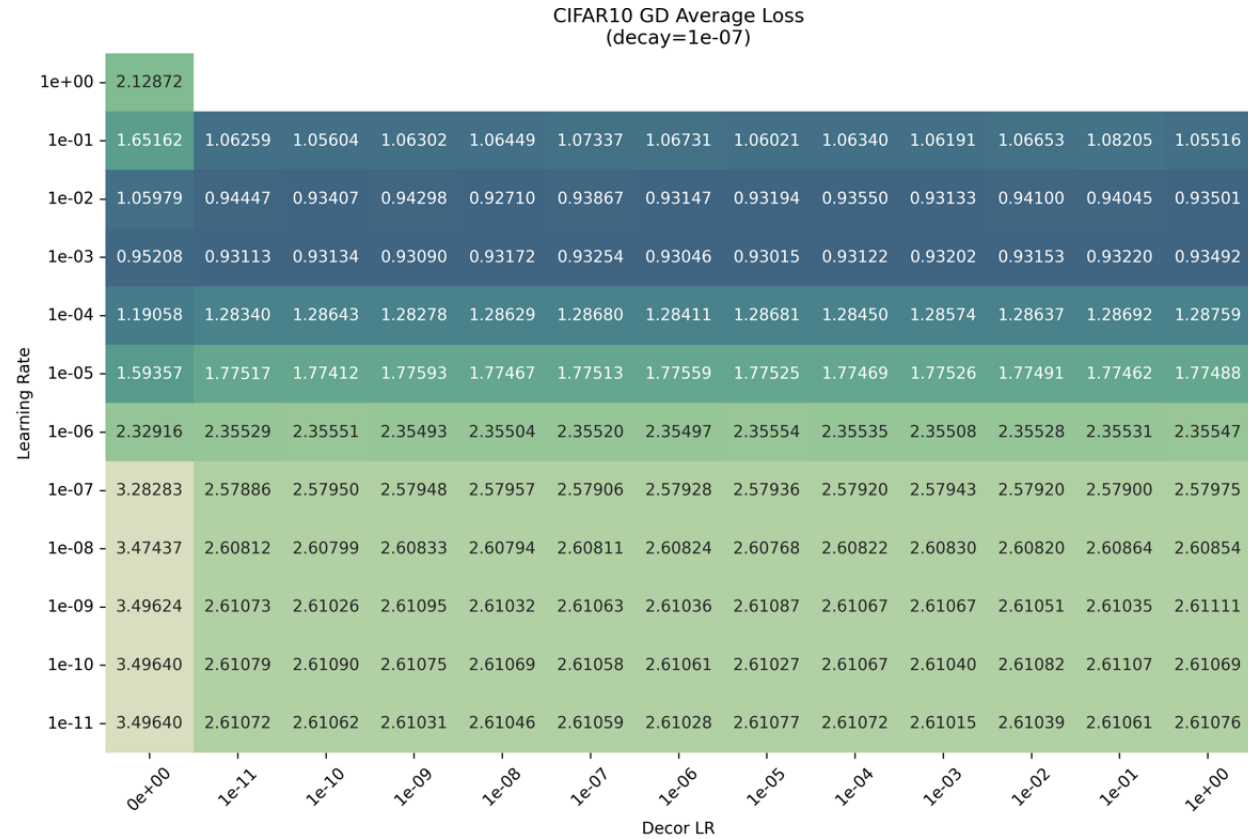
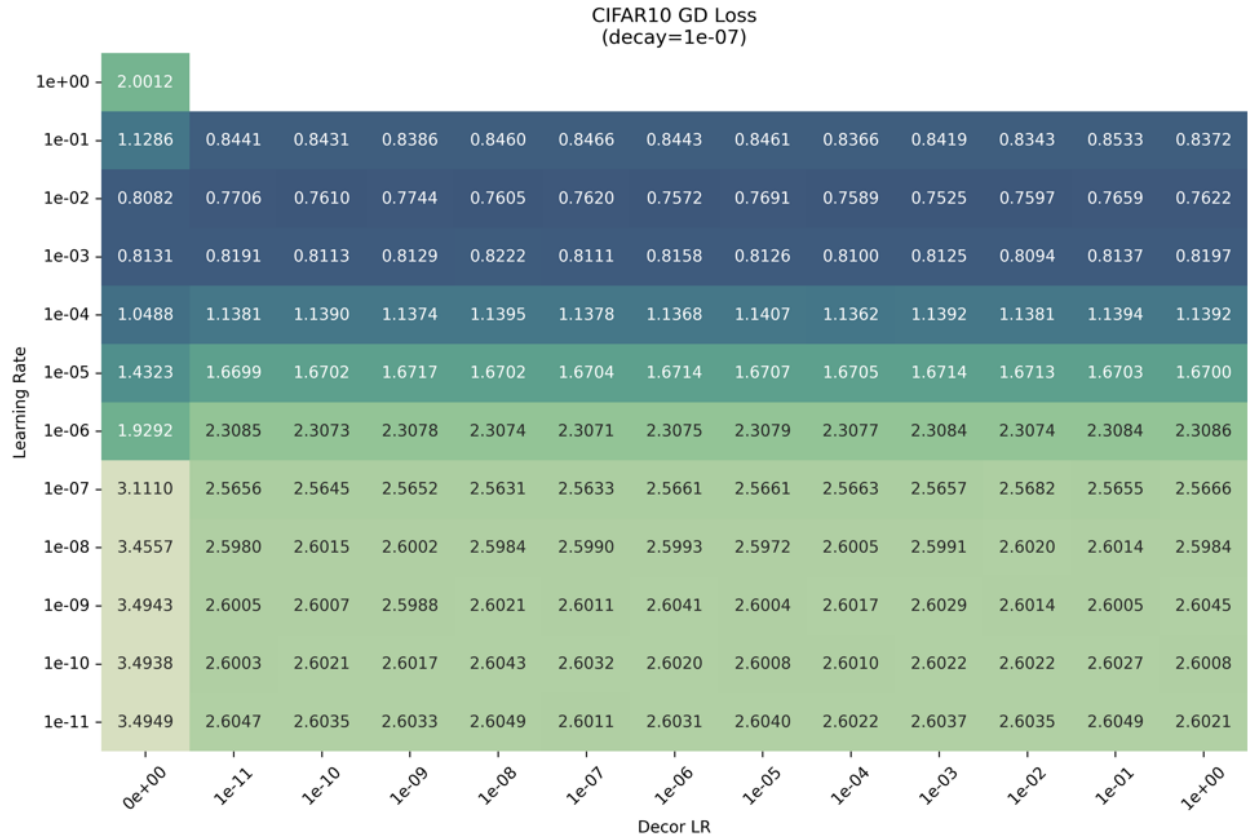
1e+00	1.88303												
1e-01	0.96838	0.91282	0.90741	0.90746	0.91440	0.90865	0.90325	0.90859	0.90538	0.90917	0.91385	0.91239	0.91209
1e-02	1.14542	1.21434	1.21343	1.21806	1.21269	1.21806	1.21689	1.21589	1.21619	1.21686	1.21691	1.21643	1.21542
1e-03	1.57874	1.74921	1.74911	1.74842	1.74846	1.74783	1.74939	1.74654	1.74910	1.74863	1.74806	1.74823	1.74722
1e-04	2.23030	2.33809	2.33802	2.33806	2.33788	2.33803	2.33791	2.33819	2.33785	2.33765	2.33796	2.33798	2.33797
1e-05	3.22816	2.57481	2.57479	2.57421	2.57459	2.57480	2.57488	2.57451	2.57450	2.57442	2.57387	2.57508	2.57465
1e-06	3.47214	2.60976	2.60948	2.60954	2.60976	2.61014	2.61033	2.60959	2.60946	2.60971	2.60943	2.60975	2.61044
1e-07	3.49650	2.61050	2.61026	2.61100	2.61057	2.61066	2.61084	2.61066	2.61061	2.61059	2.61090	2.61073	2.61082
1e-08	3.49651	2.61032	2.61072	2.61072	2.61027	2.61034	2.61074	2.61123	2.61035	2.61041	2.61076	2.61046	2.61076
1e-09	3.49650	2.61052	2.61071	2.61018	2.61083	2.61058	2.61079	2.61014	2.61065	2.61066	2.61061	2.61058	2.61044
1e-10	3.49663	2.61053	2.61040	2.61077	2.61066	2.61041	2.61042	2.61095	2.61077	2.61053	2.61006	2.61127	2.61031
1e-11	3.49637	2.61047	2.61076	2.61026	2.61088	2.61062	2.61124	2.61059	2.61077	2.61063	2.61019	2.61056	2.61056
	0e+00	1e-11	1e-10	1e-09	1e-08	1e-07	1e-06	1e-05	1e-04	1e-03	1e-02	1e-01	1e+00

Decor LR

CIFAR10 GD Accuracy
(decay=1e-07)

1e+00	0.2393	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993	0.0993
1e-01	0.7307	0.8630	0.8640	0.8627	0.8627	0.8610	0.8630	0.8623	0.8643	0.8623	0.8633	0.8560	0.8650
1e-02	0.8753	0.8920	0.8963	0.8907	0.8950	0.8950	0.8957	0.8930	0.8947	0.8983	0.8940	0.8927	0.8943
1e-03	0.8733	0.8737	0.8767	0.8763	0.8717	0.8767	0.8767	0.8757	0.8760	0.8760	0.8767	0.8743	0.8743
1e-04	0.7760	0.7353	0.7353	0.7360	0.7343	0.7357	0.7360	0.7327	0.7370	0.7360	0.7363	0.7350	0.7327
1e-05	0.5820	0.4843	0.4853	0.4853	0.4847	0.4837	0.4847	0.4827	0.4843	0.4843	0.4843	0.4850	0.4853
1e-06	0.3447	0.1997	0.2000	0.2000	0.2003	0.1993	0.1997	0.2000	0.1997	0.2003	0.2003	0.2003	0.2003
1e-07	0.1123	0.1137	0.1133	0.1137	0.1140	0.1140	0.1130	0.1137	0.1133	0.1140	0.1133	0.1140	0.1137
1e-08	0.0970	0.1050	0.1050	0.1053	0.1050	0.1053	0.1050	0.1053	0.1057	0.1057	0.1053	0.1057	0.1060
1e-09	0.0953	0.1037	0.1047	0.1047	0.1047	0.1047	0.1050	0.1037	0.1053	0.1050	0.1043	0.1047	0.1050
1e-10	0.0953	0.1050	0.1040	0.1047	0.1043	0.1047	0.1050	0.1043	0.1043	0.1047	0.1057	0.1043	0.1050
1e-11	0.0953	0.1043	0.1043	0.1050	0.1043	0.1050	0.1050	0.1047	0.1043	0.1050	0.1043	0.1050	0.1047
	0e+00	1e-11	1e-10	1e-09	1e-08	1e-07	1e-06	1e-05	1e-04	1e-03	1e-02	1e-01	1e+00

Decor LR



Appendix C

Decorrelation Best Accuracy	No Decorrelation Best Accuracy	Dataset	Method	% Difference
98.33%	98.20%	MNIST	GD	0.1%
98.17%	97.77%	MNIST	EG	0.4%
98.07%	97.77%	MNIST	EGB	0.4%
91.87%	91.13%	KMNIST	GD	0.8%
91.13%	89.23%	KMNIST	EG	2%
91.13%	89.23%	KMNIST	EGB	2%
89.17%	87.70%	CIFAR10	GD	2%
87.83%	87.03%	CIFAR10	EG	0.9%
88.13%	86.67%	CIFAR10	EGB	2%

Decorrelation Best Loss	No Decorrelation Best Loss	Dataset	Method	% Difference
0.0650	0.0848	MNIST	GD	30%
0.0953	0.1521	MNIST	EG	60%
0.0999	0.1521	MNIST	EGB	52%
0.3550	0.4483	KMNIST	GD	26%
0.5334	0.6767	KMNIST	EG	27%
0.5476	0.6767	KMNIST	EGB	24%
0.7807	0.7995	CIFAR10	GD	2%
0.8022	0.8301	CIFAR10	EG	3%
0.8048	0.8225	CIFAR10	EGB	2%

Decorrelation Best Avg Loss	No Decorrelation Best Avg Loss	Dataset	Method	% Difference
0.0762	0.01178	MNIST	GD	55%
0.0859	0.1217	MNIST	EG	42%
0.0858	0.1217	MNIST	EGB	42%
0.4440	0.6680	KMNIST	GD	50%
0.4889	0.7011	KMNIST	EG	43%
0.4963	0.7011	KMNIST	EGB	41%
0.9416	0.9582	CIFAR10	GD	2%
0.914	0.9692	CIFAR10	EG	6%
0.9170	0.9738	CIFAR10	EGB	6%

Method	Accuracy%	Accuracy (% difference vs best)	Loss	Loss (% difference vs best)	Average loss	Average Loss (% difference vs best)	Dataset
GD	98.33%	0%	0.0650	0%	0.0762	0%	MNIST
EG	98.17%	0.2%	0.0953	47%	0.0859	13%	MNIST
EGB	98.07%	0.3%	0.0999	54%	0.0858	12%	MNIST

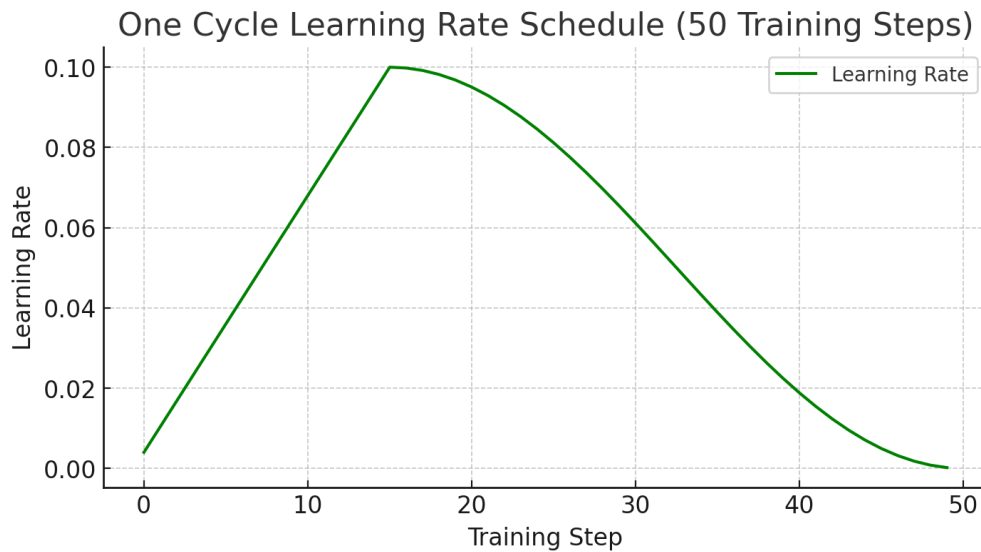
Method	Accuracy%	Accuracy (% difference vs best)	Loss	Loss (% difference vs best)	Average loss	Average Loss (% difference vs best)	Dataset
GD	91.87%	0%	0.3550	0%	0.4440	0%	KMNIST
EG	91.13%	0.8%	0.5334	50%	0.4889	10%	KMNIST
EGB	91.13%	0.8%	0.5476	54%	0.4963	12%	KMNIST

Method	Accuracy%	Accuracy (% difference vs best)	Loss	Loss (% difference vs best)	Average loss	Average Loss (% difference vs best)	Dataset
GD	89.17%	0%	0.7807	0%	0.9416	3%	CIFAR10
EG	87.83%	2%	0.8022	3%	0.9124	0%	CIFAR10
EGB	88.13%	1%	0.8048	3%	0.9170	3%	CIFAR10

Appendix D

Notable Findings

Initially, gradient clipping was performed in the training loop when running tests on the CNN model. This was due to the fact that gradient clipping has been shown to provide slightly higher accuracies when used in conjunction with ResNet9 architecture. This holds true when running configurations that do not involve decorrelation layers. However, when decorrelation layers are in the picture, the entire model refuses to converge, often floating around an accuracy of only 10% and eventually diverging completely. The second finding involves the choice of optimizers. When looking at existing examples, a OneCycleLR was often used. However, in the context of this experiment, a OneCycleLR does not produce the best results when a hyperparameter sweep is considered (Pytorch, n.d). This is due to the fact that a OneCycleLR learning rate slope will look like this:



This results in there being virtually no difference when setting learning rates, and it was proven in the tests when the accuracies across all learning rates were nearly identical.